



Election Administrator Manual

Version 10.0

Generated 2026-10-08 · 576e740

Contents

1. Election Administrator Manual
2. 00-START: Before you start
3. 01-TENANT: Set up the tenant
4. 02-EVENT: Create the election event
5. 03-KEYS: Run the key ceremony
6. 04-PUBLISH: Publish the ballot and manage the voting period
7. 05-TALLY: Run the tally ceremony
8. 06-RESULTS: Get the results
9. Configure Password Policy
10. Create Voters
11. Import Voter List
12. Voter Communication
13. Reports and Templates
14. Export Data
15. Ballot Boxes (Tally Sheets)
16. Results Website
17. Support Materials
18. Create and set google meet credentials
19. Adding User Attributes to Keycloak
20. Linking Multiple IdP Identities to a Single User via Custom Attribute
21. Logging In Without a Username (Attribute + Password or Secret)
22. Basic Navigation
23. Election Event Dashboard Overview
24. Data
25. Localization
26. Voters
27. Areas
28. Keys
29. Tally Sheet Imports
30. Publish
31. Tasks
32. Logs
33. Scheduled Events
34. Certificates
35. Tally Ceremony
36. Transmission
37. Reports
38. Templates

- 39. Dashboard
- 40. Data
- 41. Voter Assignment in an Election
- 42. Publish
- 43. Contest Data Tab
- 44. Data
- 45. Languages
- 46. Decline to Vote
- 47. Styling Ballot Errors and Warnings with Custom CSS
- 48. Voting Portal Date & Time Format
- 49. Blank Ballots
- 50. Voting Portal CSS hooks
- 51. Configuring Login and Registration Fields
- 52. Acclaimed Contests
- 53. Integrations
- 54. Languages
- 55. Localization
- 56. External Previews
- 57. Templates
- 58. Permissions

Election Administrator Manual

This manual tells election administrators and trustees how to prepare, run and close an election with the Sequent Online Voting admin portal, version **10.0**.

[Download this manual as PDF](#)

How to use this manual

1. Read [00-START](#) first. It gives the roles, the terms and the safety rules.
2. Do the procedures in the order of the table. Each procedure tells you what you must have before you start.
3. Use the [More procedures](#) part for optional features, for example voter import, voter communication and login methods.
4. Use the [Reference](#) part when you need the details of a screen.

Procedures

Code	Procedure	Who	Result
00-START	Before you start: roles, terms and safety rules	All	You know who does each task.
01-TENANT	Set up the tenant: settings, users, roles and trustees	Tenant administrator	The administrators and trustees can sign in.
02-EVENT	Create the election event: elections, contests, candidates, areas and voters	Election administrator	The ballot content and the voter list are complete.

Code	Procedure	Who	Result
03-KEYS	Run the key ceremony	Election administrator and trustees	The election public key exists. Each trustee keeps a key fragment.
04-PUBLISH	Publish the ballot and open, pause and close the voting period	Election administrator	Voters can vote. Then the voting period is closed.
05-TALLY	Run the tally ceremony	Election administrator and trustees	The votes are decrypted and counted.
06-RESULTS	Get the results, the reports, and publish the results website	Election administrator	You have the result documents. The public can see the results.

Versions and languages

Use the version menu at the top of the page to select the manual for your version of the platform. If you do not know the version of your platform, ask Sequent support.

Use the language menu to select a language. A page that is not translated shows in English.

00-START: Before you start

Read this page before you do a procedure. It gives the order of the work, the roles, the terms and the rules that keep the election secure.

The order of the work



Do the procedures in this order. You can change the ballot content after the key ceremony, but you must publish again after each change.

Roles

Role	Tasks	Default role in the admin portal
Sequent	Creates the tenant. Installs the trustee services. Gives you the first administrator account and the admin portal address.	-
Tenant administrator	Sets up the tenant. Creates the accounts of the other users.	admin
Election administrator	Creates and configures the election event. Starts the ceremonies. Publishes the ballot. Opens and closes the voting period. Gets the results.	admin
Trustee	Keeps one fragment of the election private key. Takes part in the key ceremony and in the tally ceremony.	trustee

One person can have more than one role. A trustee must not be the only person who controls the other trustees.

i NOTE

Your organization can change the default roles. If a button or a tab in this manual is not on your screen, your account does not have the necessary permission. Ask your tenant administrator.

Terms

Term	Meaning
Tenant	The space of your organization on the platform. It contains your settings, your users and your election events.
Election event	The complete election process, from the configuration to the results. It contains one or more elections.
Election	One vote inside an election event. It contains one or more contests.
Contest	One question or one race on the ballot. It contains the candidates.
Candidate	One option that a voter can select in a contest.
Area	A group of voters, for example a district. An area tells which contests a voter sees.
Voter	A person who can vote in the election event. Each voter belongs to one area.
Trustee	A person who keeps one fragment of the election private key.
Threshold	The minimum number of trustees that must take part in the tally ceremony.

Term	Meaning
Key ceremony	The procedure that makes the election keys. Voters encrypt their ballots with the public key.
Key fragment	The part of the private key that one trustee keeps. In the admin portal it is the Encrypted Private Key file.
Publication	A version of the ballot that the voting portal shows to voters.
Voting channel	A way to vote: Online (voting portal), Kiosk (voting kiosk), Early voting or Telephone voting . Each channel has its own voting status.
Tally ceremony	The procedure that decrypts and counts the votes.
Automatic ceremony	A key ceremony and a tally ceremony without trustee actions. The election event must permit it.
Results website	The public website that shows the results that you publish.
Sensitive action	An action for which the admin portal asks for your password again.

What you need

- A computer with an up-to-date version of Chrome, Firefox, Edge or Safari.
- The address of the admin portal. Sequent gives it to you.
- Your user name and password. The tenant administrator gives them to you.
- For trustees: two storage devices for the backups of the key fragment, for example two USB flash drives. Use devices that only you control.

Sign in to the admin portal

1. Open the admin portal address in the browser.

2. If the **Select Tenant** page opens, type the **Tenant Name** and click **Continue**. Sequent gives you the tenant name.
3. Type your user name and your password.
4. Click the sign-in button.
5. If the admin portal asks for a second factor, do the steps on the screen.

Expected result: the admin portal opens. The menu on the left shows your tenant and the items that your role permits:

Menu item	Use
Tenant name, with a + icon	Shows your tenant. Only Sequent can use the + icon.
Election Events	The list of election events. Click + to create or import an election event. Use Active and Archived to filter the list.
Users and Roles	The accounts of the administrators and trustees, and the roles.
Settings	The configuration of the tenant.
Templates	The templates of the messages and documents.
Help	Help links. This item shows only if your tenant has help links.

Under each election event, the menu shows a tree: election event, elections, contests and candidates. Click the three dots next to an item to see the actions for it.

Sensitive actions

Some actions need a recent confirmation of your identity, for example to publish the ballot or to open the voting period. The confirmation is valid for 60 seconds after you sign in. When you do one of these actions, the admin portal shows: "The action you are about to perform is sensitive and requires confirmation."

1. Click **Confirm**.
2. Type your password on the sign-in page.

Expected result: the admin portal opens again and continues the action.

Safety rules

WARNING

Keep the key fragments secret and safe. If fewer trustees than the threshold have their key fragment at the tally ceremony, nobody can decrypt the votes. Nobody, Sequent included, can make a key fragment again.

- Each trustee keeps their own key fragment. Do not send a key fragment by email or chat.
- Do not give your password to another person. Each person uses their own account.
- Do the procedures from a trusted computer and network.
- Record the date, the time and the persons present at each ceremony. The admin portal also records the actions in the **Logs** tab of the election event.

Conventions in this manual

Convention	Meaning
Bold	A label on the screen: a button, a tab, a field or a menu item. The text is the same as on the screen.
Election Events > Data	Go to the first item, then to the second.
Expected result	What you see when the step is correct.
WARNING	A risk to the secrecy, the integrity or the availability of the election.
CAUTION	A risk of data loss, or an action that you cannot reverse.

Next: [01-TENANT: Set up the tenant.](#)

01-TENANT: Set up the tenant

This procedure prepares the tenant for your election events. You set the tenant settings, you create the accounts of the administrators and trustees, and you make a backup of the configuration. The tenant administrator does this procedure once, before the first election event.

Before you start

- Sequent has created your tenant and has given you an account with the `admin` role.
- Sequent has installed the trustee services and has added the trustees to **Settings > TRUSTEES**.
- You have the list of persons: the administrators and the trustees, with their names and email addresses.
- For each trustee person, you know which trustee of the list they represent.

NOTE

Only Sequent can create a tenant. The + icon next to the tenant name does not work for other users.

1. Set the tenant settings

1. In the menu on the left, click **Settings**.
2. Click each tab and set the values. The table tells what each tab controls.

Tab	What it controls
ELECTION TYPES	Not in use. You do not have to set it.

Tab	What it controls
VOTING CHANNELS	The voting channels that the tenant can use: Online Voting , Kiosk Voting and Telephone Voting . The admin portal saves a change immediately.
TEMPLATES	Not in use. The switches cannot be changed.
LANGUAGES	The languages that election events can use, the Default Language and the Language Detection Policy .
LOCALIZATION	Changes to the texts of the portals, for each language. Select the language, click Add , then type the Key and the Value .
Integrations	The Google Calendar account that makes video meeting links. See More procedures .
Look & Feel	The Logo URL , the Custom CSS and the Help Links of the tenant. The help links show in the Help menu.
TRUSTEES	The list of trustees. Sequent adds the trustees. Do not change this list.
Countries	The countries from which voters cannot vote or enroll.
Backup / Restore	The backup and the restore of the tenant configuration. See step 4 .
Previews	A record of the ballot previews that external systems requested. You cannot change it.

- On the **LANGUAGES** tab, turn on each language that your election events will use. Select the **Default Language**. In **Language Detection Policy**, select **Browser Detect** to show the language of the voter's browser, or **Force Default** to always show the default language.
- On the **VOTING CHANNELS** tab, turn on each voting channel that your election events will use.
- On the **TRUSTEES** tab, make sure that the list contains all the trustees.

Expected result: the settings show the values that you set. The **TRUSTEES** tab shows one row for each trustee.

 NOTE

A new election event copies the language settings of the tenant when you create it. A change on the **LANGUAGES** tab does not change the election events that exist.

2. Check the roles

A role is a set of permissions. Each user gets one or more roles.

1. In the menu on the left, click **Users and Roles**.
2. Click the **Roles** tab.
3. Make sure that the list contains the roles that you need, for example `admin` and `trustee`.
4. To see the permissions of a role, click the role. Each permission has a switch.

 CAUTION

A change to a role applies immediately to all users with that role. Do not remove a permission from the `admin` role. If you do, you can lose access to a part of the admin portal.

To create a role:

1. On the **Roles** tab, click the create button.
2. Type the **Name** of the role.
3. Save the role.
4. Click the new role and turn on its permissions.

Expected result: the **Roles** tab shows the roles with the permissions that you need.

3. Create the user accounts

Do these steps for each administrator and for each trustee.

1. In the menu on the left, click **Users and Roles**.
2. Click the **Users** tab.
3. Click the create button. If the list is empty, click **Create user**.
4. Type the user name and the other fields of the form, for example the email address and the name.
5. Select **Enabled**.
6. Type a password in **Password:** and type it again in **Repeat Password:**.
7. Turn on **Temporary**. The user must then change the password at the first sign-in.
8. For a trustee only: in **Act as Trustee**, select the trustee that this person represents.
9. In the role list of the form, select each role of the user:
 - Administrators: `admin`.
 - Trustees: `trustee`.
10. Save the user.
11. Give the user name and the temporary password to the person. Use a secure channel.

WARNING

Give each trustee their own account and their own trustee in **Act as Trustee**. Two persons must not share one trustee. If one person controls more trustees than the threshold, that person can decrypt the votes alone.

Expected result: the **Users** tab shows the new users. Each user can sign in, changes the password and sees the menu items of their role.

4. Make a backup of the tenant configuration

Make a backup after you set up the tenant and after each important change.

1. In the menu on the left, click **Settings**.
2. Click the **Backup / Restore** tab.
3. Click **Backup**.
4. Wait for the task to finish. The browser downloads a file with the name `tenant-config-<tenant>-export.zip`.
5. Keep the file in a safe location. It contains the configuration of your tenant.

Expected result: you have the backup file.

⚠ CAUTION

A restore replaces the tenant configuration, the sign-in configuration and the roles. Do a restore only with the help of Sequent support.

If there is a problem

Problem	Action
The message "You don't have permission to access settings." shows.	Your account does not have the necessary permissions. Ask Sequent support to give your account the <code>admin</code> role.
The TRUSTEES tab is empty or a trustee is missing.	Ask Sequent support to add the trustees. You cannot do the key ceremony without them.
Act as Trustee is not in the user form.	The user profile of the tenant does not have the trustee field. Ask Sequent support.
The admin portal shows Review changes when you save a user.	Read the changes, then click Confirm changes .
A user cannot sign in.	Make sure that Enabled is selected for the user. On the Users tab, use Change password to set a new temporary password.

Next: 02-EVENT: Create the election event.

02-EVENT: Create the election event

This procedure creates the election event and its content: the elections, the contests, the candidates, the areas and the voters. The election administrator does this procedure.

Before you start

- The tenant is ready. See [01-TENANT](#).
- You have the content of the ballot: the elections, the contests and the candidates, in all the languages of the election.
- You have the list of areas and the contests of each area.
- You have the voter list as a CSV file, or the data to create each voter.
- If you import an election event: you have the file and its SHA-256 hash.

The order of the steps is important. Create the areas after the contests, and the voters after the areas.

1. Create the election event

1. In the menu on the left, click the + icon next to **Election Events**.
2. Click **Create an Election Event**.
3. Type the **Name** of the election event.
4. Type the **Description**.
5. Click the save button.

Expected result: the message "Election Event created" shows. The admin portal opens the new election event. The election event shows these tabs. You see only the tabs that your role permits:

Tab	Use
Dashboard	The numbers of the election event.

Tab	Use
Data	The configuration of the election event.
IVR	The telephone voting configuration. Only when Telephone voting is allowed.
Localization	The texts of the election event in each language.
Voters	The voter list.
Areas	The areas and their contests.
Keys	The key ceremony. See 03-KEYS .
Certificates	The certificate authorities for voter digital certificates. Only when the election event uses voter certificates.
Tally	The tally ceremony and the results. See 05-TALLY .
Tally sheet imports	The import of result files from other voting systems.
Publish	The publication of the ballot and the voting period. See 04-PUBLISH .
Tasks	The background tasks, for example imports and exports, with their status.
Logs	The record of the actions in the election event.
Scheduled Events	The automatic start and end of the voting period.
Reports	The reports and their schedule.
Approvals	The applications of voters who enroll.

Import an election event (alternative)

Use this method to make an election event from a file, for example a copy of an earlier election event.

1. In the menu on the left, click the **+** icon next to **Election Events**.
2. Click **Import Election Event**.
3. Type the hash of the file in **Integrity Check (SHA-256)**.
4. Drop the file in the upload area. The file can be a `.json` file, a `.zip` export or an encrypted `.ezip` export.
5. If the file is an `.ezip` file, type the password in **Decryption Password**.
6. Click **Import**.

Expected result: the message "Election event imported Successfully" shows.

CAUTION

If you do not type the hash, the admin portal asks "Import Without Integrity Check?". Click **Go Back** and type the hash. Without the hash, you cannot know if somebody changed the file.

2. Configure the election event

1. Open the election event.
2. Click the **Data** tab.
3. In **General**, type the **Name**, the **Alias** and the **Description** for each language.
4. In **Language**, select the languages of the election event. Select the **Default** language.
5. In **Voting Channels Allowed**, turn on the channels of the election event: **Online**, **Kiosk**, **Early voting** or **Telephone voting**.
6. In **Ballot Design**, set the logo and the other options of the voting portal.
7. In **Password Policy**, set the password rules for the voters, if voters use a password.
8. In **Advanced Configurations**, make sure that **Lockdown Status** is **Not Locked Down**.
9. In **Advanced Configurations**, set **Keys/Tally Ceremonies Policy**:
 - **Manual Ceremonies**: the trustees take part in the key ceremony and in the tally ceremony. This is the default value.
 - **Allow Automatic Ceremonies**: you can make ceremonies without trustee actions.

10. To publish the results on the results website, set **Results Website** to **Enabled**. Then set **Results Website Access** and **Results Website Visibility**.
11. Click the save button.

Expected result: the admin portal saves the data. The menu shows the new name.

 NOTE

Locked Down hides most tabs of the election event and stops all publications. Use it only when you must freeze the election event.

 WARNING

In an automatic ceremony, no person keeps a key fragment. The platform can decrypt the votes without the trustees. Use automatic ceremonies only for tests and for elections where your rules permit it.

3. Create the elections

Do these steps for each election.

1. In the menu on the left, click the three dots next to the election event.
2. Click **Create an Election**.
3. Type the **Name**.
4. Type the **External ID**. This is the identifier of the election in your other systems.
5. Type the **Description**.
6. Click the save button.
7. Click the **Data** tab of the new election.
8. In **General**, type the name and the description for each language.
9. In **Voting Channels Allowed**, select the voting channels of this election.
10. In **Advanced Configuration**, set **Initialize Report Policy**. Select **Required** if you must make an initialization report before the voting period opens.
11. In **Custom filters**, set **Allow Tally**. The table gives the values.
12. Click the save button.

Allow Tally value	Meaning
Allowed	You can start the tally at any time.
Disallowed	You cannot start the tally.
Requires Voting Period End	You can start the tally only after all voting channels are closed.

Expected result: the election shows under the election event in the menu. The election has these tabs: **Dashboard**, **Data**, **Voters**, **Publish**, **Approvals** and **Tally Sheets**.

i NOTE

You cannot change the **External ID** after you save it.

4. Create the contests

Do these steps for each contest.

1. In the menu on the left, click the three dots next to the election.
2. Click **Create a Contest**.
3. Type the **Name**, the **Description** and the **External ID**.
4. Click the save button.
5. Click the **Data** tab of the new contest.
6. In **Ballot Voting System**, select the counting algorithm:
 - **Plurality at Large:** the voter selects candidates. The candidates with the most votes win.
 - **Instant Runoff:** the voter ranks the candidates. Then select the **Tie-Breaking Policy: Random** or **External Procedure**.
7. In **Ballot Design**, set the number of votes:
 - The minimum number of selections. Type 0 if the voter can leave the contest blank.
 - The maximum number of selections. For **Instant Runoff**, this is the lowest rank that a voter can give.
 - The number of winning candidates.

8. In **Policies**, set what the voting portal does when a voter selects too few, too many or no candidates. For **Instant Runoff**, also set the policies for duplicated ranks and skipped ranks.
9. Click the save button.

Expected result: the contest shows under the election in the menu.

CAUTION

The admin portal does not check that the minimum is lower than the maximum. Check the values before you save. Make sure that the maximum is not higher than the number of candidates.

5. Create the candidates

Do these steps for each candidate.

1. In the menu on the left, click the three dots next to the contest.
2. Click **Create a Candidate**.
3. Type the **Name**, the **Description** and the **External ID**.
4. Click the save button.
5. Click the **Data** tab of the new candidate.
6. In **General**, type the name and the description for each language.
7. In **Image**, add a photo or a logo if necessary.
8. Click the save button.

Expected result: the candidate shows under the contest in the menu.

6. Create the areas

An area tells which contests its voters see. Each voter belongs to one area.

1. Open the election event.
2. Click the **Areas** tab.
3. Click **Create Area**.
4. Type the **Name** and the **Description**.

5. In **Contests**, type three or more letters of a contest name, then select the contest. Do this for each contest of the area.
6. If the area is part of a larger area, select the parent area.
7. If the voters of the area can vote early, turn on **Allow Early Voting**.
8. Click the save button.

Expected result: the message "Area created" shows. The area shows in the list.

Import areas (alternative)

1. On the **Areas** tab, click **Import**.
2. Upload a CSV file without a header row. Each row has six columns in this order: identifier, country code, code, area name, delete flag, early voting policy.
3. Use ☐ in the delete flag column for each area to import.

CAUTION

An import links each imported area to **all** the contests of the election event. After the import, open each area and remove the contests that are not for that area.

7. Add the voters

Import the voters

1. Open the election event.
2. Click the **Voters** tab.
3. Click the import button. The **Import Voters** panel opens.
4. Click the link to download the example CSV file. Use it as a model.
5. Prepare your CSV file. The first row must be the header. The table gives the columns.
6. Upload the file.

Column	Content
username	The user name of the voter.
email	The email address.

Column	Content
<code>first_name</code> , <code>last_name</code>	The name of the voter.
<code>area_name</code>	The name of the area of the voter. It must be the same as in the Areas tab.
<code>enabled</code>	<code>TRUE</code> or <code>FALSE</code> . A voter with <code>FALSE</code> cannot sign in.
<code>password</code>	Optional. The password of the voter.
<code>sequent.read-only.mobile-number</code>	Optional. The mobile number, for text messages.
<code>vote-weight</code>	Optional. A positive whole number. Use it only if the election event uses weighted voting.

Other columns become attributes of the voter. Use a comma as separator. For a tab separator, give the file the extension `.tsv`.

Expected result: the message "Voters Import Scheduled Successfully" shows. When the task is complete on the **Tasks** tab, the voters show on the **Voters** tab.

Create one voter

1. On the **Voters** tab, click the create button.
2. Type the fields of the voter.
3. Select the **Area**.
4. Select **Enabled**.
5. Click the save button.

Expected result: the voter shows in the list.

If there is a problem

Problem	Action
The + icon next to Election Events is not there.	Your account does not have the permission to create election events. Ask your tenant administrator.
The import shows "Hashes don't match. Integrity check failure."	The file is not the file that you expect. Get the file again from its source. Do not import it without the check.
A language is not available in Language .	Turn on the language in Settings > LANGUAGES . See 01-TENANT .
The voter import fails on the Tasks tab.	Open the task to see the error. Check the header row and the area names, then import again.
A contest is not in the list of Contests of an area.	Type three or more letters of the contest name.

Next: [03-KEYS](#): Run the key ceremony.

03-KEYS: Run the key ceremony

The key ceremony makes the keys of the election. The voting portal encrypts each ballot with the public key. The private key is in fragments: each trustee keeps one fragment. To decrypt the votes, a minimum number of trustees (the threshold) must give their fragment at the tally ceremony.

The election administrator starts the ceremony. Then each trustee downloads, saves and checks their key fragment.

Before you start

- All the elections of the election event exist. See [02-EVENT](#).
- Each trustee has an account with the `trustee` role and the correct **Act as Trustee** value. See [01-TENANT](#).
- You know the threshold. It must be 2 or more, and not more than the number of trustees.
- Each trustee has two storage devices for the backups, for example two USB flash drives.
- You have agreed a date and a time with the trustees. All trustees must do their part.

WARNING

In version 10.0 you cannot cancel or restart a key ceremony from the admin portal. Check the threshold, the trustees and the election before you confirm. If you must change a key ceremony, contact Sequent support.

CAUTION

A key ceremony for **All Elections** stops you from making other key ceremonies in the election event. Create all the elections before you start it. If you add an election later, it cannot get keys.

1. Create the key ceremony (election administrator)

1. Open the election event.
2. Click the **Keys** tab.
3. Click **Create Keys Ceremony**. If the election event already has a key ceremony, click **Add**.
4. In **Threshold**, type the threshold.
5. In **Trustees**, select the trustees of the ceremony. Select a number of trustees that is equal to or more than the threshold.
6. If the election event permits automatic ceremonies, decide about **Automatic Ceremony**. Leave it off for an election with trustees. See the warning in [02-EVENT](#).
7. In **Election**, select one election. To make one key for all the elections, leave the field empty. The ceremony then has the name **All Elections**.
8. Click **Create Keys Ceremony**.
9. Read the message "Are you sure you want to Create Keys Ceremony?".
10. Click **Yes, Create Keys Ceremony**.

Expected result: the message "Keys Ceremony created" shows. The ceremony shows in the list of the **Keys** tab. Its status is **STARTED**, and then **IN_PROGRESS**.

11. Tell the trustees that the key ceremony has started. For an automatic ceremony, the trustees do nothing. Go to [step 3](#).

2. Save the key fragment (each trustee)

Each trustee does these steps on their own computer, with their own account.

1. Sign in to the admin portal.
2. Open the election event.
3. Click the **Keys** tab. The message "You have been invited to participate in a Keys ceremony." shows.
4. In the row of the ceremony, click the key icon **Participate in Keys Ceremony** in the **Actions** column.

5. Read the **Trustee Keys Ceremony** page. Make sure that the trustee name on the page is your trustee.
6. Click **Next**.

If the message "Waiting for Keys Generation.." shows, the trustee services have not made the keys yet. Wait one or two minutes. **Next** becomes available.

7. Click **Download your Encrypted Private Key**. The browser saves a file with the name `encrypted_private_key_trustee_<user>_<election>.txt`.
8. Copy the file to your first storage device.
9. Copy the file to your second storage device.
10. Click **Next**. The **Backup your Encrypted Private Key** window opens.
11. Select **First backup secured**.
12. Select **Second backup secured**.
13. Click **Confirm Backups and Continue**.
14. Drop the file from one storage device into the upload area of the **Check your Encrypted Private Key Backups** page.
15. Make sure that the message "Backup verified successfully." shows.
16. Click **Next**.
17. Delete the downloaded file from the computer. Keep only the two copies on the storage devices.
18. Keep the two storage devices in two different safe locations.

WARNING

Do not give your key fragment to another person, and do not send it by email or chat. Keep it until the end of the tally ceremony and the end of the period for claims. If too many trustees lose their fragment, nobody can decrypt the votes.

Expected result: the **Finished** step shows the progress of the ceremony.

Check a backup again (optional)

You can check a backup later, for example before the tally ceremony. Do this while the ceremony status is `IN_PROGRESS` or `SUCCESS`.

1. On the **Keys** tab, open the ceremony.
2. Click **Verify key**.

3. Drop the file from a storage device into the upload area.
4. Make sure that the message "Backup verified successfully." shows.

3. Check that the ceremony is complete (election administrator)

1. Open the election event.
2. Click the **Keys** tab.
3. Click the view icon of the ceremony. The **Keys Ceremony Progress** page opens.
4. Make sure that each trustee has a tick in these columns:
 - **Key Fragment Generated**
 - **Private Key Fragment Downloaded**
 - **Private Key Fragment Checked**

Expected result: the status is **SUCCESS**. The status changes to **SUCCESS** only when all trustees have checked their key fragment. An automatic ceremony changes to **SUCCESS** when the public key exists.

4. Record the date, the time, the trustees present and the result of the ceremony.

Ceremony status

Status	Meaning
STARTED	The ceremony is created.
IN_PROGRESS	The trustees can download and check their key fragment.
SUCCESS	All trustees have checked their key fragment. The ceremony is complete.
CANCELLED	The ceremony is cancelled.

If there is a problem

Problem	Action
"You selected threshold ... but it must be between ... and ..."	Type a threshold between 2 and the number of trustees.
"You selected only ... trustee(s), but you must select at least ..."	Select more trustees, or type a lower threshold.
The Keys tab is not there for a trustee.	The account does not have the <code>trustee</code> role. Ask your tenant administrator.
The trustee does not see the key icon.	Make sure that Act as Trustee of the account is the trustee of the ceremony.
"Invalid Encrypted Private Key Backup, please try again"	Use the file from the other storage device. If both fail, click Back and download the file again.
"Private key download is no longer available because the ceremony has moved on."	You cannot download the file again. Use your backups.
"there's already an existing running ceremony ..."	The election, or all the elections, already have a key ceremony. Contact Sequent support if you must replace it.
The status stays at <code>IN_PROGRESS</code> .	A trustee has not checked their key fragment. Look at the trustee table and contact that trustee.

Next: 04-PUBLISH: Publish the ballot and manage the voting period.

04-PUBLISH: Publish the ballot and manage the voting period

This procedure makes the ballot available in the voting portal, then opens, pauses and closes the voting period. The election administrator does this procedure.

Before you start

- The content of the election event is complete and correct. See [02-EVENT](#).
- The key ceremony status is **SUCCESS**. See [03-KEYS](#).
- Your account has the permissions to publish and to change the voting status.
- You know your password. Publication and the voting status are sensitive actions. See [00-START](#).

WARNING

Do not publish the ballot for voters before the key ceremony status is **SUCCESS**. Version 10.0 does not stop you. Without the election key, the voting portal works in demo mode and does not record real votes.

NOTE

The **Publish** tab of the election event applies to all its elections. Each election also has a **Publish** tab that applies only to that election.

1. Make a publication

1. Open the election event.
2. Click the **Publish** tab.
3. Click **Generate Publication**. If the election event already has a publication, click **Publish Changes**.

4. Read the message "You are about to generate a publication."
5. Click **Confirm**. If the admin portal asks for your password, type it.

Expected result: the message "Ballot generated" shows. The **Changes to be Published** page compares the **Current** publication with the **Changes to Publish**.

6. Read the changes. Make sure that they are the changes that you expect.

2. Preview the ballot

1. On the **Changes to be Published** page, click **Preview**.
2. In **Select Area for Preview**, select an area.
3. Open the preview. To send the preview to another person, click **Copy link**.
4. Check the ballot of the area: the elections, the contests, the candidates, the texts in each language and the number of selections.
5. Do steps 2 to 4 for each area.

If you find an error, correct it in the election event. Then start again at step 1 of [Make a publication](#).

3. Publish the ballot

1. On the **Changes to be Published** page, click **Publish Changes**.
2. Confirm the action.

Expected result: the message "Ballot published" shows. The **Publish History** list shows the new publication.

NOTE

To change the ballot after you publish it, change the election event and do this procedure again from step 1. Voters see the last publication.

4. Make the initialization report (if necessary)

Do this step only if **Initialize Report Policy** of an election is **Required**. The initialization report shows that the ballot box is empty before the voting period opens. **Start Voting** stays unavailable until the report exists.

1. Open the election event.
2. Click the **Tally** tab.
3. Click **Generate Initialization Report**.
4. Select the elections in **Elections for Initialization Report**.
5. Click **Start Initialization Report**.
6. Continue as in a tally ceremony. See [05-TALLY](#).

Expected result: the **Tally** list shows a row with the **Tally Type Initialization Results**.

5. Open the voting period

Each voting channel has its own status. Open each channel of the election event. To open the voting period automatically, use [scheduled events](#).

1. Open the election event.
2. Click the **Publish** tab.
3. Click **Start Voting**. A menu shows one item for each channel.
4. Click the channel to open, for example **Start Online Voting**.
5. Read the message "You are about to start voting period."
6. Confirm the action. If the admin portal asks for your password, type it.
7. Do steps 3 to 6 for each other channel of the election event.

Expected result: the message "Election status changed" shows. Voters can vote on the channels that you opened.

NOTE

A menu item is not available if the channel is not allowed in **Voting Channels Allowed**. Open **Early Voting** before **Online Voting**. You cannot start early voting after online voting has started.

8. Sign in to the voting portal with a test voter account, if you have one, and make sure that the ballot shows.

6. Pause or close the voting period

To stop the voting for a short time:

1. On the **Publish** tab, click **Pause Voting**.
2. Click the channel, for example **Pause Online Voting**.
3. Confirm the action.
4. To continue, click **Start Voting**, click the channel and confirm the action.

To close the voting period at its end:

WARNING

You cannot open a channel again from the admin portal after you close it. Close the voting period only at its official end.

1. On the **Publish** tab, click **Stop Voting**.
2. Click the channel, for example **Stop Online Voting**.
3. Confirm the action.
4. Do steps 1 to 3 for each other open channel.

Expected result: the message "Election status changed" shows. Voters cannot vote on the closed channels.

7. Schedule the voting period (alternative)

Use scheduled events to open and close the voting period at a set time.

1. Open the election event.
2. Click the **Scheduled Events** tab.
3. Click **Create Scheduled Event**.
4. In **Type**, select **Start Voting Period**.
5. In **Election**, select an election. Leave it empty for all the elections.

6. Select the voting channels.
7. Type the date and the time. The admin portal uses the time zone of your computer. The label of the field shows the time zone.
8. Click the save button.
9. Do steps 3 to 8 again with the type **End Voting Period**.

Expected result: the **Scheduled Events** tab shows the two events.

CAUTION

Check the time zone in the label of the date field. If your computer is not in the time zone of the election, convert the times.

If there is a problem

Problem	Action
"Election event is locked down"	Set Lockdown Status to Not Locked Down on the Data tab. See 02-EVENT .
"Ballot publication not generated yet, can't publish."	Make a publication first. See step 1 .
"Insufficient privileges"	Confirm your identity again with your password, then repeat the action.
Start Online Voting is not available.	Make the initialization report, if the election needs one. If the channel is closed, contact Sequent support.
A channel is not in the menu, or is not available.	Allow the channel in Voting Channels Allowed on the Data tab.
"Ballot style generation failed: ..."	The ballot content is not valid. Read the message, correct the election event and make a new publication.
Stop Online Voting is not available.	A scheduled event does not permit the end of the voting period yet. Look at the Scheduled Events tab.

Problem	Action
Voters do not see a contest.	Make sure that the contest is in the area of the voter. Then make and publish a new publication.

Next: 05-TALLY: Run the tally ceremony.

05-TALLY: Run the tally ceremony

The tally ceremony decrypts and counts the votes. The election administrator creates the tally. The trustees upload their key fragments. When enough trustees have uploaded their fragment, the election administrator starts the tally.

Before you start

- The key ceremony status is `SUCCESS`. See [03-KEYS](#).
- The election event has a publication. See [04-PUBLISH](#).
- All voting channels are closed, unless you make an initialization report.
- **Allow Tally** of each election permits the tally. See [02-EVENT](#).
- A number of trustees equal to or more than the threshold are available, with one copy of their key fragment.
- You have agreed a date and a time with the trustees.

WARNING

Start the tally of the election results only after the official end of the voting period. If **Allow Tally** is **Allowed**, the admin portal lets you start the tally while voters can still vote. Use **Requires Voting Period End** to prevent this.

1. Create the tally (election administrator)

1. Open the election event.
2. Click the **Tally** tab.
3. Click **Start Tally Ceremony**.
4. In **Elections to Tally**, select the elections to count.
5. In **Keys Ceremony**, select the key ceremony of these elections. If there is only one key ceremony, the admin portal selects it.

6. Click **Start Tally Ceremony**. For an automatic key ceremony, the button is **Start Tally**.
7. Read the message "This action will notify the trustees to import their key fragments."
8. Click **Ok**.

Expected result: the message "Tally created" shows. The tally shows in the list of the **Tally** tab with the status STARTED.

9. Tell the trustees that the tally ceremony has started.

For an automatic key ceremony, the trustees do nothing and the tally runs immediately. Go to [step 3](#), item 7.

2. Upload the key fragment (each trustee)

Each trustee does these steps with their own account. You need one copy of your key fragment from your storage device.

1. Sign in to the admin portal.
2. Open the election event.
3. Click the **Tally** tab. The message "You have been invited to participate in a Tally ceremony." shows.
4. In the row of the tally, click **Add Tally Key** in the **Actions** column.
5. Read the list of **Elections to Tally**. Make sure that it is the list that you expect.
6. Drop your key fragment file into the upload area.
7. Make sure that the message "Backup verified successfully." shows.
8. Click **Next**.
9. Remove the storage device and keep it in its safe location.

Expected result: the trustee has uploaded the key fragment.

3. Start the tally (election administrator)

1. On the **Tally** tab, click the view icon of the tally.

2. Look at the **Trustees** section. It shows how many trustees have imported their key and how many are necessary.
3. Wait until the status is **CONNECTED**. This status shows that enough trustees have uploaded their key fragment.
4. Click **Start Tally**.
5. Read the message "Do you want to start the Tally?".
6. Click **Start Tally**.

Expected result: the message "Tally started" shows. The status changes to **IN_PROGRESS**. The **Elections Tally Progress** table shows the progress of each election.

7. Wait until the status is **SUCCESS**. The time depends on the number of votes. If the status is **AWAITING_INPUT**, resolve the tie. See [Resolve a tie](#).
8. Click **Results**. See [06-RESULTS](#).
9. Record the date, the time, the trustees present and the result of the ceremony.

Tally status

Status	Meaning
STARTED	The tally is created. The trustees can upload their key fragments.
CONNECTED	Enough trustees have uploaded their key fragment. You can start the tally.
IN_PROGRESS	The platform mixes, decrypts and counts the votes.
AWAITING_INPUT	The count has stopped at a tie that needs a decision.
SUCCESS	The tally is complete. The results are available.
CANCELLED	The tally is cancelled.

The **Elections Tally Progress** table shows these steps for each election: **WAITING**, **MIXING**, **DECRYPTING**, **SUCCESS** or **ERROR**.

Resolve a tie

An **Instant Runoff** contest with the tie-breaking policy **External Procedure** can stop at a tie. The tally page then shows "Tally paused due to unresolved tie".

1. Get the decision about the tie, as your election rules require.
2. On the tally page, enter the decision for each tie.
3. Click **Apply Resolutions and Recalculate**.

Expected result: the status changes to `IN_PROGRESS`, and then to `SUCCESS`.

Count again

You can count a completed tally again, for example after an approved change of the tally sheets.

1. On the **Tally** tab, click **Recount tally** in the row of the tally.
2. Read the message "This will generate a fresh results event for the completed tally session."
3. Click **Recount**.

Cancel a tally

You can cancel a tally before it starts, when its status is `STARTED` or `CONNECTED`.

CAUTION

You cannot undo a cancellation. The trustees must upload their key fragment again for a new tally.

1. On the **Tally** tab, click **Cancel Tally Ceremony** in the row of the tally.
2. Click **Cancel Tally**.

If there is a problem

Problem	Action
"The Tally Ceremony cannot start until the Keys Ceremony has been successfully completed."	Complete the key ceremony. See 03-KEYS .
"The Tally Ceremony cannot start until you create one publication in the Publish tab."	Publish the ballot. See 04-PUBLISH .
"Select at least one election."	Select one or more elections.
"Publish each selected election before creating its tally."	Publish the ballot. See 04-PUBLISH .
"End voting in each selected election and stop its active voting channels before creating the tally."	Close all the voting channels of the elections. See 04-PUBLISH .
"Tallying is disabled for a selected election."	Allow Tally of the election is Disallowed . Change it on the Data tab of the election.
"Elections have different keys ceremonies"	Make one tally for each key ceremony.
"You cannot continue the ceremony because the tally session is not connected or the start of the ceremony is not allowed."	Wait for more trustees. If the status is CONNECTED , check Allow Tally of each election and close the voting period.
"Trustee not part of the keys ceremony or has invalid state"	The trustee is not in the key ceremony of these elections, or has already uploaded the fragment.
A trustee's file is not accepted.	Use the copy from the other storage device. Make sure it is the file of this election event.

Problem	Action
An election shows ERROR .	Do not start a new tally. Contact Sequent support and give the name of the election event.

Next: 06-RESULTS: Get the results.

06-RESULTS: Get the results

This procedure shows the results of a tally, downloads the result documents and publishes the results on the results website. The election administrator does this procedure.

Before you start

- The tally status is **SUCCESS**. See [05-TALLY](#).
- Your account has the permission to export the results.

1. Look at the results

1. Open the election event.
2. Click the **Tally** tab.
3. Click the view icon of the tally.
4. Click **Results**.
5. Open **Results & Participation**.
6. Select the election, the contest and the area to see.

Expected result: the page shows:

- The **Participation Summary**: the eligible voters, the voters, and the valid, invalid and blank votes, with their percentages.
 - The participation by channel.
 - The **Candidate Results**: the votes of each candidate and the winning positions. For an **Instant Runoff** contest, the page also shows each round, the eliminated candidates and the winner.
7. Compare the number of votes counted with the number of votes cast in the **Dashboard** tab.

2. Download the result documents

You can download the results of the election event, an election, a contest or an area.

1. On the results page, open the actions menu of the item.
2. Click the format to download. The menu shows the formats that exist for the item.

Format	Content
PDF	The results document, ready to print and sign.
HTML	The results document, for a browser. You can also make a PDF from it.
JSON	The results in a data format, for other systems.
TAR_GZ	An archive with all the result files.

For an election, the menu can also export the results of all its areas in one HTML or JSON file. For the election event, you can also export the results in **XLSX** format, for a spreadsheet.

3. Keep the files in a safe location.

Expected result: the browser saves the file.

3. Make the reports

The **Reports** tab makes other documents, for example the electoral results report, a statistical report or the activity log.

1. Open the election event.
2. Click the **Reports** tab.
3. Click **Create Report**.
4. Select the **Report Type**, for example **Electoral Results** or **Participation Report**.
5. Select the **Election** and the **Template**.
6. In **Encryption Policy**, select **Configured Password** to protect the file with a password, or **Unencrypted**.
7. Click the save button.

8. In the row of the report, click **Generate**.

Expected result: the report is available to download when its task is complete.

4. Publish the results on the results website

Do this step only if your election event uses the results website.

Before you publish

- **Results Website** is **Enabled** on the **Data** tab of the election event. See [02-EVENT](#).
- The tally status is **SUCCESS**.
- Your account has the permission to publish the results.
- The persons responsible for the election have approved the results.

CAUTION

The public can see the results immediately after the publication. Check the results before you publish them.

1. On the results page of the tally, open **Publish to results website**.
2. In **Route**, select **Event results** or **Election results**. For **Election results**, select the **Election**.
3. In **Access**, select **Public access** or **Authenticated access**.
4. In **Visibility**, select the scope of the results.
5. In **Contests**, select the contests to publish.
6. Click **Publish selected contests**.
7. Read the message "Start publish to results website?" and confirm.

Expected result: the message "Results publication started" shows. The **Publication history** shows a new version.

8. In **Publication history**, click **Open** to see the published page.

To remove a publication from the results website, click **Revoke** in its row of the **Publication history**.

For more information, see [Results website](#).

5. Keep the records

1. Export the election event for your archive. On the **Data** tab, click the export button, then select the content to include. Select **Encrypt with Password**.
2. Keep the export file, the result documents and the records of the ceremonies together.
3. Keep the key fragments until the end of the period for claims. Then destroy them as your rules require.

If there is a problem

Problem	Action
The Results button is not available.	The tally is not complete. Wait until the status is SUCCESS .
"Results website publishing is disabled for this election event."	Set Results Website to Enabled on the Data tab of the election event.
"Results can be published after this tally has completed."	Wait until the tally status is SUCCESS .
A format is not in the menu.	The platform did not make that document for the item. Contact Sequent support.
The number of votes is not what you expect.	Do not publish the results. Check the publication, the areas and the Logs tab, then contact Sequent support.

Configure Password Policy

Each election event has its own Keycloak realm and password policy. Election managers can view and update the managed parts of that policy directly from the Admin Portal instead of editing the realm in the Keycloak Admin Console.

The policy is enforced when an administrator changes a voter's password and is also used to generate the voter credential included in a Voter Information Letter.

Required permissions

Operation	Required permissions
Open the election event Data tab	<code>election-event-data-tab</code>
View the current policy	<code>election-event-read</code>
Change and save the policy	<code>election-event-write</code>

A full administrator normally has these permissions. Custom least-privilege roles must include them explicitly.

Configure the policy

1. In the Admin Portal, select the election event.
2. Open the **Data** tab.
3. Expand **Password Policy**.
4. Set the minimum and maximum lengths.
5. Select the character classes that every new password must contain.
6. Select **Save** to update the election event realm.

Hover over the information icon beside a field to see its meaning.

Field	Meaning
Minimum length	The minimum number of characters required for the password.
Maximum length	The maximum number of characters allowed for the password.
Include uppercase letters	Require at least one uppercase character.
Include lowercase letters	Require at least one lowercase character.
Include digits	Require at least one numeric digit.
Include special characters	Require at least one supported special character.

Lengths must be whole numbers from 1 through 256, and the minimum cannot exceed the maximum. Selecting a character class requires at least one character from that class. Clearing it removes that requirement; it does not prohibit that character class from appearing in a password.

⚠️ DEFAULT POLICY

If the event realm does not yet have a password policy, the Admin Portal shows defaults of 12 minimum characters, 72 maximum characters, and all four character classes selected. Saving applies those defaults to the realm.

What changes

Saving updates these Keycloak rules in the election event realm:

- `length`
- `maxLength`
- `upperCase`

- `lowerCase`
- `digits`
- `specialChars`

Other Keycloak password-policy rules that are not managed by this form are preserved.

The update affects passwords set after the policy is saved. It does not replace existing voter passwords automatically. If an administrator enters a non-compliant password through **Voters > Change password**, the dialog remains open, marks both password fields as invalid, and explains that the password must be corrected.

VOTER INFORMATION LETTERS

A Voter Information Letter cannot be generated until the election event has a configured password policy. Each letter generation creates a new voter credential that complies with the current policy.

Create Voters

Create a Voter

1. Open the election event and select the **Voters** tab.
2. Select **Add**.
3. Complete the voter fields. Custom fields and their order come from the election event's Keycloak User Profile.
4. Review the changes and save the voter.

For instructions on defining custom fields, see [Adding User Attributes to Keycloak](#).

Secret Voter Fields

Fields configured with `sequent.secret=true` are stored encrypted. They are not available as voter list columns, filters, or sort fields, and their stored value is never loaded with the ordinary voter record.

The permissions to reveal and edit these fields are independent:

- With `voter-secret-attribute-write`, enter a value when creating a voter or type a replacement while editing one. The previous value does not need to be revealed before replacement.
- With `voter-secret-attribute-read`, select the eye button to reveal the value. The button is disabled while decryption is in progress. Select it again to hide the value.
- Both operations also require the corresponding ordinary voter read or write permission.

When editing a voter, an existing unrevealed value is displayed as `.....`. Leaving it untouched preserves the encrypted value. Use the **Clear** (×) icon at the right of the field and save to remove a value without revealing it. Required fields still need a value. For multivalued fields, each value can be edited, added, or removed separately. Review screens mask new and changed secret values rather than repeating them.

A secret can also be used as the login credential with the [Multi-Attribute + Password Form](#). In that mode, fill the configured secret field (for example, **Mysecret**); the separate Keycloak password and password-reset controls do not set or reset that secret.

Revealed values are limited to the open voter editor. Closing the editor removes them from its state; the voter list continues to receive only redacted values.

Export Voters

Select **Export** from the Voters tab to generate a CSV.

- The default export omits every configured secret column. It does not export encrypted envelopes.
- A user with `voter-secret-attribute-read` can select **Include decrypted secret voter fields** in the confirmation dialog. This adds the secret columns with plaintext values.
- Treat a decrypted export as sensitive data. The generated document remains protected by the secret-read permission when it is downloaded.

For the complete permission combinations, see [Permissions](#).

Import Voter List

Prepare the CSV

Use one row per voter. Column names must match the canonical voter and Keycloak User Profile attribute names; the display label shown in the Admin Portal is not the import header. For example, an attribute named `customerReference` uses this header:

```
username,email,customerReference  
voter-001,voter-001@example.com,REF-12345
```

Use `|` between values for a multi-valued User Profile attribute. Keep the source CSV private: it contains the values in plaintext even when a column is configured as secret.

Import the Voters

1. Open the election event and select the **Voters** tab.
2. Select the voter import action and upload the CSV.
3. Review the task result and correct any reported row or header errors.

Importing Secret Voter Fields

A CSV can contain a field configured with `sequent.secret=true`. Supply the normal plaintext value; do not pre-encrypt it and do not copy an encrypted `seqenc:` value from Keycloak. The import worker creates the voter ID and encrypts each non-empty secret cell before it is written to Keycloak.

The initiating user needs `voter-secret-attribute-write` in addition to the normal voter import and create access. If the CSV contains a secret header and the user lacks that permission, the whole import is rejected and the error identifies the affected columns.

After import:

- ordinary voter views and exports do not return the secret value or its encrypted envelope;

- users with secret-read permission can reveal it from the voter editor;
- a decrypted export can include it only when explicitly requested by an authorized user.

See [Protecting a Voter Attribute as Secret](#) for configuration restrictions, and [Permissions](#) for authorization details.

Voter Communication

Send a Voter Communication

1. Open an election event and select the **Voters** tab.
2. Select the intended voters or choose an audience such as all voters, voters who have voted, or voters who have not voted.
3. Open **Send communication**, select email or SMS, and write the localized content.
4. Send immediately or schedule delivery.

The normal communication permission is required. Delivery runs per voter, so template variables are resolved separately for every recipient.

Using Secret Voter Fields

Secret voter fields can be used in email and SMS content without making them visible in the voter list. For an attribute named `customerReference`, use the same Handlebars shapes as an ordinary custom voter attribute:

```
Your reference is {{user.customerReference}}.
```

`user.<attribute>` contains the first value. The complete value array is available through:

```
{{lookup user.attributes "customerReference"}}
```

When the communication is submitted, the Admin Portal declares each configured secret attribute whose name appears in the template content. The worker validates that declaration, decrypts only those fields for the current voter, renders the message, and discards the plaintext context. Secret fields not referenced by the content are not decrypted or added to the template variables.

Sending or scheduling content that declares a secret field requires `voter-secret-attribute-read` in addition to the normal notification permission. If the field is no longer configured as secret, its envelope is invalid, or the operator lacks permission, the job fails without substituting the stored value.

See [Permissions](#) and [Templates](#).

Reports and Templates

Overview

To generate a report, you need to select the Election Event in the Admin Portal and add an entry for a specific report type.

Creating a Report

1. Go to the **REPORTS** tab and click the **Create Report** or **ADD** button.
2. Select the type (see special notes for specific types below).
3. **Optional:** Select Election for reporting only a specific election.
4. **Optional:** Choose a template. You must have created one earlier (see [more about creating templates](#)). If none is selected, the system's default will be used.
5. Click **Save**.

Generating Reports

After creating a report entry, you will see it in the list. Click on the actions menu (3 dots on the right side) and choose:

- **Generate:** Create a report based on real data
- **Preview:** Generate an example of that report template based on mock data

Report Types

Voters Turnout

Note: To display male and female percentages, the sex attribute must have been added. Check out [how to add the sex attribute](#).

Other Report Types

Additional report types will be documented here as they become available.

Secret Voter Fields in Reports

Secret voter fields are available only to reports that render one identified voter at a time. Current examples are voter information letters and manual verification reports. Aggregate reports such as turnout, election activity, tally, and results do not receive a voter object and cannot use secret fields.

The assigned template must both reference and declare every secret field it uses. For example:

```
{
  "secret_attribute_names": ["customerReference"],
  "document": "<p>Reference: {{user.customerReference}}</p>"
}
```

At generation time, Step validates the declaration against the election event's current Keycloak User Profile and decrypts only the declared names for the report's voter. The operator needs `voter-secret-attribute-read` in addition to the normal report permission. An undeclared, removed, or unreadable secret causes generation to fail rather than exposing its stored representation.

See [Secret Voter Variables](#) for variable shapes and declaration rules.

Downloading Secret-Bearing or Encrypted Reports

Secret-bearing files use private storage. Downloading one requires `voter-secret-attribute-read` as well as the ordinary document permissions, even when the file was generated by another administrator.

Secret classification and password encryption are separate: declaring a secret does not force every report to be encrypted. Reports retain their configured encryption policy; voter information letters retain their existing password-protected PDF behavior.

For a newly generated password-encrypted report, download it from **Reports** or the completed entry in **Tasks**. The Admin Portal opens the existing report-password dialog; retrieving the password requires `document-password-read` and, for a secret-bearing document, `voter-secret-attribute-read`. The password belongs to that generated document, so changing the report configuration later does not change an earlier file's password. Older encrypted files without a stored document password still show decryption instructions; use the password supplied when they were generated.

Email and SMS are delivery messages, not password-encrypted report files. Their bodies remain readable by the recipient, including declared secret fields. See [test-only Console delivery](#) before testing a template containing credentials.

Export Data

The Admin Portal provides two voter export paths:

- **Voters > Export** creates a CSV containing the voters in the current election event.
- **Election Event > Export** creates an election-event archive and can optionally include a voters CSV.

Secret voter fields

Secret voter fields are omitted from exports by default. This prevents encrypted storage values from appearing in a CSV when the user did not request access to their plaintext values.

To include decrypted secret fields in a standalone voters CSV, select **Include decrypted secret voter fields**. The user starting the export must have the `voter-secret-attribute-read` permission. The export task verifies this authorization before reading the fields.

An election-event archive includes decrypted secret voter fields only when all of the following are true:

1. **Include Voters** is selected.
2. **Encrypt with Password** is checked, so the resulting archive is password protected.
3. The user starting the export has the `voter-secret-attribute-read` permission.

The **Encrypt with Password** checkbox must be selected explicitly. Choosing another export option that requires archive encryption (such as reports, applications, or bulletin-board data) does not opt in to exporting voter secrets.

If the user does not have that permission, the election-event export still includes the ordinary voter fields but omits all secret fields. Archives that contain decrypted voter secrets are marked as sensitive, so downloading them also requires the secret-read permission.

The **S3 files** option can also include secret-bearing reports when **Encrypt with Password** is explicitly selected and the operator has secret-read permission, even if **Include Voters** is off. Without both safeguards, these documents are excluded. Objects without committed document metadata are always excluded. CSV secret exports remain plaintext downloads with an explicit warning; store and transfer them securely. Importing these CSV columns still

requires secret-write permission and encrypts each value for the destination event and voter.

Ballot Boxes (Tally Sheets)

This tutorial explains how to digitalize results from non-electronic voting channels (such as **Paper** or **Postal** ballots) using **Ballot Boxes**, and how to review and approve them so they are included in the final tally alongside electronic votes.

! INFO

A **Ballot Box** (internally called a **Tally Sheet**) is the digitalized vote count for one unique combination of **Contest**, **Area**, and **Channel**. Every time you submit new figures for that combination, a new **version** is created — nothing is overwritten, so you always keep a full audit trail of who entered and who reviewed each version.

Step 1: Open the Tally Sheets Tab

Ballot boxes are managed at the **Election** level (not per contest).

1. Navigate to the **Election Event** and select the specific **Election**.
2. Select the **Tally Sheets** tab.

The page header reads **Ballot boxes — Digitalized ballot boxes by channel**.

! INFO

This tab is only visible to users with the **View Tally Sheet** permission. Creating ballot boxes requires **Create Tally Sheet**, and approving or disapproving versions requires **Review Tally Sheet**.

Step 2: Understand the Ballot Boxes List

The main list shows one row per unique **Contest + Area + Channel** combination, with:

- **Channel:** e.g. **PAPER** or **POSTAL**.
- **Contest** and **Area** the ballot box belongs to.
- **Latest version:** the version number of the most recently submitted figures.
- **Approved version:** the version number that has been approved and will be used in the final tally (- if none has been approved yet).

Each row has two actions available from the actions menu:

- **Add:** submit a new version of figures for that ballot box.
- **Versions:** open the full version history for that ballot box.

Step 3: Digitalize a New Ballot Box

1. Select the **+ Add** button above the list (or, if no ballot boxes exist yet, **Generate Tally Sheet** on the empty state).
2. Choose the **Contest**, **Area**, and **Channel** the figures belong to.
3. Enter the vote counts:
 - **Total Valid Votes**
 - **Implicitly Invalid Votes** and **Explicitly Invalid Votes** (the **Total Invalid Votes** field is calculated automatically as their sum)
 - **Blank Votes**
 - **Census** (must be entered manually and must be greater than or equal to the total votes)
 - The number of votes for each **Candidate**
4. Select **Confirm**. The system validates that the sum of candidate votes plus blank votes equals the total valid votes, and that the census is large enough; if not, an error is shown and you cannot proceed.
5. Review the read-only summary and select **Save**.

This creates **version 1** of the ballot box, with status **Pending**.



TIP

You cannot create a second, independent ballot box for the same Contest + Area + Channel combination. To correct or update figures for an existing ballot box, use the **Add** action from the list instead (see next step).

Step 4: Submit a New Version (Correction)

If figures need to be corrected (for example, after a recount), do not start a new ballot box — add a new version to the existing one:

1. From the Ballot Boxes list, select **Add** on the row of the ballot box you want to update.
2. The form opens pre-filled with the values of the latest version. Edit the fields as needed.
3. Select **Confirm**, review the summary, and **Save**.

This creates a new version (e.g. version 2) with status **Pending**, without affecting the previous versions yet.

Step 5: Review Version History

1. From the Ballot Boxes list, select **Versions** on the row of the ballot box you want to inspect.
2. The versions table shows, for every version submitted for that Contest + Area + Channel:
 - **Version** number
 - **Created by** and **Created at**
 - **Reviewed by** and **Reviewed at** (- until reviewed)
 - **Status:** PENDING, APPROVED, or DISAPPROVED
3. Use **Show** to view the read-only figures of any version.
4. Select **Back** to return to the Ballot Boxes list.

Step 6: Approve or Disapprove a Version

Pending versions can be approved or disapproved from the versions table (requires the Review Tally Sheet permission):

1. Select **Approve** or **Disapprove** on a pending version.
2. Confirm the action in the dialog.

 INFO

Approving a version sets its status to **Approved** and records the reviewer and review time. It also automatically discards the other versions of that same ballot box, so only one version per ballot box remains as the authoritative, approved record.

Disapproving a version marks it as **Disapproved**; you can still submit a new corrected version afterward using **Add**.

 TIP

Only **Approved** ballot boxes are included when the Tally Ceremony is executed — their figures are merged with the electronically cast and decrypted votes to produce the final results. Make sure every relevant ballot box has an approved version before running the tally.

Results Website

The results website publishes a deliberately reduced, read-only view of a completed tally. Publication creates a new version; it never modifies the tally database or exposes ballots, document references, internal statistics, or data from unselected events, elections, contests, or areas.

Policy and permissions

Configure the policy in the election event data screen using its dedicated **Save** action. Saving the rest of the election event does not change this policy.

- `Disabled` removes the event from discovery and cleans up active publication artifacts.
- `Enabled` permits publication using the configured access and visibility values.
- `Public` access supports only `Full event` visibility.
- `Authenticated` access supports `Full event` or `Area based` visibility. Area-based voters receive only the artifact for the area in their event-realm token.

`publish-results-write` is required to configure the policy, publish, revoke, or rebuild discovery. `publish-results-read` is required to read publication history. The generic `admin-user` role cannot read history or change `presentation.results_website` directly.

Publishing and routes

After a tally completes, select the contests and publish from the tally screen. Event publications are available at `/:electionEventId`; election-specific publications are available at `/:electionEventId/elections/:electionId`. An election route may use an event publication only when that publication explicitly contains the requested election.

Each route has monotonically increasing versions. A successful replacement supersedes the prior version and deletes its public or private artifacts. Mutable discovery indexes and latest aliases use `no-store`; immutable versioned artifacts may be cached.

Empty tallies still render General Information and Participation Summary pie charts as 100% non-voters. Candidate, blank, explicit/implicit blank, invalid, and preferential results retain their typed tally meaning.

Revocation and audit

Revocation records `Revoked` and `revoked_at` before cleanup, so retrying the same action safely resumes cleanup after an object-store failure. Revocation removes the publication from discovery and deletes its public or private backing objects and document rows. Publication history shows the revocation time.

Publish and revoke actions are written to the electoral log with the publication id, action, route, access, visibility, contest ids, and acting user. If discovery becomes stale after an infrastructure failure, use the results-publication refresh action; retry a revoked publication's revoke action to resume artifact cleanup.

Custom CSS

The published manifest captures `presentation.css` from the election event and each selected election. Event CSS loads first; the current election CSS loads second and therefore can override it. This works on both event and election routes and when the selected election changes.

Stable selectors use the `seq-results-` prefix. Important examples include:

- `.seq-results-page`, `.seq-results-summary`, `.seq-results-selector`, and `.seq-results-contest`;
- BEM-style elements such as `.seq-results-summary__pie`, `.seq-results-contest__title`, and `.seq-results-selector__selected-result`;
- fixed state classes such as `.seq-results-access--public`, `.seq-results-visibility-area_based`, and `.seq-results-contest--preferential`;
- entity classes such as `.seq-results-event--<id>`, `.seq-results-election--<id>`, `.seq-results-contest--<id>`, `.seq-results-area--<id>`, and entity-specific row/tab classes.

Entity values are reproducibly normalized by replacing characters outside letters, digits, `_`, and `-` with `-`; a missing area uses `global`. Prefer semantic classes for layout and entity classes for election-specific branding.

Deployment configuration

`RESULTS_PORTAL_URL` must be configured before application deployment so event creation/import can create the dedicated `results-portal` Keycloak client with only the results-site redirect pattern. The results portal uses `RESULTS_PORTAL_CLIENT_ID=results-portal`. `DISABLE_AUTH` is development-only and must not be enabled in deployed environments.

Support Materials

Support Materials are event-level documents (guides, sample ballots, instructional videos) that voters can review before voting. The Support Materials Policy controls whether they are shown at all, and whether reading them is a precondition for voting.

Policy

Configure the policy in the Support Materials accordion on the Election Event Data tab:

- **Off** hides Support Materials entirely in the Voting Portal and does not gate voting.
- **Optional** shows Support Materials as a generic link from the Ballot list; voting is never blocked.
- **Mandatory for Voting** requires each voter to open every listed Support Material and acknowledge having read them before any ballot's Start Voting control is enabled.

The policy applies to the whole Election Event; there is no per-election override. Election events created before this policy existed carry only the legacy

`presentation.materials.activated` boolean — **Optional** when **true**, **Off** when **false** or absent — until an administrator sets the policy explicitly.

Voter experience under Mandatory for Voting

When the policy is **Mandatory for Voting** and a voter has not yet acknowledged the materials for this Election Event, the Ballot list shows an instruction banner ("You must read the Support Materials before you can vote.") and disables Start Voting on every ballot. The Support Materials button remains available so the voter can open it.

On the Support Materials screen, each material must be opened at least once via Preview; a material is marked viewed the moment its preview dialog is closed. The "I have read the Support Materials" checkbox stays disabled until every listed material has been viewed, and Continue stays disabled until the checkbox is checked. Choosing Continue records the acknowledgment and returns the voter to the Ballot list with Start Voting enabled; choosing Back to ballot list leaves the Ballot list gated, since nothing is recorded until Continue succeeds.

Once acknowledged, the gate is skipped on later visits for that Election Event — the Support Materials button stays available to reopen the documents, but Start Voting no longer requires it.

Acknowledgment storage

Acknowledgment is recorded as a Keycloak user attribute (`support-materials-acknowledged`) on the voter's account, holding the list of Support Material document ids they confirmed reading. Since each Election Event has its own Keycloak realm, the attribute is inherently scoped per voter per Election Event — the same pattern used for `voted-channel`. Reopening materials after acknowledging does not clear it; there is currently no bulk-reset action for administrators.

The Election Event Voters list shows a "Support Materials Viewed" column — a plain boolean read from that same attribute — when the policy is `Mandatory for Voting`.

Requests and permissions

The Voting Portal reads and writes acknowledgment through two Hasura Actions backed by Harvest, both authenticated with the voter's own bearer token (`role: user`, `forward_client_headers: true`):

- `get_support_materials_acknowledgment(election_event_id)` returns the document ids the calling voter has already acknowledged.
- `acknowledge_support_materials(election_event_id, document_ids)` records acknowledgment for the calling voter, after checking the Election Event's policy is not `Off`.

Both actions derive the voter's identity from their own JWT claims — a voter can only ever read or write their own acknowledgment.

Both actions also require the voter's Keycloak session to carry a dedicated `ack-support-materials` realm role, checked the same way ballot casting checks the voter's `user` role. This role must exist in the Election Event's own realm (`tenant-<tenant_id>-event-<election_event_id>`) and be granted to the `voter` group — without it, every voter is rejected regardless of the Support Materials Policy.

Adding the role in the Keycloak Console

1. Open the Keycloak Admin Console and switch to the Election Event's realm (`tenant-<tenant_id>-event-<election_event_id>`).
2. Under **Realm roles**, click **Create role**, set **Role name** to `ack-support-materials`, and save.
3. Under **Groups**, open **voter**, go to its **Role mapping** tab, click **Assign role**, filter by realm roles, select `ack-support-materials`, and click **Assign**.
4. Voters who are already signed in keep whatever role set was in their current token — they need to log out and back in (or wait for their token to refresh) before the new role takes effect.

Create and set google meet credentials

Overview

Create google meet credentials and set them in the Admin portal.

Note: This configuration must be done via the google cloud console.

Configuration Steps

Google Cloud Console Setup

1. Go to [Google Cloud Console](#)
2. Create a new project or select an existing one. You must have superadmin access to set up the domain-wide delegation later on.
3. Enable the Google Calendar API.
4. Navigation menu > APIs and services > Credentials.
5. Create a Service account.
6. Edit > Keys > Add key. Then download the json file.
7. Add authorized domains for your application.
8. Paste the json file containing the Service Account key into Settings > Integrations > Google Calendar Service Account key.

Set up domain-wide delegation for a service account

1. Follow the [steps in this link](#)
2. Add the scopes for the authorized client ID: <https://www.googleapis.com/auth/calendar>,
<https://www.googleapis.com/auth/calendar.events>

3. Paste the authorized email address into Settings > Integrations > Google Calendar Authentication Email. **Note:** This email will appear as the organizer of the meeting.

Adding User Attributes to Keycloak

Overview

The system supports adding additional user attributes that will appear as new fields in the user data. Once configured, these attributes will appear in the Add or Edit actions in the Admin Portal's Voters tab.

Note: This configuration must be done via the Keycloak's admin console.

Configuration Steps

1. Log in to Keycloak and select the realm of the election event you want to edit.
2. Navigate to **Realm settings** > **User profile** > **Create attribute**.
3. Give an attribute name, such as `sex` in the first example.
4. Set the display name as `${sex}` if you want to override the translations in **Localization** > **Realm overrides**.
5. Continue configuring Annotations and other parameters (see examples below).

For the full list of annotations and what each one does to the rendered field, see [Configuring Login and Registration Fields](#).

Supported Attribute Types

The following attribute types are supported:

Sex

To enable sex selection:

1. In **Annotations** > **Add annotation**, set Key: `Input type`, Value: `select`
2. **Add Validator** > **Validator type**: `options` and add the desired options (e.g., M, F)

3. Optionally describe each option, so that whoever edits a voter does not have to interpret the stored values. In **Annotations > Add annotation**, set Key: `inputOptionLabels` and a JSON value mapping each option to its description, such as `{"M": "Male", "F": "Female"}`. A description written as `${sex_male}` is resolved through the Admin Portal's own localization overrides (see below) and falls back to a readable form of the key when there is no override. The `inputOptionLabelsI18nPrefix` annotation is honoured on the voter-facing login forms only; the Admin Portal ignores it, because it reads a different set of translations.

The dropdown in the Admin Portal shows the stored option followed by its description, for example `M - Male`, so the value written to the voter stays visible.

Birth Date

To show a date input field:

1. In **Annotations**, set Key: `Input type`, Value: `html5-date`
2. Add validation if desired.

Checkboxes

1. In **Annotations > Add annotation**, set Key: `Input type`, Value: `multiselect-checkboxes`
2. Add an `inputOptionLabels` annotation containing the option values and labels.

Ordering and Attribute Groups

The Admin Portal renders supported attributes in the order shown under **Realm settings > User profile**. Drag attributes into the required order in Keycloak; no tenant CSS is needed to control the semantic or keyboard order.

To organize related fields:

1. In **Realm settings > User profile**, create an Attribute Group.
2. Set the group's name, display header, and optional display description.
3. Assign attributes to the group while editing each attribute.

Only groups containing a field visible to the current administrator are rendered. Consecutive attributes in one group share a section. If the same group occurs in separate parts of the

profile, each consecutive run is rendered as a separate section so the configured field order is preserved. Attributes without a group are rendered in an unlabelled section.

Group headings use the Keycloak display header and fall back to the group's name. Values written as Keycloak expressions, such as `${personal_details}`, are displayed as readable labels. Group descriptions are displayed below the heading.

Customizing the Voter Editor with Tenant CSS

Tenant CSS configured under **Settings > Look and Feel** can target the following supported selectors:

- `.voter-editor` and `[data-mode="create"|"edit"]`: editor root and mode.
- `.voter-editor__groups`: collection of attribute sections.
- `.voter-attribute-group` and `[data-group-name="..."]`: one consecutive Attribute Group run. The data value is the raw Keycloak group name.
- `.voter-attribute-group__legend`: group heading.
- `.voter-attribute-group__description`: group description.
- `.voter-attribute-group__grid`: fields within a group.
- `.voter-field`, `[data-field-name="..."]`, `[data-input-type="..."]`, and `[data-required="true"|"false"]`: field wrapper and metadata. Field names are the canonical raw Keycloak names.

Step-owned fields also expose stable names: `enabled`, `area`, `password`, `confirm_password`, and `password_temporary`.

The following complete example creates lightly tinted, bordered cards for named Attribute Groups, uses a responsive two-column grid, keeps form controls white, and lets the postal address span both columns. The empty-name exclusion leaves the unlabelled section containing Step-owned fields unboxed.

```

.voter-editor .voter-attribute-group[data-group-name]:not([data-group-
name=""]) {
  min-width: 0;
  margin: 0 0 24px;
  border: 1px solid rgba(15, 5, 76, 0.16);
  border-radius: 12px;
  padding: 20px 22px 24px;
  background-color: rgba(15, 5, 76, 0.02);
}

.voter-editor .voter-attribute-group__legend {
  padding: 0 8px;
  color: #0f054c;
  font-size: 16px;
  font-weight: 700;
  line-height: 1.4;
}

.voter-editor .voter-attribute-group__description {
  margin-bottom: 24px;
  color: #666670;
}

.voter-editor .voter-attribute-group__grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  column-gap: 16px;
  row-gap: 20px;
}

.voter-editor [data-field-name="postal_address"] {
  grid-column: 1 / -1;
}

@media (max-width: 700px) {
  .voter-editor .voter-attribute-group[data-group-name]:not([data-group-
name=""]) {
    padding: 16px;
  }

  .voter-editor .voter-attribute-group__grid {
    grid-template-columns: 1fr;
  }
}

.voter-editor .voter-attribute-group[data-group-name]:not([data-group-
name=""]) .MuiInputBase-root {

```

```
background-color: #fff;  
}
```

Do not target generated MUI or Emotion class names such as `.css-abc123`; they are implementation details and can change during upgrades. `.MuiInputBase-root` in the example is a stable MUI component class, scoped beneath the stable voter group selector. CSS `order` can change only the visual order, not keyboard or screen-reader order, so configure field order in Keycloak instead.

Hiding an Attribute

An attribute that is carried on the voter but is not meant to be seen or edited can be marked hidden: in **Annotations > Add annotation**, set Key: `hidden`, Value: `true`.

The value must be exactly `true`. This is matched literally, so `TRUE` or a value with spaces around it does not hide anything — the voter-facing forms read the annotation the same way, and matching it any more loosely here would hide an attribute from administrators while voters still saw it.

A hidden attribute is left off the voter-facing enrollment and login forms, and off the Admin Portal's voter list and its create and edit forms — it is not shown as a column, is not offered in the columns selector or the filters, and is not shown as a field. The Approvals screens are not affected and still show it.

Hiding an attribute does not remove or alter it. Its value is still stored, still carried through when a voter is edited, and still included when voters are exported.

Hiding is therefore about keeping a form readable, not about restricting access: the value is still sent to the browser and still appears in exports. To control who may read or write an attribute, set its permissions in **Realm settings > User profile** instead.

Do not mark an attribute both hidden and required: creating a voter through the Admin Portal would then be impossible, since the field it insists on is one the form does not show.

Note this is different from setting the `Input type` annotation to `hidden`, which is Keycloak's own way of rendering an attribute as a hidden input on a form rather than keeping it off the form.

Protecting a Voter Attribute as Secret

An election-event attribute can be stored as an encrypted **secret voter attribute**. Use this for data that administrators or voter-level outputs need, but which must not be exposed in ordinary voter lists or default exports. An eligible custom secret can also serve as the credential for the explicitly configured **Multi-Attribute + Password** authenticators.

In **Annotations > Add annotation**, set:

Key	Value
<code>sequent.secret</code>	<code>true</code>

The value may also be the JSON boolean `true` when the profile is configured through an API. Secret attributes are supported only in election-event realms.

Before enabling the annotation:

- Restrict the User Profile attribute's view and edit permissions to administrators. Do not use a secret attribute in voter registration, identity-attribute login matching, token mappers, reconciliation, filters, sorting, uniqueness checks, or voter self-service. Keycloak stores ciphertext, so those features cannot use its original value. The supported authentication exception is [encrypted-attribute credential verification](#): the extension decrypts it server-side and verifies the existing password input. This requires `SECRET_ATTRIBUTE` policy and the matching master key in Keycloak; the annotation alone does not change login behavior.
- Do not configure a Keycloak value validator on the attribute. The `person-name-prohibited-characters` validator is the only supported exception because it accepts the encrypted envelope. A **Required field** rule is supported.
- Keep each plaintext value at or below 150 bytes. The encrypted envelope must fit the 255-character Keycloak attribute value column that voter imports and listings use.
- Protect or migrate any existing plaintext values before enabling the annotation. Existing values are not encrypted retroactively, and Step refuses to reveal an unencrypted value as a fallback.

Only custom voter attributes can be secret. The following identity and operational attributes cannot:

- `username`, `email`, `first_name`, `last_name`, `dateOfBirth`, `area-id`, and `tenant-id`
- `authorized-election-ids`, `authorized-to-election-alias`, and `permission_labels`
- `vote-weight`, `voted-channel`, and `disable-comment`
- `sequent.read-only.id-card-number-validated` and `sequent.read-only.mobile-number`

Step reads the secret-attribute configuration through a short cache, so a change to the annotation can take up to 30 seconds to be reflected in the voter list and editor. If the configuration is invalid (a forbidden attribute or a value validator on a secret attribute), voter lists still redact the attribute, but revealing, editing, importing, exporting and voter-level outputs refuse to use it until the profile is corrected.

Secret and hidden are different classifications. A hidden attribute is omitted from the editor but its plaintext can still reach the browser and ordinary exports. A secret attribute is encrypted at rest, removed from ordinary list/filter/export paths, and represented by a masked value in the voter editor.

Authorized operators may explicitly include decrypted secrets in [voter CSV](#) and [password-protected event exports](#). Anyone who can reveal or export a secret used for login can authenticate as that voter; grant secret-read permission accordingly. CSV imports accept plaintext and encrypt it for storage.

Combining `hidden=true` with `sequent.secret=true` keeps the field out of the editor, but it remains available to authorized secret exports and explicitly declared communication templates.

Users need separate permissions to reveal or modify secret values. See [Permissions](#) and [Create Voters](#).

Limiting the Number of Characters

To bound how much text an attribute accepts, **Add Validator > Validator type:** `length`, and set a minimum, a maximum, or both. Keycloak enforces the bounds when the record is saved.

The Admin Portal states the bounds under the field, so they are known before they are broken, and checks the value when the field is left. A value that breaks a bound marks the field and says which bound it broke, and the voter cannot be saved until it is corrected.

Bounds nobody types into are left unstated, to keep the form readable: a maximum in the hundreds, which Keycloak's own base attributes carry as scaffolding, and a minimum of one, which says only that the value is present. They are still checked, and still reported when broken.

A field with a maximum also stops accepting characters once it is reached, so the maximum cannot be exceeded by typing. That count is of the characters as typed, so a value padded with spaces can stop being accepted a little before the validator would object to it. Note that pasting a longer value into such a field keeps only what fits, without warning, and that a stored value already longer than the maximum can only be shortened, never extended — in that case the field reports the value as too long until it is brought within the bound. A minimum cannot be applied while typing at all, so it is only checked when the field is left.

By default Keycloak measures the value with leading and trailing spaces removed, and the Admin Portal measures it the same way. Setting the validator's `trim-disabled` option changes both.

A bound that is somehow reached anyway — a value written by an import, or an attribute the form does not show — is refused on save and reported naming each field and the bound it broke. Note that on Datafix election events the save is carried out by a background task, so its refusal is reported through that task rather than in the form.

Localization Overrides

To customize the display label of a user attribute, add a translation override via the Admin Portal under **Settings > Localization > Add**.

User attribute keys must use the prefix `usersAndRolesScreen.users.fields.` followed by the attribute name.

Example: to display the `personal_administrative_number` attribute as "PAN", add the following to the tenant's localization settings:

Key: `usersAndRolesScreen.users.fields.personal_administrative_number` Value: `PAN`

This works for any attribute name, including custom ones not present in the default translations.

Linking Multiple IdP Identities to a Single User via Custom Attribute

Overview

By default, Keycloak's identity brokering links an external Identity Provider (IdP) user to a Keycloak user on a 1-to-1 basis, usually by matching `email` or `username`. This tutorial explains how to configure the **Custom Attribute IdP Identity Linking** authenticator so that multiple IdP identities (e.g., different emails or subject IDs from the same or different IdPs) can be mapped to a single Keycloak user.

The authenticator reads a configurable claim from the incoming IdP identity and searches for a Keycloak user whose **multi-value custom attribute** contains that value. When exactly one match is found the new IdP identity is linked to the existing user automatically.

Prerequisites

- Access to the Keycloak Admin Console.
 - The `sequent.idp-linking-authenticator.jar` extension deployed in Keycloak's `providers/` directory (included in the Sequent Keycloak Docker image by default).
 - An existing External Identity Provider configured in the target realm.
-

Step 1 - Create the Custom User Attribute

The authenticator searches for users by a Keycloak user-profile attribute. You must create this attribute before configuring the flow.

1. In the Keycloak Admin Console, select the realm you want to configure.
2. Navigate to **Realm settings** → **User profile** → **Create attribute**.
3. Set the **Name** to `linked_idp_identities` (or any name you will use in the authenticator configuration).
4. Enable the attribute **multi-valued** (do not restrict it to a single value).
5. Optionally restrict read/write permissions so only administrators can manage it.
6. Click **Save**.

Tip: After creating the attribute you can pre-populate it for existing users via the Admin Console (**Users** → select user → **Attributes** tab) or via the Admin REST API.

Step 2 - Create the First Broker Login Flow

In this step we will create `sequent first broker login multivalue` flow directly.

1. Navigate to **Authentication** → **Flows**.
2. Click on create flow
3. Give the new flow a descriptive name such as `sequent first broker login multivalue`.

Step 3 - Add the Custom Authenticator to the Flow

1. Open the duplicated flow.
2. Click **Add step** inside the appropriate sub-flow.

3. Search for **Custom Attribute IdP Identity Linking** and add it.
4. Set the requirement to **REQUIRED** if you want the flow to fail when no match is found or **ALTERNATIVE** (the authenticator will call `attempted()` when no matching user is found, allowing the next step to run, if you want to create a user if not found for example).
5. Click  **Config** (gear icon) next to the new step to configure it:

Parameter	Description	Default
IdP Claim	Claim/attribute name to read from the incoming IdP identity. Use well-known names (<code>email</code> , <code>username</code> , <code>id/sub</code> , <code>firstname</code> , <code>lastname</code>) or a custom mapped attribute (e.g., <code>SAFE_ID</code>).	<code>email</code>
User Attribute	Keycloak user attribute (multi-value) to search for the claim value.	<code>linked_idp_identities</code>

6. Click **Save**.

Step 4 - Bind the New Flow to the Identity Provider

1. Navigate to **Identity Providers** and select the IdP you want to configure.
 2. In the **First Login Flow** (or **First Broker Login Flow**) dropdown, select the duplicated flow you created in Step 2.
 3. Click **Save**.
-

Step 5 - Map the Custom Claim from the IdP Token

If the claim you want to use (e.g., `SAFE_ID`) is not a standard OIDC/SAML field, add an attribute mapper on the IdP:

1. In the IdP configuration, open the **Mappers** tab.
2. Click **Add mapper**.
3. Set **Mapper type** to **Attribute Importer** (for OIDC) or **SAML Attribute** (for SAML).
4. Map the IdP claim name to the Keycloak attribute name that matches what you set in **IdP Claim** (e.g., `SAFE_ID`).
5. Click **Save**.

Behavior Summary

Scenario	Authenticator action
Configuration is missing	Passes to the next step (<code>attempted</code>).
The IdP claim is empty or absent	Passes to the next step (<code>attempted</code>).
No user found with the attribute value	Passes to the next step (<code>attempted</code>).
Exactly one user found	Links the IdP identity to that user and succeeds.
More than one user found	Fails the flow with <code>IDENTITY_PROVIDER_ERROR</code> to prevent ambiguous linking.

Logging In Without a Username (Attribute + Password or Secret)

Overview

By default, voters log in with a username and password. Some elections instead identify voters by attributes they already know - a date of birth, a national ID - without asking them to remember a separate username. This tutorial explains how to configure the **Multi-Attribute + Password Form** authenticator so voters log in with one or more configured user attributes plus a password, instead of a username. The password box can verify either the normal Keycloak password (the default) or an explicitly configured encrypted voter attribute.

The authenticator finds every user whose configured attribute(s) match the submitted value(s) - **all** configured attributes must match the same user - then checks the submitted password against those candidates. With the default `REJECT_AMBIGUOUS` match policy, login succeeds only when **exactly one** candidate's credential matches and that account is enabled and not locked out.

A single attribute like date of birth is not unique on its own (many voters share a birth date). This still works: the authenticator collects every user with that birth date as candidates and relies on the password to uniquely identify one of them. Configuring a second attribute (e.g. also a national ID) narrows the candidate set before the password check, and is recommended whenever a second identifying attribute is available.

Prerequisites

Optional: use an encrypted voter attribute as the credential

The default **Credential verification policy** is `PASSWORD`; existing flows do not change. To authenticate with a secret voter field instead of a Keycloak password:

1. Configure a custom User Profile attribute, such as `mysecret`, with `sequent.secret=true`, then populate it using the authorized voter editor or CSV importer. Do not enter plaintext directly into Keycloak's attribute editor.
2. Provision the same 64-character hex master key used by Harvest/Windmill in Keycloak's `MASTER_SECRET` environment variable. The supplied Compose configurations pass the single `MASTER_SECRET` deployment variable to all three services. If Harvest/Windmill use an external vault, inject that vault's existing master key into Keycloak, not a newly generated key. Recreate Keycloak after provisioning the environment variable. Receiving the key does not enable encrypted-attribute login by itself; ordinary `PASSWORD` verification remains the default.
3. On this authenticator's configuration, set **Credential verification policy** to `SECRET_ATTRIBUTE` and **Encrypted credential attribute** to `mysecret`. Keep ordinary identifying attributes in **User attributes to match**, not the secret attribute.
4. Keep **Multiple-candidate match policy** at `REJECT_AMBIGUOUS`. `FIRST_MATCH` is rejected in this mode. For IVR, set the same two options on **Multi-Attribute + Password Direct Grant**; its existing single `kind=secret/maps_to=password` input carries this credential.

For example, to ask for a date of birth and use `mysecret` as the password:

Authenticator setting	Value
User attributes to match	<code>dateOfBirth</code>
Credential verification policy	<code>SECRET_ATTRIBUTE</code>
Encrypted credential attribute	<code>mysecret</code>
Multiple-candidate match policy	<code>REJECT_AMBIGUOUS</code>

Use the exact attribute **name**, not its display label. The secret is the credential, not a lookup attribute: do not add `mysecret` to **User attributes to match**. The [secret-attribute setup guide](#) explains the required annotation, permissions and validator restrictions.

The form remains `login.ftl`, and voters enter the credential in the existing `password` field. Credential placement, masking/show-hide, autofocus, and `credential-input-policy: "pattern"` keep their existing behavior. Pattern formatting is presentation: store the exact submitted credential (for digit patterns, the digits, not visual separators). Do not add a second input. A Keycloak password is not required or consulted in this mode; password-reset and password-policy operations still manage the separate Keycloak password and do not replace this encrypted attribute.

Verification decrypts only the selected attribute of bounded candidates, binds each envelope to the tenant/event/voter/attribute, compares fixed-size SHA-256 digests in constant time, and examines all stored values. It does not trim or change credential case. Missing, malformed or tampered envelopes, missing/wrong keys, disabled/locked accounts and ambiguous matches fail closed without password fallback. At most 100 stored values per attribute and 150 UTF-8 bytes per submitted credential are accepted. Existing tuple throttling and Keycloak account brute-force protection remain active; guessed credentials are never included in the tuple throttle key.

Treat this field as a login credential: anyone authorized to reveal or export it can impersonate its voter. Choose high-entropy values, restrict secret-read/write access, enable realm brute-force protection, and avoid sending it in report previews, logs or untrusted message transports. Email/SMS delivery remains readable; declaring a secret does not encrypt the message. The explicit [Console test transport](#) also prints decrypted values, so use only synthetic credentials when that transport is enabled.

Standard prerequisites

- Access to the Keycloak Admin Console.
- The `sequent.message-otp-authenticator.jar` extension deployed in Keycloak's `providers/` directory (included in the Sequent Keycloak Docker image by default).
- The user attribute(s) you want to log in with must already exist in the realm's **User Profile** - see [Adding User Attributes to Keycloak](#). If an attribute is configured there with **Input type** `html5-date` (e.g. a date of birth field), the login form automatically renders it as a native date picker too, matching what voters already see at registration. Most other User Profile annotations apply the same way; the page-specific exceptions are listed in [Configuring Login and Registration Fields](#).

- **Date-valued attributes must be stored as** `YYYY-MM-DD` (e.g. `1990-01-05` for January 5, 1990) - the same format the browser's native date picker always submits, so no reformatting is needed on the browser path.
-

Step 1 - Create a Browser Flow with the New Authenticator

1. Navigate to **Authentication** → **Flows**.
 2. Duplicate an existing browser flow (e.g. the realm's default `browser` flow) or create a new one, and give it a descriptive name such as `attribute password login`.
 3. Inside the appropriate sub-flow, click **Add step**.
 4. Search for **Multi-Attribute + Password Form** and add it.
 5. Set the requirement to **ALTERNATIVE** (to offer it alongside other login methods already in the flow) or **REQUIRED** (to make it the only way to log in on that flow).
-

Step 2 - Configure the Attributes to Match

1. Click  **Config** (gear icon) next to the new step.
2. Give the config a descriptive alias, e.g. `attribute-login-dateOfBirth`.
3. Under **User attributes to match**, click **+ Add** and enter each user attribute name to require, e.g. `dateOfBirth`, then **+ Add** again for `nationalId` if you want a second attribute (all listed attributes must match the same user). These must be existing User Profile attribute names (see Prerequisites).
4. Optionally adjust the DoS-mitigation settings below (sensible defaults are pre-filled - see [Denial-of-Service Considerations](#)):
 - **Max candidates per request** (default `10`)
 - **Max failures per attribute-value combination** (default `10`)
 - **Failure window (seconds)** (default `60`)
 - **Max user-store rows per attribute lookup** (default `5000`)
5. Leave **Honor User Profile required attributes** off to keep every attribute listed above mandatory. Enabling it makes each field follow its User Profile **Required field** setting - see [Required fields](#).

6. Leave **Multiple-candidate match policy** at its default, `REJECT_AMBIGUOUS`, unless you have read and understood [Multiple-Candidate Match Policy](#) below - the alternative, `FIRST_MATCH`, is only safe when password uniqueness across every possible candidate is guaranteed.
 7. Leave **Existing browser session policy** at `KEEP` for normal login flows. For a shared-device flow where each visit must allow a different user to sign in, select `TERMINATE_BEFORE_LOGIN`. Opening that login form then ends the user session attached to the browser before credentials are submitted. This policy works with both `PASSWORD` and `SECRET_ATTRIBUTE` credential verification.
 8. Click **Save**.
-

Step 3 - Bind the Flow to a Client

1. Navigate to **Clients** and select the client this login page applies to (e.g. `onsite-voting-portal`).
2. Open the **Advanced** tab → **Authentication flow overrides**.
3. Set **Browser Flow** to the flow you created in Step 1.
4. Click **Save**.

This keeps the change scoped to the client(s) you explicitly bind it to - every other client keeps using the realm's default browser flow unchanged.

Step 4 - Optional Page-Level Settings

Two realm attributes change how the login page this flow renders behaves. Both are set on the election event realm rather than on the authenticator, and neither is required:

Realm attribute	Default	Effect on this login page
<code>credential-field-position</code>	<code>LAST</code>	<code>FIRST</code> moves the password or PIN box above the attribute fields and gives it the page's initial focus - useful when a PIN from a voter letter is the main thing being entered.

Realm attribute	Default	Effect on this login page
login-validation-policy	BROWSER	SERVER_ONLY stops the browser from rejecting badly formatted values, so every submission reaches this authenticator and gets its generic message instead.

Configuring Login and Registration Fields covers them in [Putting the credential first](#) and [Who reports invalid formats on the login page](#), and explains where realm attributes are edited.

Behavior Summary

Scenario	Authenticator action
A configured attribute field is left blank	Generic "invalid credentials" error (no lookup performed).
No user matches all configured attributes	Generic "invalid credentials" error.
Exactly one candidate, correct password	Login succeeds.
Exactly one candidate, wrong password	Generic "invalid credentials" error - this attempt is counted toward that account's Brute Force Detection lockout, same as a standard login.
Exactly one candidate, currently locked out by Brute Force Detection	Generic "invalid credentials" error; the locked account cannot log in.
Multiple candidates share the configured attribute(s), and the password matches exactly one	Login succeeds as that user.

Scenario	Authenticator action
Multiple candidates match the password (or none do)	Generic "invalid credentials" error - see the brute-force note below.

Authentication failures use a generic message rather than identifying the incorrect attribute or credential. Dummy password-hash work on early rejection paths helps reduce timing differences; it does **not** guarantee identical response times for an entire login request. In `SECRET_ATTRIBUTE` mode, fixed-size digest comparison is constant-time, while database lookup, the number of candidates and decryption still contribute to total request time.

Note on brute-force protection: Keycloak's built-in per-account lockout only engages once resolution narrows to a single candidate - the same account that ends up locked out is also the one whose failed attempts get counted, matching how the standard username/password form behaves. When more than one candidate still shares the configured attribute(s), there is no single account a failed attempt can honestly be attributed to, so the counter can't engage for that specific request. In `PASSWORD` mode, locked accounts are excluded before password verification. In `SECRET_ATTRIBUTE` mode, enabled but locked accounts still participate in ambiguity detection: locking one account must not make a shared secret authenticate as another voter. A locked account itself can never authenticate. Configuring more attributes narrows the candidate set before the password check, making the single-candidate (fully protected) case the common one; keep **Brute Force Detection** enabled at the realm level regardless.

Voters who share the configured attribute(s) can log in at the same time without blocking each other: while several candidates match, their lockout state is read without reserving them in Keycloak's brute-force protector, and only the account that authenticates is checked through it. Developers can verify this against a running environment with `packages/keycloak-extensions/message-otp-authenticator/src/test/integration/concurrent-shared-dob-login.py`.

Denial-of-Service Considerations

Because this authenticator matches on attributes rather than a unique username, a single request can legitimately resolve to many candidates (e.g. every voter born on a common date) - and each candidate normally costs one password-hash comparison to rule out. Three independent settings bound that cost per request, on top of Keycloak's standard Brute Force Detection:

- **Max candidates per request** (`maxCandidates`, default `10`): once the configured attributes match more candidates than this, the request fails generically without checking any of their passwords. This bounds the worst-case number of password hashes a single request can force, regardless of how many voters share the submitted attribute value(s).
- **Max failures per attribute-value combination / Failure window (seconds)** (`tupleMaxFailures` / `tupleFailureWindowSeconds`, defaults `10` / `60`): failures are also counted per distinct combination of submitted attribute values, independent of any single account. This closes a gap that per-account Brute Force Detection can't cover on its own: when a request matches more than one candidate, Keycloak has no single account to attribute the failure to, so its lockout counter never engages for that request - an attacker could otherwise repeat a common attribute value (e.g. a shared date of birth) indefinitely at full cost. Once a combination's failures reach the configured maximum within the window, further attempts against it are rejected without any user lookup at all, until the window elapses or a matching request succeeds (which clears the count). This throttle is tracked cluster-wide, so it can't be evaded by spreading requests across Keycloak nodes.
- **Max user-store rows per attribute lookup** (`maxAttributeLookupResults`, default `5000`): a hard ceiling on how many rows the underlying user-store query may return, applied before any candidate is even loaded into memory. This is deliberately much larger than **Max candidates per request** - it exists only to stop a truly pathological match (e.g. an attribute value shared by most of the realm) from pulling an unbounded number of rows from the database, not to replace the tighter candidate cap above. Keep it well above the largest realistic combined match count you'd expect a legitimate voter lookup to produce; setting it too low can make configuring a second identifying attribute (the recommended lever below) less effective at narrowing large buckets.

Raising **Max candidates per request** trades this protection for looser matching (useful if a single attribute alone is expected to match unusually large groups); lowering the failure threshold or widening the window trades convenience for tighter DoS protection. Configuring a second identifying attribute (see [Step 2](#)) is generally the better lever - it keeps genuine candidate sets small without needing to loosen these guards.

If PINs or passwords used with this authenticator are short (e.g. a numeric PIN), do **not** compensate for guessability by weakening the password hash algorithm - that only makes offline cracking easier if hashes ever leak. Use the settings above instead; they bound CPU cost without touching hash strength.

Multiple-Candidate Match Policy

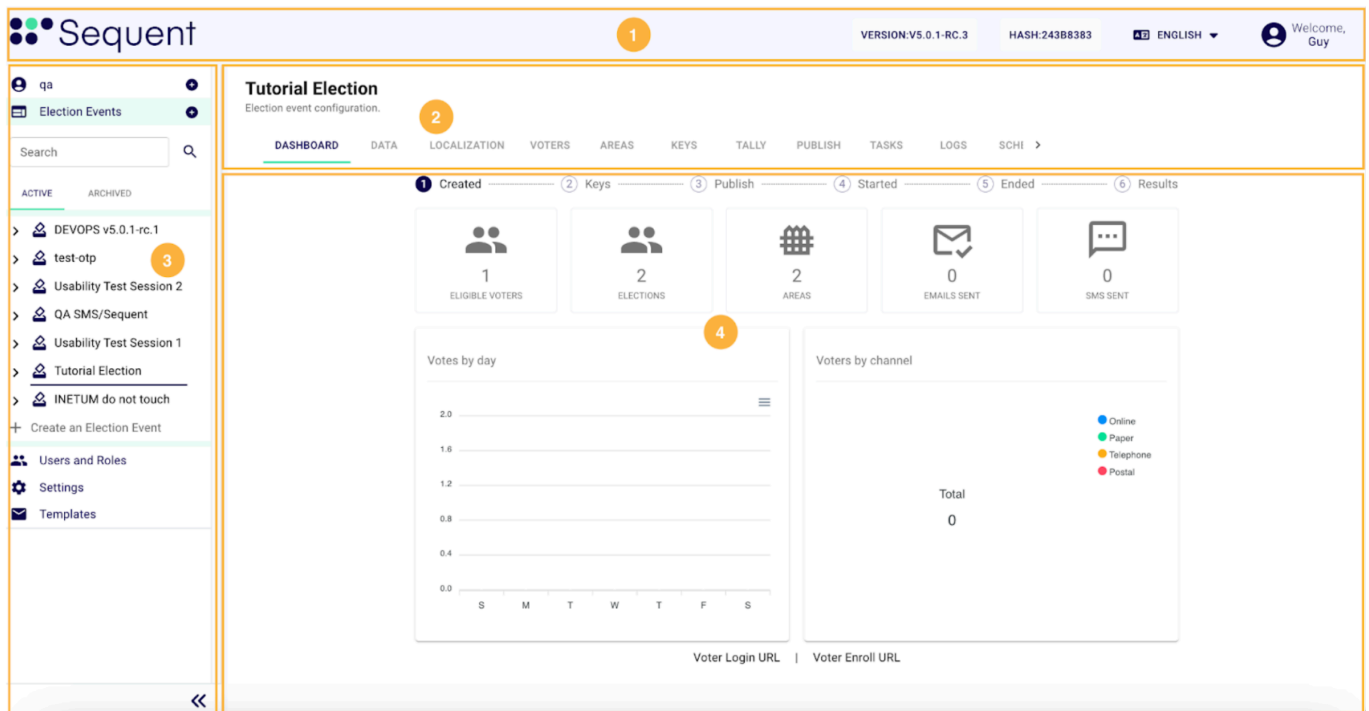
When more than one candidate shares the configured attribute value(s) (e.g. several voters born on the same date), **Multiple-candidate match policy** governs how the submitted password picks one:

`SECRET_ATTRIBUTE` mode requires `REJECT_AMBIGUOUS`; the `FIRST_MATCH` option below applies only to ordinary `PASSWORD` verification.

- `REJECT_AMBIGUOUS` (default): checks every candidate's password. Only succeeds if the submitted password matches **exactly one** of them; if it matches more than one, the request fails generically, the same as if none had matched.
- `FIRST_MATCH`: succeeds as soon as **any** candidate's password matches, without checking whether another candidate would also have matched. Cheaper (stops at the first hit instead of hashing every candidate), but only correct under one condition.

 **Security warning:** `FIRST_MATCH` is only safe to enable when **password uniqueness across every candidate a request could ever match is guaranteed** - for example, passwords assigned centrally and never reused, with no possibility of two voters who share an attribute value also sharing a password. If two candidates in a matched set share the same password, **which one authenticates is unspecified** - meaning one voter could end up authenticated as a different voter's account. If you cannot guarantee password uniqueness across candidates, leave this at `REJECT_AMBIGUOUS`.

Enabling `FIRST_MATCH` does not change anything about the DoS mitigations above - the candidate cap and per-tuple throttle still apply exactly the same way regardless of match policy.



Basic Navigation

The system's user interface (UI) is divided into four main areas, each serving a specific function:

1. **Header (Top Bar)**
2. **Navigation Tabs (Under Election Event Title)**
3. **Side Menu (Left Sidebar)**
4. **Main Content Area (Data Display)**

1. Header

The header, located at the top of the screen, provides global controls and essential information that are always accessible. This area includes:

- **System Logo and Version Number:** The logo of the system, along with the current version number, is displayed prominently for easy identification.
- **Language Selector:** A dropdown menu that allows users to switch between different languages, ensuring that the system is accessible to a broader audience.

- **Account Icon:** An icon that provides access to user account settings and the option to log out of the system. This is where users can manage their profile information and security settings.
-

2. Navigation Tabs

The navigation tabs under the selected entity title let you easily browse different sections, each showing specific information. Each entity has several unique tabs designed for its specific functions and data. This setup makes it easy to find and manage all the details related to each entity.

3. Side Menu

The side menu, located on the left of the screen, provides easy access to **Election Events**, **Settings**, **Users**, **Roles**, and **Communication Templates**. This omnipresent menu ensures seamless navigation across all screens, allowing you to quickly access system settings and manage Election Events.

This section of the screen is one of the most critical areas of the system. It contains the **Election Event tree**, where you can access and create **Elections**, **Contests/Questions**, or **Candidates/Answers**.

To facilitate navigation, a **search field** is provided that allows you to search within the election event section and its entire tree structure for any entity name, whether it be a Question, Candidate, or Election.

Election Events are categorized into **Active** and **Archived** for better organization. This allows users to archive election events that have been finalized or used for testing purposes, maintaining a clean tree structure with only the events currently in progress.

4. Main Content Area

The main content area, situated in the center of the screen, displays the currently selected section, whether it is an **Election Event**, **Settings**, or **Communication Templates**,

among others.

Each section can contain additional tabs that provide more detailed options and settings related to the chosen topic. This layout ensures that users have a comprehensive view and easy access to all relevant functionalities and information.

Election Event Dashboard Overview

At the top of the **Data Display** section on the dashboard of an Election Event, there is a **Step Crumb** that indicates the status of the election event. These statuses are **sequential** and described below:

Election Status Steps

- **Created**

The initial stage where the election event is set up. Administrators configure:

- Ballot designs
- Voters
- Areas (districts/precincts)

- **Keys**

After configuration, the administrator executes a **Key Ceremony**, generating cryptographic keys. Once complete, the election status changes to **Keys**.

- **Publish**

Once keys are generated:

- Administrators **publish** ballot styles to the voting portal.
- This is comparable to printing paper ballots.
- Voters accessing the voter portal can now view the election.
- Administrators may publish multiple times during edits.

- **Started**

The **voting period** has begun.

- **Ended**

The voting period has **concluded**.

- **Results**

- The **Tally Ceremony** is executed.

- Previously generated cryptographic keys are used to **decipher encrypted ballots**.
 - **Results are made available** to administrators.
-

Metrics

- **Eligible Voters**

Total number of voters imported into the system, including those not yet enabled.

- **Elections**

Number of individual elections within the event.

- **Areas**

Number of geographic areas involved in the event.

- **Emails Sent**

Total number of emails sent to voters.

- **SMS Sent**

Total number of SMS messages sent to voters.

Charts

- **Votes by Day**

A graphical representation of **daily voting activity** per ballot cast.

- **Voters by Channel**

A pie chart showing the distribution of voters (who voted on at least one ballot) by voting channel:

- Online
 - Paper
 - Telephone
 - Postal
-

This dashboard provides administrators with a **clear overview** of the election event's progress and key statistics, ensuring **efficient management and oversight** throughout the election lifecycle.

Data

The Data tab is similar across multiple entities in the system (Election Events, Elections, Contests, and Candidates). In this tab, you can configure the main values of each entity. Specifically for Election Events, all related data can be managed here.

Actions buttons in the Election Event Data Tab

- **Export:** Export election event data.
- **Import candidates:** Import candidates for the election event.
- **Google meet:** Generate google meet link and create event in google calendar.

Sections in the Election Event Data Tab

Each section serves a specific purpose and provides a comprehensive breakdown of information:

- **General:** Includes basic details.
- **Dates:** Start and End dates of the election event.
- **Language:** Supported languages for this event.
- **Ballot Design:** Custom ballot features including design, logos, links, and more.
- **Voting Channels Allowed:** Applicable voting methods.
- **Custom URLs Prefix:** Define custom URLs for the Voting / Enrollment portals and SAML endpoint.
- **Support Materials:** Documents available in the Voting Portal for voters to review.
- **Advanced Configurations:** Enable system lockdown, Voting Portal session timeout, and forced logout.

Detailed descriptions of each section are provided below.

General

Set up basic details and configure multilingual names for your Election Event.

- **Languages Tabs:** Configure how your Election Event appears in different languages in the Voting Portal.
- **Name:** Enter the official name of your Election Event.
- **Alias (optional):** Internal alias used only in the system's side menu.
- **Description (optional):** Provide a description for your Election Event.

Language

Manage language options for your Election Event. The selected languages will be available for elections within this event.

- Use radio buttons to select the languages available.
- Set the default language by selecting **Default** next to the appropriate language.
- **Language Detection Policy:** Affects the default language in the Voting Portal.
 - **Browser Detect:** The default language will be determined by the browser.
 - **Force Default:** The default language will be the one selected as **Default**.

Ballot Design

Manage how the ballot appears in the Voting Portal.

- **Disable Ballot Audit Support:** Enable or disable the ability for voters to verify ballot encryption.
- **Skip Election List Screen:** Skip election selection in the portal.
- **Show User Profile:** Show user profile information in the Voting Portal.
- **Show Cast Vote Logs Tab:** Policy to enable the CastVote Immutable logs in the Ballot Locator.
- **Logo URL (optional):** Provide a link to a logo to display.
- **Redirect Finish URL (optional):** Redirect users to a URL after completing voting.
- **Custom CSS:** Apply custom styles to the ballot design. Ballot error and warning messages expose stable CSS classes that can be targeted here — see [Styling Ballot Errors and Warnings with Custom CSS](#).

Voting Channels Allowed

Define the voting methods available for this Election Event.

- Use radio buttons to enable applicable voting channels.
- **Online:** Main remote voting channel. Starting/closing Online also governs Early Voting lifecycle (see below).
- **Kiosk:** In-person device-based voting. Its status is independent from the others.
- **EARLY_VOTING:** Enables an early voting period prior to the Online voting window for voters whose Areas have Early Voting enabled.
 - Appears only if allowed here; when started in Publish, only voters assigned to Areas with the Early Voting policy enabled can vote.
 - Online channel governs Early Voting:
 - When Online is started or closed, Early Voting will automatically close if it was enabled.
 - Early Voting cannot be started once Online voting has been started (and thereafter).
 - If a channel that is already started is later manually disallowed in this section, no immediate action is taken; action buttons remain disabled until the channel is allowed again.

Custom URLs Prefix

Create custom URL prefixes for the Voting and Enrollment portals, and SAML endpoint.

- Input the desired prefix for each endpoint.

Examples:

- Input "myelection" into **Login:**
URL becomes `https://myelection.sequent.vote`
- Input "enrollment" into **Enrollment:**
URL becomes `https://enrollment.sequent.vote`

Support Materials

Provide documents that voters can access in the Voting Portal.

- **Support Materials Activated:** Enable or disable additional support documents.
- **Add:** Attach documents with the following fields:
 - Title
 - Subtitle
 - **Is Hidden:** Controls visibility in the portal
 - Drag and drop the file
 - Save

Advanced Configuration

Configure advanced system behaviors for this Election Event.

- **Contest Encryption Policy:**
 - **Single Contests:** Encrypt contests individually.
 - **Multiple Contests:** Encrypt multiple contests together to enable ballot-level audit.
- **Lockdown Status:** When enabled, no changes can be made to this Election Event. This action is irreversible.
- **Voting Portal Countdown Policy:**
 - Define the session timeout duration in seconds.
 - Configure the countdown warning and logout alert thresholds.
- **Keys/Tally Ceremonies Policy:**
 - Allow for the automatic generation of keys and tallies, eliminating the need for trustees involvement.
- **Weighted Voting Policy:**
 - **Weighted Voting for Areas:** Enable weighted voting for areas.
 - **Weighted Voting for Voters:** Give each voter their own weight, so that a voter with weight `w` contributes `w` votes. Add a `vote-weight` column to the imported voters csv holding a whole number between 1 and 100000. A voter with no column, or with a blank cell, votes with weight 1. Near spellings that differ only in case or in `_` and `.` — `vote_weight`, `voteWeight`, `vote.weight` — are rejected rather than imported, because they would be stored under a name the tally does not read. A column named simply `weight` is **not** rejected: it is the name of the unrelated area weight, so it imports as an ordinary attribute and every voter tallies at 1. The column is named after the `vote-weight` voter attribute it becomes, so an exported voters file can be edited and re-imported unchanged. Cannot be combined with Delegate Voting, and every contest in the election event must use the Plurality at

Large counting algorithm — counting a ballot more than once has no defined meaning for the others, so the tally refuses to run. Approved tally sheets are refused for the same reason: a tally sheet reports a count of paper ballots and has nowhere to record a weight, so its votes would be added to the weighted totals at a weight of one each, which decides contests rather than merely under-counting them. Decoded ballots cannot be included in the published results either, for the reason in the warning below. Check both before ballots are published: neither the counting algorithm nor a published ballot can be changed once voting has begun. The weights of everyone who votes in one area of a contest must also add up to no more than 1000000, so a small number of voters on very large weights is rejected even though each weight is individually allowed. **This is checked when the tally runs, which is after voting has closed.** The import writes a warning to the server log when the weights in one file exceed it, but nothing surfaces that in the Admin Portal, and it counts only that file rather than everything already in the area. Two hundred and one voters each carrying 5000 exceed it while every one of them imports cleanly. Plan around it before voting opens: the remedy afterwards is to rescale every weight, which changes the result. The limit does not come from the mix, which under this policy is at most one ciphertext per voter per batch however large the weights are; it comes from the tally expanding each batch's votes by that batch's multiplier afterwards.

Turnout figures under this policy count voting power rather than voters: the eligible-voter census and the cast-ballot total are sums of weights, so they will not match a headcount shown elsewhere. Ballots with no matching voter are the exception — they have no voter and so no weight, and are counted one each, which mixes units into the cast-ballot total and into the auditable ballot percentage. The Total voters and Non-voters figures on the Tally screen are voting power under headcount labels.

If any area still carries a Weight from a previous Weighted Voting for Areas configuration, the tally is refused until it is cleared, because the two weightings would multiply. **Check this before publishing ballots.** Selecting this policy hides the Weight column and field without clearing what they hold, so an area can keep a weight that nothing in the interface shows, and the refusal arrives only when the tally is created — after voting has closed, when the ballots can no longer be republished and there is no remedy left. Clearing it means switching back to Weighted Voting for Areas, which makes the Weight field visible again, clearing the Weight on each area, switching to Weighted Voting for Voters, and only then publishing the ballots.

⚠ SET THE WEIGHTS BEFORE EXTRACTING BALLOTS FOR A TALLY

Extracting the ballots for a tally writes them to the bulletin board, which is append-only. If a vote weight is changed after that, or a voter who had voted is disabled or removed, the board no longer matches and the tally refuses rather than publishing a total that counts some voters at their old weight and their new one at once. Putting back exactly what changed clears it. Nothing else does: the ballots already on the board will be mixed and published whatever the tally counts, so weights extracted wrongly cannot be corrected afterwards, and a voter whose account was deleted rather than disabled cannot be restored at all — that area can then never be tallied.

The refusal is not a guarantee that the weights used are the ones configured now. A weight raised so that it only adds batches the interrupted run had not yet written is applied as raised, because nothing already on the board contradicts it. Get the weights right before starting the tally rather than relying on this to catch a change.

VOTER WEIGHTS ARE PUBLIC, AND RESULTS ARE ATTRIBUTABLE

A voter's weight is applied by splitting it into powers of two. A contest area is mixed as up to 17 batches, the batch at position n counting each ballot in it 2^n times, and a voter's ballot is placed in the batch for each power of two that adds up to their weight — weight 21 goes into the batches for 1, 4 and 16. No batch ever holds the same ballot twice, so nothing on the bulletin board repeats in a way that spells out a weight.

That is not enough to keep a weight private. The board is public so that the mix can be verified, the same ciphertext appears in every batch its weight selects, and each ciphertext is linkable to the voter who cast it, so adding up the batches a ballot appears in recovers that voter's weight exactly. **A voter's weight is therefore public, and it is public who holds it.** This is the flip side of a property some bodies want: published weights can be audited against the board.

The published per-candidate totals are sums over the weights of the voters who chose each candidate. Where weights differ from one another, that sum frequently identifies exactly who voted for whom — with distinct weights such as 1, 2, 4, 8 it always does. Turning off decoded ballots in the results does not prevent this; the totals alone are enough. Write-in answers are worse again, since an uncommon write-in appears with exactly the weight of the voter who wrote it.

A batch holding very few ballots publishes those voters' choices when it is decrypted, because a mix of one hides nothing. Which batches are small depends on how the weights are spread, not on how large they are: 999 voters on weight 2 and one on weight 3 leave the batch for 1 holding a single ballot. Extracting the ballots logs a warning naming any batch below five. It does not refuse, since by then voting has closed and there is no remedy left.

That warning is a floor, not a guarantee. A voter is exposed whenever the batches single them out at all — two batches whose voters differ by one person expose that person however large both batches are — and nothing detects that. Check the spread of weights before opening voting.

Do not use this policy where ballot secrecy is required. It suits bodies that already publish how each member voted, such as some shareholder or delegate votes. Where secrecy matters and voters fall into a small number of

weight classes, Weighted Voting for Areas gives weighting without this disclosure, because every voter in an area shares one weight.

 CAUTION

The `vote-weight` attribute must be declared in the election event realm's Keycloak user profile before use. Until it is, the weight column is missing from the voters export, the field does not appear when editing a voter, and editing a voter through the Admin Portal clears any weight that voter had. Importing weights and tallying them work regardless. Realm user profiles are not managed from this application, so declaring the attribute is an administrator step on the realm configuration.

- **Disabled Weighted Voting:** Disable weighted voting.
- **Delegate Voting Policy:**
 - Allows for voters to delegate their vote to another voter. An additional column needs to be included in the voters imported csv with the name `delegate-vote-to` with the username of the voter to delegate the vote to.

Localization

This section allows you to override portal text per language. Election-event overrides can target the Voting Portal, Ballot Verifier, Results Portal, or every public portal.

Steps

- **Select Language**

Choose the language for which the localization will be applied.

- **Add**

- **Portal scope:** Select the portal that should receive the override. Select **Global** only when the same key and wording should be used by every public portal.
- **Key:** Specify the translation key to customize, without a portal prefix.
- **Value:** Enter the text to be displayed for the selected element.

Actions

- **Edit** a Localization element or change its portal scope.
- **Delete** a Localization element.

Presentation fields

Portal scopes also apply to translated presentation fields such as an election-event, election, contest, or candidate name, alias, and description. Each application automatically uses its own scope. For example, the Voting Portal reads `votingPortal:description`, while the Ballot Verifier reads `ballotVerifier:description`.

For the selected language, presentation fields are resolved in this order:

1. The current portal's value, such as `votingPortal:description`.
2. The value for all portals, such as `global:description`.
3. The unprefixed presentation value, such as `description`.
4. The legacy database field, when present, for compatibility with older election events.

If no value exists in the active language, the same order is applied to the configured default language before using the legacy field.

Existing overrides

Overrides created before portal scopes were introduced are shown as **Legacy (Voting portal)**. They continue to affect only the Voting Portal, exactly as before. Editing and saving a legacy override promotes it to the explicit scope selected in the editor; no migration is required.

At the storage/API level, explicit scopes use a colon prefix, for example

`resultsPortal:resultsPortal.pageTitle`. The Admin Portal adds this prefix automatically. Known prefixes are `global`, `votingPortal`, `ballotVerifier`, `resultsPortal`, and `adminPortal`. Unprefixed election-event keys remain Voting Portal-only for backward compatibility. The Ballot Verifier and Results Portal ignore unprefixed keys.

Results Portal overrides are part of the published results snapshot. Republish the results after changing an override for it to appear on the published site.

Voters

This section displays the currently configured voters for this Election Event.

Overview

This tab displays and manages voters. The table lists all currently configured voters.

- **Columns:** Enable/Disable which columns are displayed in the table.
- **Add Filter:** Create a text filter by column.
- **Add:** Provide a voter's information and add them to the Election Event.
- **Import:** Import voters using a CSV file.
- **Export:** Export the list of voters as a CSV file.
- **Send:** Message voters using the **Send Notification** tab.
- **Custom Filters:** Use preset custom filters.
 - These filters can be defined under:

Election Event > Data > Advanced Configuration

Actions

The **Actions** column provides options to interact with voter records:

- **Send:** Send a notification to the voter.
- **Edit:** Modify voter details or change their assigned Area.
- **Delete:** Remove the voter from the system.
- **Manually Verify:** Confirm voter's identity (see below).
- **Change Password:** Update the voter's password (see below).
- **User's Logs:** View actions performed by the voter.

Adding and editing a voter

Fields are checked as they are left. One that is required and left empty, or whose value breaks a limit configured for that attribute, is marked and explains what it expects, and the voter cannot be saved until it is corrected. Where an attribute sets a limit worth knowing in advance, it is stated under the field before it is reached.

If a voter still cannot be saved, the editor stays open with the values as entered, so nothing has to be typed again, and reports why. Where the rejection concerns particular fields, each one is named along with the rule it broke — up to ten of them, with any beyond that counted rather than listed.

Limits come from the attribute's configuration in Keycloak; see [Adding User Attributes to Keycloak](#) for how they are set and which ones the portal states up front.

Send Notifications

Use this tab to send notifications to voters through various methods, schedules, and templates.

- **Audience:** Define who the notification is for.
 - **Schedule:** Set when the notification will be sent.
 - **Communication Template:** Choose a preset message template.
 - **Communication Method:** Email / SMS.
 - **Communication Type:** Type of content (if applicable).
 - **Template Alias:** Name of the preset template.
 - **Email Subject:** Only applicable for email.
 - **Message Body:** Plain or rich text to be sent.
-

Manual Verification

Confirm the voter's identity without requiring additional verification steps.

- A popup dialog will appear.
 - Scanning the generated downloadable QR Code allows the voter to:
 - Set their password
 - Verify themselves
 - Bypass the KYC process
 - Once complete, the voter will be eligible to vote.
-

Change Password

Change the voter's password.

- **Password and Repeat Password** must match.
 - Enabling the **Temporary** radio button will require the voter to change their password on next login.
-

User's Log

View logs of all actions performed by the voter.

Additional User Data Fields

You can add additional fields like the voter's birthday or sex. [Check out the tutorial for that.](#)

Important: Additional attributes for voters must be added before the enrollment process. For instance, if the sex attribute is not added, this trait will not be reflected in the reports and statistics.

External system (i.e. Datafix) voter-list reconciliation

Users with the `election-event-voter-list-reconciliation` permission can open the reconciliation wizard from the Voters tab. Run reconciliation only during an agreed external-system freeze window:

1. Upload the complete external-system reconciliation CSV. The first line must be the `#META` line and file channels must be uppercase.
2. Review the external-system and Sequent tables. If the external-system table is non-empty, download its patch, have the external system apply it, and upload the newly generated reconciliation file. Never apply the Sequent side first.
3. When the external-system table is empty, review the category totals and explicitly apply the Sequent changes.
4. Review the row-failure table. A completed apply can contain rows that were safely rejected; these are business-level reconciliation results, not a failed background task. Resolve them and retry the same `Sequence`.
5. Upload the last file again as a diff-only convergence check. Both tables must be empty before the round is considered converged.
6. Compare the source and patch hashes with the electoral logs before ending the freeze.

An Internet ballot that is still `in-progress` is deliberately reported as a row failure. During a hard-down external-system freeze it cannot resolve because the review beat must reach the external system. If all remaining failures have this cause, complete the freeze hash checks, restore external-system connectivity, wait for the review beat to resolve the ballots, and retry the same `Sequence`. Apply-time snapshot validation prevents that retry from overwriting voter data changed in the meantime.

Both the real-time `/unmark-voted` operation and file reconciliation clear the voted-channel marker. They only re-enable a voter when the corresponding `MarkVoted` operation owns the disable. An independent administrator disable and its comment are preserved, but the channel is still cleared so the voter does not remain permanently blocked in an unresolved external-system state. The administrator can then decide separately whether the account should be re-enabled.

The task result shows at most the first 1,000 apply-time row-failure details and always reports the complete failure count. Resolve the common cause and retry the same `Sequence` to reveal any remaining failures.

Capacity validation

Before production use, UAT must exercise the largest expected voter roll, including a first synchronization where most voters are additions. For a 100,000-voter test, record the generated envelope and audit-artifact sizes, backend peak memory, browser peak memory, wizard rendering responsiveness, and total Keycloak apply duration. These measurements determine the operational batch size and freeze-window duration for that deployment.

Areas

Represents geographical or organizational divisions within which elections are conducted. These can range from small precincts or wards to larger regions such as districts or states.

Table Options

- **Columns:** Enable or disable specific columns in the table view.
- **Add Filter:** Create a text-based filter for the table by column.
- **Add:** Manually add a new Area.
- **Import:** Import new Areas using a CSV file.
- **Export:** Export existing Areas to a CSV file.
- **Upsert:** Update or insert Areas using a CSV file.

Early Voting per Area

When adding or editing an Area, you can enable an option called **Early Voting** for that specific area. If enabled, voters assigned to that area will be allowed to vote as soon as the **EARLY_VOTING** channel is started for the Election Event.

- **Visibility:** The Early Voting option appears only if the **EARLY_VOTING** channel is allowed at the Election Event level (Admin Portal > Election Event > Data > Voting Channels Allowed).
- **Effect:** Enabling Early Voting on an area does not start voting by itself; voting becomes possible for those area voters when the Election Event's **EARLY_VOTING** channel is started in the Publish tab.
- **Import/Export:** The Early Voting setting is supported in CSV Import and Upsert operations, so you can manage this policy at scale.

Note: Kiosk voting status is independent. The Online voting channel governs Early Voting availability (see Election Event > Data > Voting Channels Allowed for details).

Keys

The Key Ceremony establishes the collective private key used to decrypt votes and publishes a corresponding public key for voters to encrypt their ballots. During this ceremony, trustees collaboratively generate fragments of the private key, ensuring no single party holds the full secret. Each trustee downloads, securely backs up, and verifies their key fragment. Only after all trustees complete these steps can the election proceed to stages such as ballot publication, voting, and tallying. This distributed approach guarantees that votes encrypted under the shared public key remain confidential and tamper-resistant, and that the combined private key—reconstructed only when a threshold of trustees collaborates—preserves integrity and availability of decryption throughout the election lifecycle.

Opening Key Distribution Ceremony

During the cryptographic key ceremony for an Election Event, trustees gather to generate essential cryptographic keys used to secure the voting process. This ceremony typically involves:

- **Key Generation:** Creating new keys according to the system's security requirements.
- **Backup Procedures:** Backing up each key to ensure redundancy and prevent loss.
- **Verification Steps:** Performing cryptographic tests and audits to validate key integrity and functionality.
- **Secure Storage:** Storing keys safely so they are ready for deployment during election events to encrypt and decrypt sensitive voting data.
- **Confidentiality & Authenticity:** Ensuring that all keys remain confidential and unaltered, maintaining trust in the election process.

Creating Cryptographic Keys

To initiate the Key Ceremony in the Sequent Online Voting System (OVS):

1. Select Election Event

- In the Administration Portal, choose the relevant Election Event.

2. Navigate to Keys Tab

- Go to the “Keys” section for that Election Event.

3. Start Key Ceremony

- Click **Create Key Ceremony** (or equivalent button/screen).

4. Automatic Key Ceremony (Optional)

- Only available for events with an 'Allow Automatic Ceremonies' policy.
- Enabling this setting automates the ceremony, so trustees are not required to take any action.

5. Configure Threshold & Trustees

- **Input Threshold:** Specify the minimum number of trustees required to reconstruct/decrypt (the threshold).
- **Select Trustees:** Check the trustees who will participate in this ceremony for the Election Event.

6. Create Keys Ceremony

- Confirm and submit to generate the keys.
- Approve any confirmation prompt if prompted.

7. Trustee Actions

- Each trustee must:
 - **Download** the generated encrypted private key.
 - **Secure** multiple backups of their key.
 - **Verify** the key with the system (see next section).

Note: In the UI, you may see indicators (icons or status) showing which trustees have completed download/backup/verification. For instance, a green icon indicates completion.

Trustee Cryptographic Key Distribution Process

Each trustee follows this procedure to download, backup, and verify their key. Once all trustees complete these steps, the Key Ceremony is finalized and the Election can proceed.

1. Log In & Select Election Event

- Log in to the Administration Portal.
- Select the relevant Election Event.

2. Access Key Ceremony Invitation

- Click on the ceremony's key action invitation, which may be highlighted (e.g., an orange box) or via a key icon.

3. Follow On-Screen Instructions

- Proceed through the guided steps for key distribution.

4. Download Encrypted Private Key

- Download the encrypted private key file provided by the system.

5. Create Secure Backups

- Make **multiple secure backups** of the encrypted private key (e.g., on separate secure storage/media).
- Confirm backups exist before proceeding.

6. Confirm Backup Creation

- Click **Next** (or equivalent) and confirm that backups have been created.

7. Verify Key with System

- Drag and drop (or upload) the encrypted private key file to the UI to verify it matches the expected key fingerprint.
- The system will confirm successful verification.

8. Completion

- Once all trustees have downloaded, backed up, and verified their keys, the Key Ceremony is successfully completed.
- The Election Event status will update to indicate that keys have been distributed and verified.
- A participating trustee can reopen the ceremony and select **Verify key** to check an existing backup again. This remains available after the ceremony completes and does not generate or download a new key.

Tally Sheet Imports

Tally sheet imports let election administrators upload precinct-level results from an external tabulator and review them as a batch before those results become active tally-sheet versions.

An import is election-event scoped. One uploaded file can produce many pending tally sheet versions across areas, contests, elections, and a selected voting channel. Saving an import does not change active results. Active tally-sheet results change only after the generated versions are approved.

In the Admin Portal, open an election event and go to **Tally**. The **Tally sheet imports** section appears above the tally ceremony list for users with `tally-sheet-import-view`.

Permissions

The feature uses these permissions:

- `tally-sheet-import-view` to see import history, import detail, generated import items, source-file metadata, and imported-version source links
- `tally-sheet-import-create` to upload, preview, and save imports for review
- `tally-sheet-import-review` to approve or disapprove a saved import
- `tally-recount-execute` to manually trigger a recount of a completed tally session

Supported File Formats

STEP uses a vendor-neutral canonical CSV format for tally sheet imports. ES&S Enhanced XML uploads are converted server-side into this CSV format before validation, preview, saving, review, and audit.

The canonical CSV is a long format with one row per field value:

```
channel,area_name,contest_external_id,field,candidate_external_id,value
PAPER,WARD 15 SUB 01,1,total_votes,,111
PAPER,WARD 15 SUB 01,1,total_valid_votes,,111
PAPER,WARD 15 SUB 01,1,implicit_invalid,,0
PAPER,WARD 15 SUB 01,1,explicit_invalid,,0
PAPER,WARD 15 SUB 01,1,total_blank_votes,,0
PAPER,WARD 15 SUB 01,1,census,,111
PAPER,WARD 15 SUB 01,1,candidate_votes,1,2
PAPER,WARD 15 SUB 01,1,candidate_votes,2,1
```

Rows are grouped into ballot boxes by:

```
channel + area_name + contest_external_id
```

The selected import channel must match every `channel` value in the CSV.

Each ballot box must include exactly one row for every scalar field: `total_votes`, `total_valid_votes`, `implicit_invalid`, `explicit_invalid`, `total_blank_votes`, and `census`. It must also include one `candidate_votes` row for every candidate in the matched STEP contest.

Uploaded files are stored as normal STEP documents. The UI and CLI send a SHA-256 hash of the source file when they upload it, and the import actions verify that hash before parsing or persisting the import. The import record stores both the original source hash and the canonical CSV hash used for validation and review.

Columns

`channel` is the STEP voting channel for the generated tally sheet versions, such as `PAPER`.

`area_name` must exactly match an existing area name in the election event.

`contest_external_id` must match the contest external ID configured in STEP. Because the canonical CSV format has no election column, **contest external IDs must be unique within an election event** for tally sheet import to resolve rows correctly. If an election event has multiple elections that reuse the same external ID for different contests, the import will fail with an ambiguous-match error — rename the duplicate external IDs before importing.

`field` must be one of:

- `candidate_votes`
- `total_blank_votes`
- `implicit_invalid`
- `explicit_invalid`
- `total_valid_votes`
- `total_votes`
- `census`

`candidate_external_id` is required only when `field` is `candidate_votes`. It must match a candidate external ID in the matched contest.

`value` must be a non-negative integer.

Vote Buckets

Candidate vote rows are summed into candidate results.

`total_blank_votes` is treated as part of valid votes.

`explicit_invalid` is for an intentional invalid-vote option in STEP. ES&S Enhanced XML does not have an equivalent concept, so ES&S imports set it to 0.

`implicit_invalid` is for invalid votes caused by tally rules, such as overvotes and non-blank undervotes.

The importer validates these invariants per ballot box:

```
total_invalid = implicit_invalid + explicit_invalid
total_valid_votes = sum(candidate_votes) + total_blank_votes
total_votes = total_valid_votes + total_invalid
total_votes <= census
```

ES&S Enhanced XML Mapping

For ES&S Enhanced XML uploads:

- precinct names are matched to STEP areas by exact area name
- contest `altId1` is matched to contest external ID
- candidate `altId1` is matched to candidate external ID
- only ES&S reporting group `1` is read: its values are summed per precinct into one result for the selected STEP channel, and all other reporting groups are ignored
- ES&S overvotes become STEP implicit invalid votes
- ES&S blank votes become STEP blank votes
- ES&S undervotes are treated with the overlap-safe rule `max(underVotes - blankVotes, 0)` before adding them to implicit invalid votes

Import Lifecycle

Preview validates the file and compares each incoming ballot box against the latest approved version for the same election, area, contest, and channel. The preview shows new, changed, unchanged, and validation-error counts.

Save creates an immutable import record and generated `PENDING` tally sheet versions for changed or new ballot boxes. Unchanged ballot boxes remain visible in the import detail but do not need duplicate generated versions.

Approve all approves the generated versions if the baseline approved tally sheets have not changed and no newer non-deleted version exists for any affected ballot box. If a conflict is found, approval is stopped and the conflicted rows are marked.

Disapprove all marks the generated versions and import items as disapproved. Existing approved tally sheet versions remain active.

Import records can have these statuses:

- `PENDING_REVIEW`
- `APPROVED`
- `DISAPPROVED`
- `FAILED_VALIDATION`
- `CONFLICTED`

Import items can have these statuses:

- `PENDING_REVIEW`

- `APPROVED`
- `DISAPPROVED`
- `CONFLICTED`

The first implementation reviews a whole import at once. There is no selected-row or partial approval flow.

If the election event automatic recount policy is enabled, approving an import starts recount tasks only for elections where approved import items generated new tally sheet versions. Previewing, saving, disapproving, or approving an unchanged-only import does not start a recount.

The source file remains linked from the import history for later download and audit.

Import Diff And Traceability

Each import item represents one ballot box, identified by election, area, contest, and channel. The detail view stores and shows two canonical per-ballot-box CSV snippets:

- `previous_csv`, rendered from the latest approved tally sheet at preview time
- `incoming_csv`, rendered from the imported source data

Rows are classified as:

- `NEW` when there is no approved baseline for the ballot box
- `CHANGED` when the imported content differs from the approved baseline
- `UNCHANGED` when the imported content matches the approved baseline

Only `NEW` and `CHANGED` rows generate pending tally sheet versions. Imported tally sheet versions keep an `import_id` reference so the version history can link back to the source import, show import status, and download the original XML or CSV source file.

CLI

The same flow is available in `step-cli` under `cli step tally-sheet`:

- `import-preview`
- `import-create`

- `import-review`
- `import-list`
- `import-show`
- `import-download-source`
- `convert-ess-xml`
- `recount`

Use `convert-ess-xml` to convert an ES&S Enhanced XML file into canonical CSV for offline inspection before upload.

Publish

To commit any changes to an Election Event, they must be published using this tab. Additionally, you can start, pause, and stop Election Events here.

All publications and their respective changes can be viewed in the table using the "eye" icon.

Table Options

- **Columns:** Enable or disable specific columns for display.
- **Add Filter:** Create text filters for the table by column.

Actions

Any action performed in this tab will effect all Elections associated with the Election Event.

- **Start Election:** Begin the Election Event.
- **Pause:** Temporarily halt the Election Event.
- **Stop Election:** End the Election Event.
- **Publish Changes:** Apply any changes to this Election Event.

Publish Changes / Preview

Changing settings in the Admin Portal will not apply them until they have been published. Publishing changes is done by selecting **Publish Changes** in the Publish tab.

Steps to Publish Changes

1. Select **Publish Changes**.
2. Enter your password (sensitive operations are password protected).
3. Review the changes.
4. (Optional) Select **Preview** to open the Voting Portal changes in a separate tab (per Area).

5. Select **Publish Changes** again to apply the changes.

Additional Actions

- **Regenerate:** Regenerate the diff files.
- **Export:** Export the diff files.

Tasks

Monitor the execution of previous tasks and access detailed information about system actions. This feature provides insights into task progress, system behavior, and operational history, ensuring transparency and traceability.

View a Task

- Select the **eye icon** in the Actions column.
- View detailed information about this specific task.
- Select **Back** to return to the table view.

Logs

The Logs tab provides a holistic view of all ongoing activities, offering detailed insights into the system's operations.

These are application-level actions being recorded. Access the system log to monitor activity across the entire platform, including voter events, Keycloak events, system events, and user events.

- Select **Columns** in order to hide/show different points of data per log entry.

Immutability

The Logs tab is implemented using Immudb, a tamper-evident database. Each entry is inserted into a SQL table of actions. The immutability allows to verify that no change has been performed, for anyone with credentials to access to the immutable log.

External system (Datafix) API entries

Every call an external voter-registry system makes to the Datafix API (`add-voter`, `update-voter`, `delete-voter`, `mark-voted`, `unmark-voted`, `replace-pin`), and every `SetVoted/SetNotVoted` request the platform sends back to it, is recorded as an `ExternalApiRequest` entry.

- **Description** states the operation and its outcome only, for example `Inbound request UpdateVoter Succeeded.` or `Inbound request ReplacePin Failed.`
- **Message** carries the details in the operation string of the statement body:

```
voter_id="<datafix voter id>"; <Operation> <Outcome>[: <reason>] (<key>=
<value>, ...)
```

Values that come from free text (the voter id, the area name, the channel) and values read back from Keycloak (`area_id`, `previous_area_id`, `previous_birthdate`, which are attributes an administrator can edit) are written double-quoted, with `"` and `\` escaped, so a value containing `,` or `=` is still read as a single value and not as another field.

Only the fixed vocabulary (`none`, `unchanged`, `true`, `false`) and the validated `birthdate` of the request are written bare.

Operation	Values recorded on success
AddVoter	<code>area</code> , <code>area_id</code> , <code>birthdate</code> (<code>none</code> when not sent), <code>enabled=true</code>
UpdateVoter	What the voter carried before the write (<code>previous_area</code> , <code>previous_area_id</code> , <code>previous_birthdate</code> , <code>previous_enabled</code> ; <code>none</code> when the voter had no such value) next to what the request applied (<code>area</code> , <code>area_id</code> , <code>birthdate</code> , <code>enabled</code>); a field that was not sent is written as <code>unchanged</code>
DeleteVoter	<code>enabled=false</code> and the <code>disable_comment</code> written to the voter (Datafix voters are disabled, never deleted)
MarkVoted	<code>channel</code> , <code>enabled=false</code> , <code>disable_comment</code>
UnmarkVoted	<code>previous_channel</code> , <code>channel=NONE</code> , and whether the account was re-enabled (<code>enabled</code> , <code>disable_comment</code>); <code>unchanged</code> means an administrator's disable was preserved
ReplacePin	<code>temporary</code> (the PIN itself is never recorded)

A failed operation records the internal reason and the error code returned to the external system, for example `voter_id="123456"; ReplacePin Failed: Cannot replace pin because the user is disabled (error_code=invalid-request)`. The reason is only recorded here; the API reply carries the error code alone.

Before writing to Keycloak, an inbound operation checks that the election event's user profile stores every attribute it is about to write (for example `dateOfBirth`, `voted-channel` or `disable-comment`). If the realm would silently drop one, the operation fails without changing the voter, the external system receives `internal-error`, and the entry names the attributes the realm does not store.

Only these inbound operations make the check. A voter edit from the Admin Portal, the Internet channel recorded when a vote is cast and the edits reconciliation applies to existing voters all write through Keycloak without it. On a realm that does not declare an attribute, and whose unmanaged-attribute policy does not let administrators write it,

their value is dropped silently. The voters reconciliation adds are written directly to the database and keep their attributes whatever the realm declares.

A request is attributed to the election event whose `datafix:id` annotation matches the Datafix id in the caller's token. If several election events of the tenant share that id, the request cannot be attributed to any of them and is rejected with `internal-error` before touching any voter, since only an administrator can fix the configuration. The failed entry is then recorded in the electoral log of **every** event configured with that id, naming all of them in the reason, with the voter's username alone (no `user_id` or `area_id`, because the voter cannot be resolved without an event).

- **User id** and **username** identify the voter (the Datafix voter id is the username) and the message's `area_id` is the voter's area, as in Keycloak events. A field the entry has no value for is left out of the message instead of being written as `null: election_id` always, because these operations apply to the whole election event, and `user_id/area_id` when the voter cannot be resolved, as in an `AddVoter` that failed before the voter was created.

Request bodies in the service logs

Besides the electoral log entry, the API service records the full body of every inbound Datafix request in its tracing output, so a rejected request can be debugged without reproducing it. This data is deliberately not treated as sensitive: the voter id is an anonymous registry identifier, and the same values are already stored in the electoral log entry described above and in the voter files exchanged with the external system. Only the caller's token is kept out of the service logs.

Scheduled Events

Allows the automation of an Election Event by scheduling key actions to occur automatically at specified times during the event lifecycle.

Adding a Scheduled Event

- Select **Add**.
- Select a **Scheduled Event Type**.
- Select an **Election**.
 - If no Election is selected, the Scheduled Event will apply to the Election Event and all related Elections.
- For **Start Voting Period** or **End Voting Period**, select one or more **Voting Channels**: Online, Kiosk, Early voting, or Telephone voting. Online and Kiosk are selected by default.
 - All four channels are always available in the form. When the schedule runs, it changes only channels enabled for each targeted election.
 - Existing schedules with no channel selection (or an empty selection in the payload) use Online and Kiosk for both start and end. Editing or exporting/importing a schedule preserves its channel selection.
- Select the **starting date and time** for this event.
- Select the **starting date and time** for when this scheduled event is triggered.

Note:

- Start Date and Time for **Start Voting Period** is the time the voting period starts.
- Start Date and Time for **Stop Voting Period** is the time the voting period ends.

Certificates

Voters can authenticate using client certificates issued by a Certificate Authority (CA). For this to work, the relevant CAs must be uploaded to the election event. Different election events can have different sets of CAs configured.

Enabling certificate authentication

Certificate authentication is disabled by default. To enable it:

1. Go to the election event's **Data** tab.
2. Open the **Advanced Configurations** section.
3. Enable the **Voter Digital Certificate Policy** setting.

Once enabled, a **Login with Certificate** button will appear on the voting portal login page, allowing voters to authenticate using their client certificate.

Managing certificates

The **Certificates** tab lets you manage the CA certificates for the election event.

Certificates must be in **PEM format**. Importing a PEM file that contains multiple certificates will import all of them at once.

Available actions:

- **Import:** Upload a PEM file containing one or more CA certificates.
 - **Export:** Download all certificates as a PEM file or select one or more and click on Export.
 - **Delete:** Remove one or the selected certificates.
 - **View:** Display the certificate's details in human readable format.
-

Keycloak configuration

Uploading CA certificates to the admin portal is only part of the setup. Keycloak must also be configured separately to classify and authenticate voters based on those CAs.

Certificate browser flow

Authentication is handled by the **certificate browser flow** in Keycloak. This flow contains two top-level subflows:

- **Credentials subflow** — classifies the certificate and resolves the voter identity.
- **Deny subflow** — handles error cases (see [Error cases](#) below).

Classifier subflows

Inside **Credentials subflow**, the `x509-cert-classifier` authenticator runs first and sets a `cert-type` auth note based on the certificate's CN. Each CA is then handled by a **conditional subflow** that checks `cert-type` and, if it matches, runs the `x509-user-resolution` authenticator to look up the voter.

Two example subflows are created by default at election event creation and can be edited, removed, or used as a reference:

- **Dev Sequent** — matches certificates with `cert-type = "Sequent Dev CA"`.
- **Other CA** — matches certificates with `cert-type = "Other CA"`.

Each classifier subflow contains:

1. A **Condition - auth note** step that checks `cert-type` equals the expected CA CN (e.g. `"Sequent Dev CA"`).
2. An **X509/Validate Username Form** step (`x509-user-resolution`) configured to match voters by SubjectDN using the regex `(?:CN=)([^\, ]*)`.

Adding a new CA subflow

For each new CA imported in the Certificates tab, add a corresponding conditional subflow in the **Credentials subflow**:

1. In the Keycloak admin console, navigate to the election event's realm → **Authentication** → **certificate browser flow** → **Credentials subflow**.
2. Add a new **conditional subflow** (e.g. named after the CA).
3. Inside the new subflow, add:
 - A **Condition - auth note** step with `cert-type` = the CN of the new CA (set `value-is-regex` to `false`).
 - An **X509/Validate Username Form** step configured to match voters by SubjectDN.
4. Update the **No classifier match** subflow's condition: edit the `NoClassifierMatch condition` authenticator config and add the new CA's CN to the negated regex. For example, if the existing regex is `(Sequent Dev CA)|(Other CA)`, extend it to `(Sequent Dev CA)|(Other CA)|(New CA CN)`.

TIP

The `NoClassifierMatch condition` uses a negated regex. Any certificate whose `cert-type` does not match the regex will trigger the deny-access authenticator, resulting in an `accessDenied` error. Keep this regex in sync with all configured classifier subflows.

Error cases

The **Deny subflow** handles three error scenarios. Voters will see the corresponding message on the login page:

Error key	Message	Cause
<code>certNotProvided</code>	Authentication failed: No certificate was provided	The voter did not present a client certificate.
<code>userNotFound</code>	Authentication failed: Valid certificate detected, but no matching voter or user was found	The certificate was valid and classified, but no voter record matched.

Error key	Message	Cause
<code>accessDenied</code>	Authentication failed: Invalid certificate, access denied	The certificate was not classified by any subflow (issuer CA not configured or not trusted).

These messages are defined in the Keycloak theme translation files at:

```
packages/keycloak-extensions/sequent-  
theme/src/main/resources/theme/sequent.voting-  
portal/login/messages/messages_<lang>.properties
```

They can be overridden per-realm in the Keycloak admin console under **Realm Settings** → **Localization**.

Tally Ceremony

Introduction

The Tally section covers the essential procedures required to consolidate results after voting has concluded. Election Board members are guided through starting the tally ceremony, verifying key fragments, running the tally, and reviewing results. By following these instructions, the post-election phase is handled with integrity and precision, ensuring that results are verified and published correctly and that the process remains transparent and trustworthy.

Closing Key Ceremony

During the Closing Key Ceremony for an Election Event, trustees use their cryptographic key fragments to decrypt the final tally:

- **Retrieve stored key fragments:** Each trustee securely retrieves their previously backed-up fragment of the private key.
- **Verify integrity and functionality:** Trustees perform cryptographic tests or audits to confirm each fragment is unaltered and functional.
- **Decrypt encrypted votes:** Trustees combine fragments (meeting the threshold) to decrypt the accumulated encrypted voting data, ensuring accurate and transparent results.
- **Secure key storage:** After decryption, key fragments are securely stored again to maintain confidentiality and authenticity of the election outcome.

This ceremony ensures that only a quorum of trustees can reconstruct the private key for decryption, preserving security throughout the election lifecycle.

Prerequisites for Tally

Before initiating the tally process:

1. **Opening Key Ceremony completed:** All key fragments have been generated and verified.

2. **Voting phase concluded (optional):** Votes may or may not have been cast; starting a tally with no votes is technically possible but yields empty results.

Note: According to the Election Event Keys/Tally Ceremonies Policy, if the Key Ceremony is set to 'Automatic,' the Tally Ceremony will also be automated, and no trustee action is required.

Starting the Tally Process

1. Navigate to the relevant **Election Event** in the Administration Portal.
2. Go to the **Tally** tab.
3. Click **Start Tally Ceremony**.
4. Select one or more Elections to include in this tally.
5. Confirm to proceed; the system will notify trustees to verify their key fragments.
6. Trustees verify their fragments as prompted; once the threshold of fragments is verified, the tally begins automatically.
7. Wait for the system to finalize the tally. A success notification appears when complete.

Trustee Key Verification for Tally

1. Log in to the Administration Portal and open the relevant Election Event.
2. Go to the **Tally** tab.
3. Click the Key Action invitation or the key icon next to the Tally ceremony.
4. Select the Election to tally.
5. Drag and drop or upload your encrypted key fragment for verification. After a valid upload, the upload area is locked. Re-uploading the same key from a stale browser tab is accepted as already restored, while a different key is rejected.
6. Repeat for each trustee until the threshold is reached.
7. Once all required fragments are verified, click **Start Tally** to begin decryption and tallying.

Reviewing Results

After tally completion, results become available under the **Tally** tab:

- **View Results:** Click the View Results button for each Election.
- **Columns:** Enable or disable columns as needed for clarity.
- **Elections Tally Progress:** Monitor status (e.g., “In Progress,” “Success”).
- **Logs:** Access detailed logs for auditing and troubleshooting.
- **General Information:** Displays tally date, total voter participation, etc.
- **Results and Participation:** Detailed breakdown by Areas, Contests, and Elections.

General Information

- **Tally Date and Time:** When the tally was finalized.
- **Voter Participation:** Summary of how many eligible voters participated.

Results & Participation Breakdown

- **By Election:** Results for each Election within the Event.
- **By Contest:** Results for each Contest across Elections.
- **By Area:** Results segmented by geographic or organizational divisions.
- **Participation Summary:**
 - **Eligible Voters:** Total number eligible to vote.
 - **Total Voters:** Number who cast a vote.
 - **Total Valid Votes:** Count of valid ballots, including blank ballots.
 - **Total Invalid Votes:** Count of invalid ballots, subdivided into:
 - **Explicitly Invalid Votes:** Null votes, spoiled ballots, protest actions.
 - **Implicitly Invalid Votes:** Invalid due to configuration (e.g., multiple selections where only one is allowed).
 - **Blank Votes:** Explicit and implicit blank ballots. Blank ballots are included in total valid votes.

Candidate Results

For each Contest, display:

- **Option / Candidate Name**
- **Number of Votes:** Count of valid votes for that candidate/option.
- **Percentage of Votes:** Proportion relative to valid non-blank votes when the result is a candidate row.
- **Ranking / Position:** Placement among candidates by vote count.

Note: According to the Election Event Weighted Voting Policy, if the policy is set to 'Weighted Voting for Areas', the tally will be calculated using the weights assigned to each area during the last publication prior to the tally.

Summary

By following this guide, administrators ensure the tally ceremony is conducted securely, transparently, and accurately. Key fragments are verified by trustees, encrypted ballots are decrypted correctly, and results are reviewed with full auditing capabilities. This maintains the integrity and trustworthiness vital for credible elections.

Transmission

The Transmission module under the Tally tab enables sending election results to external servers that have been predefined in the system environment. This feature ensures that encrypted and signed tally data can be securely delivered to authorized endpoints for archival, auditing, or further processing.

Prerequisites

- A completed Tally Ceremony: The tally must have been finalized and results are available.
- Preconfigured destination servers: External servers (with URLs, credentials, or endpoints) must be set up in the system environment beforehand.
- Trustee PnPKI files: Each trustee must possess their PnPKI signing files to sign the transmission package.

Usage Steps

1. Open the Transmission Module

- In the Administration Portal, navigate to the relevant **Election Event**.
- Go to the **Tally** tab.
- Locate and open the **Transmission** module or section.

2. Select the Election for Transmission

- Click the antenna (transmission) icon associated with the Election or Election Event you wish to transmit.
 - Typically, this icon appears next to the “Actions” for a completed tally.
- Alternatively, under the Election “Actions” menu, choose **Send Transmission Package....**

3. Prepare the Transmission Package

- The system will package the finalized tally results into a transmission bundle.
- Review any prompts or metadata fields (e.g., timestamp, package identifier) as required by your organization’s workflow.

4. Trustee Signing

- Trustees must sign the transmission package using their PnPKI signing files:
 - a. The interface will prompt each trustee to upload or apply their PnPKI file.
 - b. Trustees follow on-screen instructions to digitally sign the package.
 - c. Repeat for each required trustee until the threshold of signatures is collected.
- Signing ensures authenticity and non-repudiation of the transmitted data.

5. Send the Transmission Package

- Once all required signatures are applied, click the **Send** button.
- The system will transmit the signed package to the designated external servers configured in the environment.

6. Monitor Transmission Status

- **Destination Servers** section:
 - Review the status of each server (e.g., Pending, In Progress, Success, Failure).
 - Details may include timestamps, response codes, or error messages from the target endpoint.
- **Logs:**
 - Access the system logs for detailed entries about the transmission process.
 - Logs record actions such as package creation, trustee signatures, transmission attempts, and server responses.

Error Handling & Troubleshooting

- **Signature Issues:**
 - If a trustee's signature fails (invalid PnPKI file or mismatch), the system will prompt an error.
 - Verify the correct PnPKI file and re-attempt signing.
- **Transmission Failures:**
 - If sending to a server fails (network error, authentication issue, server down), the status will indicate an error.
 - Check destination server configuration (URL, credentials, connectivity).
 - Consult logs for detailed error messages and retry when resolved.
- **Partial Success:**
 - If multiple destination servers are configured, some may succeed while others fail.

- Address failures individually and retransmit as needed.

Best Practices

- **Predefine and Test Servers:**
 - Configure destination endpoints in a test environment first to validate connectivity and authentication.
 - **Secure Key Management:**
 - Ensure trustees' PnPKI files are stored and accessed securely during signing.
 - **Audit Trails:**
 - Keep logs and transmission receipts for audit purposes.
 - Retain transmitted packages or checksums as evidence of delivery.
 - **Retry Policies:**
 - Establish procedures for automatic or manual retries upon transient failures.
 - **Notification:**
 - Configure notifications or alerts for successful or failed transmissions to relevant stakeholders.
-

By following these steps, administrators can securely package, sign, and transmit election results to external systems, while maintaining an auditable trail and ensuring data integrity throughout the process.

Reports

The Reports tab allows for the configuration, generation, and scheduling of reports using predefined Templates (see the Templates section for details). Some reports (e.g., Initialization Report, Manual Verification, Ballot Receipt) are triggered automatically by system actions. Other reports (e.g., Statistical Report, Voter List Summary) can be scheduled or generated manually as needed. This tab provides full control over report configuration and scheduling, ensuring flexibility and automation where required.

Adding a Report Configuration

1. Select **Add**.
2. Select a **Report Type**.
 - This determines the kind of data to be generated.
 - Example: **Statistical Report** generates statistical data about Tally results.
3. Select an **Election** from which data will be gathered.
 - If left blank, the configuration applies to ALL Elections.
4. Select the appropriate **Template** (optional).
 - If left blank, a default template designed by Sequent will be used.
 - Templates control how report data is displayed (HTML code). Templates are associated with Report Types in the Templates tab.
5. Add the appropriate **Permission Label** for the report.
 - This determines which users can see, generate, and preview the report.
6. Select **Save**.

Repeatable Reports

Enabling the **Repeatable** option allows generating and sending reports repeatedly according to a schedule (Cron expression).

1. Enable **Repeatable**.
2. Define your **Cron Expression**.
 - See “Setting up a Cron Expression” below for guidance.
3. Set the **Email Recipients** for this report.
4. Select **Save**.

Setting up a Cron Expression

A cron expression is a sequence of five fields indicating when to run a task:

- **Minute** (0–59)
- **Hour** (0–23)
- **Day of Month** (1–31)
- **Month** (1–12)
- **Day of Week** (0–6, where 0 = Sunday, 6 = Saturday)

Examples:

- **Hourly Report:** `0 * * * *`
Runs every hour at minute 0 (e.g., 1:00, 2:00).
- **Daily Report:** `15 9 * * *`
Runs every day at 9:15 AM.
- **Monthly Report:** `30 10 1 * *`
Runs on the first day of every month at 10:30 AM.
- **Day-of-Week Report:** `0 14 * * 5`
Runs every Friday at 2:00 PM.

Encrypted Reports

To protect sensitive information, reports can be encrypted with a user-chosen password. Encryption status and password are preserved during export/import, so re-encryption is not needed.

1. Click **Create Report** or **Add**.
2. Choose **Report Type**.
3. Choose **Election** (optional).
4. Choose **Template** (optional).
5. Set **Encryption Policy** to “Configured Password”.
6. Enter a password in the **Password** field.
7. Repeat the password in the **Repeat Password** field.
 - This password will be required to decrypt all generated reports from this configuration.
8. Click **Save Password** (or **Save** depending on UI).

Note: Encrypted reports retain their encryption when exported and re-imported. The same password must be used to decrypt them regardless of context.

With these instructions, users can configure one-off or scheduled reports, and choose to encrypt sensitive report outputs. Adjust terminology or examples according to your UI labels and workflow.

Templates

Managing Templates is essential for consistent report generation. Each Report Type is associated with a Template to form a “recipe” used when generating reports.

Adding a New Template

1. **Select Add** to create a Template.
2. **Fill in the fields:**
 - **Template Alias (Optional):** Display name shown in the Admin Portal.
 - **Template Name:** Internal name for the template in the Admin Portal.
 - **Template Type:** Category or area that will use this template. (E.g., Ballot Receipt, Statistical Report, etc.)
 - **Email / SMS / Document:** Choose whether this template includes an email/SMS message or attaches a document. Select the appropriate radio button.
3. **Save** the Template.

Once configured, the Template becomes available for its associated Report Types and other system areas.

Key Points

- **Consistency:** Use predefined or default formats where possible to ensure consistency across reports.
- **Reuse:** A single Template can be applied to multiple Report Types if suitable.
- **Preview:** After saving, preview the Template in context (e.g., generate a sample report) to confirm formatting.
- **Updates:** Editing a Template will affect all future report generations that reference it; consider versioning or alias changes if you need to preserve older formats.
- **Examples:**
 - Configuring the Ballot Receipt template: select “Ballot Receipt” as Template Type, define alias/name, choose Document radio, set layout/content, then Save. This will be used whenever a Ballot Receipt is generated.

Tips

- Maintain clear, descriptive Template Names and Aliases so administrators can identify their purpose quickly.
- Document any special placeholders or variables used in templates (e.g., voter name, election date) in a separate reference or within the template description.
- Test email/SMS templates by sending to a test address or number before enabling in production.
- For document templates, ensure any required assets (logos, images) are accessible and correctly referenced.
- If your system supports previewing or templating languages (e.g., handlebars, Liquid), include sample data to verify rendering.

Voter variables in notification templates

Email and SMS Handlebars templates can use these standard voter variables:

- `user.first_name`
- `user.last_name`
- `user.username`
- `user.email`

Custom Keycloak user attributes are also available. The first value is exposed as `user.<attribute>`. The complete value list remains available as `user.attributes.<attribute>`. Standard variables take precedence if a custom attribute uses the same name. The `attributes` name is reserved for the complete attribute map. Empty custom value lists are present only under `user.attributes`.

Dot notation works for simple names such as `dateOfBirth`. Use Handlebars `lookup` for names containing dots or dashes. For example:

```
{{lookup user "sequent.read-only.mobile-number"}}
{{#each (lookup user.attributes "sequent.read-only.mobile-number")}}{{this}}
{{/each}}
```

For example, a `reference` attribute with values `ABC-123` and `legacy-456` can be rendered as follows:

```
Primary reference: {{user.reference}}
All references: {{#each user.attributes.reference}}{{this}} {{/each}}
```

Prefilled voting links

Voting Portal `/login` and `/enroll` links accept up to five prefilled fields named `login_hint__<field>`. Field names may contain letters, numbers, `.`, `_`, and `-`; names are limited to 128 characters and values to 255 characters.

Use the `url_encode` helper around every dynamic query value. Keep the parameter names and URL structure static:

```
https://vote.example/tenant/TENANT_ID/event/EVENT_ID/login?
login_hint__username={{url_encode user.username}}&login_hint__reference=
{{url_encode user.reference}}
```

```
https://vote.example/tenant/TENANT_ID/event/EVENT_ID/enroll?
login_hint__username={{url_encode user.username}}&login_hint__dateOfBirth=
{{url_encode user.dateOfBirth}}
```

The Voting Portal removes accepted hint parameters from its visible URL before redirecting to Keycloak. Invalid, duplicate, or over-limit hint sets are rejected as a whole.

Per-field prefill policy

Each registration field decides how it accepts a prefilled value through the `loginHintPrefillPolicy` annotation of its Keycloak user profile attribute:

Policy	Behaviour
<code>EDITABLE</code>	Prefill the field and let the voter change the value. Applied when the annotation is absent.

Policy	Behaviour
READ_ONLY	Prefill the field, render it read-only, and reject the registration if the submitted value was changed.
IGNORE	Never prefill the field from a voting link.

Set the annotation in **Realm settings → User profile → Attributes → (attribute) → Annotations**, for example `loginHintPrefillPolicy = READ_ONLY`. An unrecognised value is treated as `IGNORE`, so a typo never prefills a field.

Some attributes are never prefilled, whatever the policy says: credential fields, unmanaged attributes, attributes the voter cannot write, and attributes the voter would not be able to see or edit on the form. The last group covers an attribute annotated `hidden`, one rendered as a hidden or password input, and one whose annotations set `html-attribute:disabled`, `hidden`, `inert`, `readonly`, `aria-hidden="true"`, or an inline `html-attribute:style` that hides it (`display:none`, `visibility:hidden`, `visibility:collapse` or `opacity:0`).

Cosmetic annotations do not affect prefill — `html-attribute:class` and inline styles that only change appearance leave the field prefillable. When an attribute is skipped, Keycloak logs the attribute name and the reason at `DEBUG` level under `sequent.keycloak.voter_enrollment.LoginHintPrefill`; hint values are never logged.

The policy applies to registration forms once the **Sequent: Login hint registration prefill** action is part of the registration flow.

Locking the username on the login page

The login page is not rendered from the user profile, so it cannot read attribute annotations. Set the realm attribute `loginHintUsernamePolicy` to `READ_ONLY` (default `EDITABLE`) to render a prefilled username read-only. A username restored by *remember me* stays editable, so voters can still sign in as somebody else.

This is a presentation-only lock: it stops a voter from editing the field by accident, but Keycloak still authenticates whichever username is submitted. That is the same guarantee as an unprefilled login page — the password decides which account is entered. Registration is different: there the value is stored, so `READ_ONLY` is enforced on the server as well.

 WARNING

Prefilled values are convenience data, not verified identity claims. They never bypass authentication, registration validation, or required actions. `READ_ONLY` stops a voter from changing a prefilled field; it does not make the value trustworthy, because whoever built the link chose it. Do not include passwords, one-time passwords (OTPs), tokens, secrets, government identifiers, or other inappropriate sensitive values. Percent encoding protects URL structure; it does not provide confidentiality or authenticity.

Note: For further guidance on template fields or syntax, refer to the Reports section of the guide where Template usage in report configuration is detailed.

Dashboard

The Dashboard provides administrators with a snapshot of an individual Election's progress and key statistics. Note that this displays a subset of data specific to the selected Election (not the broader Election Event).

Metrics

- **Eligible Voters**

Displays the total number of voters eligible to participate in this Election.

- **Actual Voters (Enrolled Voters)**

Shows the count of enabled/enrolled voters who can cast a ballot in this Election.

- **Areas**

Indicates the number of geographic or organizational areas assigned to this Election.

- **Emails Sent**

Total number of email notifications sent to voters for this Election.

- **SMS Sent**

Total number of SMS notifications sent to voters for this Election.

Charts

- **Votes by Day**

A time-series chart illustrating daily voting activity (number of ballots cast per day) for this Election.

- **Voters by Channel**

A pie or donut chart showing the distribution of votes by channel (e.g., Online, Paper, Telephone, Postal) for this Election.

Notes

- This Dashboard reflects only the data for the specific Election you are viewing.

- For broader context or aggregated metrics across multiple Elections, refer to the Election Event Dashboard.
- Use these insights to monitor participation trends, identify potential issues early, and guide communications or interventions as needed.

Data

1. General

Manage data related to the Election. This tab allows configuration of core settings for an individual Election, similar in structure to the Election Event Data tab but scoped to this specific Election.

- **General:** Set up basic details and multilingual names.
- **Dates:** Configure start and end dates/times.
- **Language:** Define supported and default languages.
- **Voting Channels Allowed:** Specify applicable voting methods.
- **Ballot Design:** Manage presentation and ordering of contests and audit options.
- **Receipts:** Configure how voters receive confirmation or receipts.
- **Image:** Upload an image associated with this Election.
- **Advanced Configuration:** Import advanced settings and define vote limits, confirmations, and permissions.

More information on each section is available below.

General

Setup basic details and configure multilingual names for this Election. This mirrors the Election Event “General” section but applies only to the current Election.

- **Languages Tabs:** Configure how this Election appears in different languages in the Voting Portal.
 - **Name:** Enter the official name of the Election.
 - **Alias (optional):** Specify an internal alias for display in side menus.
 - **Description (optional):** Provide a description to clarify the purpose or scope of this Election.
-

Dates

Set the start and end dates for your Election.

- **Start Date & Time:** Commence the voting period for this Election.
 - **End Date & Time:** Conclude the voting period for this Election.
 - Use the calendar/date-time pickers to select the exact timestamps.
-

Language

Manage language options for this Election. Similar to Election Event, but scoped per Election.

- **Define Languages:** Select which languages are enabled for this Election via radio buttons.
 - **Set Default Language:** Mark one language as default by selecting “Default” next to it.
 - Note: Available languages here are constrained by the Election Event’s language settings.
-

Ballot Design

Manage presentation of ballot elements for this Election.

- **Audit Button Display Options:** Toggle whether the “Audit Ballot” button appears in the Voting Portal.
 - **Presentation Contests Order:**
 - **Random:** Contests appear in a random order for each voter session.
 - **Alphabetical:** Contests are listed alphabetically by name.
 - **Custom:** Manually reorder contests. When “Custom” is selected, a drag-and-drop or similar interface appears to arrange contests in the desired sequence.
 - **Additional Design Settings** (if applicable):
 - Logo, CSS overrides, or other display options (depending on system capabilities).
 - Receipt format settings may be covered under “Receipts” if separate.
-

Voting Channels Allowed

Specify which voting channels (methods) are applicable for this Election.

- Use radio buttons or checkboxes to enable/disable channels (e.g., Online, EARLY_VOTING, Kiosk, Paper).
 - Only the selected channels will accept votes in this Election.
 - **EARLY_VOTING:** Enables an early voting period for voters assigned to Areas where the area policy "Early Voting" is enabled. This appears only if the channel is allowed at the Election Event level.
 - When the Election Event starts the **EARLY_VOTING** channel (Publish tab), only voters in eligible Areas can vote early.
 - The **Online** channel governs Early Voting lifecycle: starting or closing Online will automatically close Early Voting; Early Voting cannot be started once Online has been started.
 - **Kiosk:** In-person voting at kiosks; its status is independent from Online/Early Voting.
 - If a channel that has already been started is later manually disallowed in this section, no immediate action is taken; action buttons remain disabled until the channel is allowed again.
-

Receipts

Configure how voters receive confirmation of their vote for this Election.

- **Receipt Method:** Choose whether voters receive an on-screen confirmation, email receipt, SMS receipt, or a combination (depending on system capabilities).
 - **Audit Support:** Enable or disable voter's ability to verify encryption (if separate from Ballot Design audit button).
 - **Customization:** If system allows, configure templates or text shown on receipts.
-

Image

Upload an image to be displayed within the system for this Election (e.g., a logo or banner).

- **Upload:** Drag and drop or select a (or supported format) file.

- **Preview:** After upload, preview how the image appears in the portal or admin interface.
 - **Optional:** If no image is needed, this section can be left blank.
-

Advanced Configuration

Import advanced settings for this Election and define vote limits, confirmations, and permission labels.

- **Cast Vote Confirmation Modal:**

- Toggle display of a confirmation dialog before finalizing a vote.
- If enabled, configure the confirmation text or warning message.

- **Number of Allowed Votes:**

- Specify how many times a voter may revote (e.g., allow re-submissions up to N times).
- If unlimited revotes are not permitted, set the maximum count here.

- **Permission Label:**

- Assign a permission label to this Election, controlling which Admin Portal users can access/manage this Election.
- Example workflow:
 - a. Create or select a permission label (e.g., “Election A Manager”).
 - b. Associate one or more admin users with this label.
 - c. Only users with this permission label see or modify this Election in the Admin Portal.

- **Upload Advanced Configuration** (Optional):

- Drag and drop a configuration file (e.g., JSON or system-specific format) to apply pre-defined advanced settings.
- This may include settings like contest encryption policies, custom validations, or integrations.

- **Start Screen Title Policy**

- Set the title of the Start Screen (Voting Portal) to either the election title or the Election Event Title. The default value is the Election title.

- **Consolidated Report Policy**

- If **Generate** option is selected, consolidated election results for all areas will be generated in the Tally.

- **Decline to Vote Policy**

- Set to **Enabled** to offer voters a **Decline to Vote** option on the Start Screen for the entire Election across all contests.
- Set to **Disabled** (default) to hide the option.
- This setting appears only when the Election Event uses the **Multiple Contests** encryption policy.
- See [Decline to Vote Policy](#) for full configuration, voter flow, and results behavior.

- **Blank Ballots Policy**

- Set to **Enabled** to report ballots left empty in every contest as a distinct **Total Blank Ballots** figure, separate from Decline to Vote and from per-contest blank votes.
- Set to **Disabled** (default) to leave this figure unavailable.
- This setting appears only when the Election Event uses the **Multiple Contests** encryption policy. There is no voter-facing button — blank is inferred when the voter leaves every contest empty during normal voting.
- See [Blank Ballots](#) for full configuration, voter flow, and results behavior.

- **Voting Screen Back Button Policy**

- Controls where the **Back** button on the first voting screen takes the voter.
- Set to **Go to the election selection screen** (default) to keep the previous behavior.
- Set to **Go to the election start screen** to return the voter to the Election's Start Screen, where the ballot is reset to its initial state. Recommended when [Decline to Vote](#) is enabled, so the decline option remains reachable from the voting screen.

Notes & Best Practices

- Many settings here depend on or inherit from Election Event configurations (e.g., available languages, global voting channels). Always verify consistency between the Election Event and individual Election settings.
- Before enabling advanced options (e.g., revote limits, permission labels), ensure you understand their impact on voter experience and security.
- Use descriptive names and aliases to help administrators quickly identify Elections in multi-election events.
- Test changes in a staging environment when possible, especially for advanced configurations or ballot design modifications.

Voter Assignment in an Election

This section explains how to view and assign voters to a specific Election. Note that voters are created and managed at the Election Event level; assigning them to an Election happens indirectly via Areas and Contests.

Overview

- **Voters are not created per Election:** Voter records exist at the Election Event level.
- **Assignment is indirect:** To include a voter in an Election, you assign them to an Area that is linked to one or more Contests within that Election.
- **Preconditions:** You must have at least one Contest defined for the Election before assigning voters.

Key Concepts

1. Election Event Voters

- This tab shows all voters registered for the overall Election Event.
- At this stage, a voter may have no Area or Contest assignment.

2. Area

- Represents a geographic or organizational division within the Election Event.
- Contests are linked to Areas; voters assigned to an Area gain access to the Contests associated with that Area (i.e., the Elections within that Event).

3. Contest

- A contest belongs to an Election within the Election Event.
- Contests must exist before voters can be effectively assigned to participate in them.

4. Assignment Flow

- i. Create or verify the Contest in the Election Event.
- ii. Create an Area (or verify existing Areas) in the Election Event.
- iii. Link the Contest to the Area.
- iv. Assign the voter (from Election Event Voters) to the Area.
- v. As a result, the voter is now part of the Election(s) associated with that Area and Contest.

Detailed Steps: Assign a Voter to an Election

Follow these steps once you have defined Contests in your Election Event:

1. Ensure the Contest Exists

- In the Election Event, confirm that the desired Contest is created under the relevant Election.

2. Check Current Voter Assignment

- Navigate to the **Voters** tab in the Election Event.
- Observe that the voter record exists but has no Area assignment (and thus is not yet included in any specific Election).

3. Create or Select an Area

- Go to the **Areas** tab in the Election Event.
- If no suitable Area exists, create a new Area (e.g., “District A”).
- If Areas exist, choose the relevant one.

4. Link Contest to Area

- In the Areas list, select the **Edit** action for the chosen Area.
- In the Area edit screen, assign the appropriate Contest(s) to this Area.
 - This links those Contest(s) (and their parent Election) to the Area.
- Save changes.

5. Assign Voter to the Area

- Return to the **Voters** tab in the Election Event.

- Locate the voter record you wish to include.
- Select **Edit** (or “Assign”) for that voter.
- In the voter edit screen, set the Area field to the one just configured.
- Save changes.

6. Result

- The voter is now assigned to the Area linked to the Contest/Election.
- In the context of this specific Election: the voter will appear under that Election’s Voter list (via the Area assignment).

Example Walkthrough

1. Election Event Voters

- Open the Election Event and go to **Voters**.
- Notice the voter “Alice Example” exists but shows no Area.

2. Create Area

- Go to **Areas**. Click **Add Area**.
- Name: “Downtown District”. Save.

3. Link Contest to Area

- In the Areas list, click **Edit** next to “Downtown District”.
- Under “Assigned Contests”, select “Mayor Race” (example Contest).
- Save.

4. Assign Voter

- Return to **Voters**. Click **Edit** next to “Alice Example”.
- In the “Area” dropdown, choose “Downtown District”.
- Save.

5. Verify

- Navigate to the specific Election (e.g., “Mayor Election”) and view its Voters.
- “Alice Example” now appears as an eligible voter for that Election.

Notes & Tips

- If an Area is linked to multiple Contests/Elections, assigning a voter to that Area enrolls them in all linked Contests.
 - To exclude a voter from a particular Contest, do not link that Contest to the Area, or assign the voter to a different Area.
 - To manage multiple Contests per Area, use the Area's edit screen to add or remove Contests as needed.
 - Changes take effect immediately; after assignment, the voter can access the Voting Portal for that Election.
 - If you later remove the Contest from the Area or reassign the voter to a different Area, their Election access adjusts accordingly.
-

Publish

To commit any changes to an Election, they must be published using this tab. Additionally, you can start, pause, and stop individual Elections here.

All publications and their respective changes can be viewed in the table using the "eye" icon.

Table Options

- **Columns:** Enable or disable specific columns for display.
- **Add Filter:** Create text filters for the table by column.

Actions

- **Start Election:** Begin the Election.
 - **Pause:** Temporarily halt the Election.
 - **Stop Election:** End the Election.
 - **Publish Changes:** Apply any changes to this Election.
-

Publish Changes / Preview

Changing settings in the Admin Portal will not apply them until they have been published. Publishing changes is done by selecting **Publish Changes** in the Publish tab.

Steps to Publish Changes

1. Select **Publish Changes**.
2. Enter your password (sensitive operations are password protected).
3. Review the changes.
4. (Optional) Select **Preview** to open the Voting Portal changes in a separate tab (per Area).
5. Select **Publish Changes** again to apply the changes.

Additional Actions

- **Regenerate:** Regenerate the diff files.
- **Export:** Export the diff files.

Contest Data Tab

The Contest Data tab allows administrators to manage core settings and configurations for a specific contest within an Election. Many elements mirror those in the Election Event Data tab, adapted for contest-specific context. Below are the primary sections and details.

Sections Overview

- **General:** Basic details and multilingual naming.
 - **Ballot Voting System:** Configuration for different voting systems (work in progress).
 - **Ballot Design:** Display and behavior settings for ballot presentation.
 - **Page Name:** Controls grouping of contests on voting pages.
 - **Image:** Upload an image/icon for this contest.
 - **Advanced Configuration:** Import custom configuration files for specialized behavior.
-

General

Configure fundamental details and multilingual names for this contest.

- **Contest Name**
 - Enter the official name.
 - Support for multiple languages if Election is multilingual.
 - **Alias (optional)**
 - System-only display alias in side menus or internal references.
 - **Description (optional)**
 - Brief description or instructions shown to administrators; may appear in tooltips or help texts if implemented.
 - **Languages Tabs**
 - If the Election supports multiple languages, configure how this contest's name/description appears per language.
-

Ballot Voting System

Voting System configuration will affect the tally and the Voting portal interface (e.g. whether the system is preferential or not).

Counting Algorithm

- **Plurality at Large**
 - Voters select up to a specified number of candidates. Candidates with the most votes win.
 - Best suited for non-preferential elections where voters choose their top choices without ranking.
 - **Instant Runoff or IRV**
 - Preferential voting system where voters rank candidates in order of preference.
 - Candidates with the fewest first-choice votes are eliminated and their votes redistributed until winners are determined.
 - When selecting IRV, set Max votes parameter (in Ballot Design) to at least the number of candidates available to rank.
-

Ballot Design

Manage how the ballot for this contest is displayed and how voters interact with it.

Display Settings & Alerts

- **Is acclaimed**
 - Marks a contest whose candidates have been elected without a vote.
 - The voting and review screens show the contest, its acclamation notice, and all available candidates. Its options are disabled and cannot be selected. A contest-specific translated notice can be supplied as `presentation.i18n.<language>.acclamation_description`; otherwise the standard translated notice is used.
 - Minimum and maximum selection rules do not apply. Explicit blank and invalid options and write-ins are ignored for this contest.
 - The contest is not encoded in the ballot. In a mixed election, the voter still casts the other contests normally; if every contest is acclaimed, no ballot or Ballot ID is

created.

- Tally results contain zero votes, report every eligible configured candidate as a winner, and replace participation figures with an acclamation notice. Tally sheets cannot be added to an acclaimed contest.
- Set this value before publishing ballots and do not change it afterwards, because it determines the ballot encoding layout.
- See [Acclaimed Contests](#) for the complete voter, verifier, tally, and results behavior.

- **Min Votes**

- The minimum number of selections a voter must make before the ballot is considered valid.
- Setting this to 0 allows voters to submit without selecting any candidate.

- **Max Votes**

- The meaning depends on the counting algorithm:
 - **Non-preferential (Plurality at Large):** the maximum number of candidates a voter may select.
 - **Preferential (Instant Runoff / IRV):** the highest ranking position available to voters. For example, a value of **5** means voters can assign ranks 1 through 5. Must be at least the number of candidates available to rank.
- Works in conjunction with the Over Vote Policy to determine what happens when a voter exceeds this limit.

- **Under-Vote Alert**

- Show a warning if a voter selects fewer than the maximum required options.

- **Over-Vote Alert**

- Show a warning if a voter selects more than the allowed number of options.

- **Winning Candidates Number**

- Specify how many candidates can win this contest.

- **Candidate Order**

- Determine display order:
 - Random
 - Alphabetical
 - Custom (when selected, enable manual reordering).

Edit Lists & Selection Types

- **Enable Checkable Lists**

- Allow grouping candidates into lists or combined list-candidate selections.
- When enabled, configure list-based voting behavior.

- **Max Selections Per Type**

- Define maximum selections for each candidate/list type.
- Example: Maximum of 2 individual candidates and 1 list selection.

Policies

Define voter behavior rules and system responses in various voting scenarios. For each policy type below, choose one of the options:

- **Allowed:** Voter may submit without warning.
- **Warn:** Voter may submit but receives warning(s).
- **Warn in Review** (if available): Warning appears only during review phase.
- **Warn and Alert:** Warning appears and a confirmation dialog forces acknowledgement.
- **Not Allowed:** Voter cannot submit; may include warning/alert before blocking.

Under Vote Policy

When voter selects fewer options than the minimum required:

- **Allowed:** Submit without warning.
- **Warn:** Warning during ballot and review phases.
- **Warn in Review:** Warning only in review phase.
- **Warn and Alert:** Warning during ballot; confirmation required to proceed.

Over Vote Policy

When voter selects more options than allowed:

- **Allowed:** Submit without warning.
- **Warn:** Warning during ballot and review phases.
- **Warn and Alert:** Warning during ballot; confirmation required to proceed.
- **Not Allowed with Warn and Alert:** Voter cannot submit; warning appears.
- **Not Allowed with Warning and Selection Lock:** Warning appears and prevents extra selections.

Invalid Vote Policy

When voter selection is invalid (e.g., null vote or too many selections):

- **Allowed:** Submit without warning.

- **Warn:** Warning during ballot and review phases.
- **Warn Implicit and Explicit:** Warning for explicitly invalid (null/spoiled) and implicitly invalid (e.g., too many).
- **Not Allowed:** Cannot submit if invalid options selected.
- **Allowed with Exclusive Explicit:** Same as Allowed, except selecting the explicit-invalid option is mutually exclusive with any other selection — picking it clears other selected candidates, and picking a candidate clears it, mirroring how blank vote already behaves. Combining an explicit-invalid selection with other candidates (e.g. via an older client) is still tallied like Allowed, since that combination remains an intentionally supported ballot shape for clients that rely on it.

Preferential systems (e.g. Instant Runoff)

What is considered Implicit invalid error?

- **Duplicated Position:** Whether casting the vote is allowed or not depends on the policy. Either case ranking 2 candidates with the same position will invalidate the vote.

What will cause an alert but it's still valid?

- **Preference order with gaps:** Missing one of the positions in between, e.g. 1,2,4,5.

Blank Vote Policy

When voter selects blank vote option:

- **Allowed:** Submit without warning.
- **Warn:** Warning during ballot and review phases.
- **Warn in Review:** Warning only in review phase.
- **Not Allowed:** Cannot submit blank ballot.

Invalid Vote - Duplicate Rank Policy

Applies to preferential voting (e.g. Instant Runoff) only.

When a voter assigns the same ranking position to two or more candidates, the vote is considered invalid. This policy controls the UX response:

- **Show Warning and Dialog (voter can proceed):** A warning dialog is shown, but the voter is allowed to submit the ballot as-is.

- **Show Warning and Dialog (voter not allowed to proceed):** A warning dialog is shown and the voter is blocked from submitting until the duplicated rank is resolved.

Invalid Vote - Skipped Ranks Policy

Applies to preferential voting (e.g. Instant Runoff) only.

When a voter's ranking has gaps (e.g. positions 1, 2, 4, 5 — skipping 3), this policy controls the UX response:

- **Show Warning and Dialog (voter can proceed):** A warning dialog is shown, but the voter is allowed to submit the ballot with the gap.
- **Show Warning and Dialog (voter not allowed to proceed):** A warning dialog is shown and the voter is blocked from submitting until the gap is filled.

Page Name

- **Define Page Name**
 - Contests sharing the same page name will be grouped and displayed on the same voting page.
 - Contests without a page name appear on their own page by default.
 - **Usage**
 - Helps organize multi-contest ballots to reduce navigation steps for voters.
-

Image

Upload an image or icon representing this contest (e.g., logo, symbol).

- **Upload**
 - Drag and drop a PNG (or supported format) file.
 - **Preview**
 - The system displays a preview after upload.
 - **Usage**
 - May appear in voter-facing ballot UI or administration screens.
-

Advanced Configuration

Import or upload a custom configuration file for specialized contest behavior.

- **Upload Configuration**

- Drag and drop a configuration file (JSON/YAML or system-specific format).
- Applies settings such as custom validation rules, extended metadata, or integrations.

- **Example Use Cases**

- Custom ballot behavior not covered by default options.
 - Integration with external systems or specialized auditing tools.
-

Notes & Tips

- Many settings mirror those in the Election Event Data tab; apply similar workflows where applicable.
 - Changes take effect immediately or on next publication, depending on system design.
 - Validate policy combinations to avoid conflicting rules (e.g., a policy cannot be both “Allowed” and “Not Allowed”).
 - Ensure that “Is acclaimed” is only enabled once the contest outcome is determined, to prevent accidental closure of voting.
 - For multilingual elections, verify translations of contest names and alerts.
-

Data

The Candidate Data tab allows administrators to configure and manage details for a specific candidate within a Contest. Many settings parallel those in other data tabs (e.g., Election Event, Election, Contest), adapted for candidate-specific context. Below are the main sections and their details.

Sections Overview

- **General:** Basic details and eligibility toggles.
 - **Type:** Attributes affecting ballot behavior (invalid, blank, list, write-in).
 - **Image:** Upload a picture/icon for the candidate.
-

General

Configure fundamental details and voting eligibility for this candidate.

- **Name**
 - Enter the official name of the candidate.
 - Supports multilingual names if the contest/election is multilingual.
 - **Alias (optional)**
 - System-only alias for internal display or sorting.
 - **Description or Bio (optional)**
 - Brief description or notes about the candidate (if supported).
 - **Enabled / Disabled Toggle**
 - **Enabled:** Candidate appears on ballots; voters may select.
 - **Disabled:** Candidate is hidden/greyed out; cannot be selected by voters.
 - Use this to deactivate a candidate without deleting their record.
 - **Additional Fields (if available)**
 - Any custom metadata fields shown to administrators (e.g., affiliation, qualifications).
-

Type

Define special roles or behaviors that affect how votes for this candidate are counted.

- **Invalid Vote**

- Marks this entry as an explicitly invalid vote option (e.g., “Null Vote,” “Spoil Ballot”).
- When selected by a voter, counts as an explicitly invalid vote in tally summaries.

- **Blank Vote**

- Marks this entry as a blank vote option.
- When selected, counts as a valid vote (i.e., a conscious choice to cast a blank ballot).

- **Category/List**

- Marks this entry as a list or party grouping rather than an individual candidate.
- Behavior may differ in list-based contests (e.g., list vote counting, seat allocation).
- If enabled, ensure contest settings (Ballot Design) support list-based voting.

- **Write-In**

- Indicates this entry represents a write-in candidate.
- Voters can submit free-text entries if write-in is allowed; system may aggregate write-in entries under this profile or handle per input.
- Ensure contest permits write-ins before enabling.

Note on Multiple Types

Only one type should apply per candidate entry unless the system supports combined roles (e.g., a list that also allows blank votes is unlikely). Validate that settings do not conflict (e.g., a candidate cannot be both “Invalid Vote” and a normal candidate).

Image

Upload or manage a picture/icon for the candidate, used in administration screens and voter-facing ballots (if supported).

- **Upload**

- Drag and drop a PNG (or other supported format) file.
- The system may enforce size or aspect ratio guidelines; follow any displayed recommendations.

- **Preview**

- After upload, verify the image displays correctly (e.g., not distorted, properly cropped).

- **Usage**

- Candidate image may appear alongside name on ballot pages, in result summaries, or in admin lists.

- **Replace or Remove**

- To update the image, upload a new file.
 - To remove, use the delete/remove action if available; ensure fallback behavior (e.g., default icon) is acceptable.
-

Workflow & Tips

- **Ordering**

- The order of candidate entries often follows the contest's Ballot Design settings (random, alphabetical, custom). After adding/editing candidates, verify ordering as needed.

- **Disabling vs. Deletion**

- Use the Enabled/Disabled toggle to temporarily remove a candidate from ballots without losing historical data.
- Only delete a candidate if certain they should never appear; note potential audit implications.

- **Type Consistency**

- Ensure only one candidate entry is marked as "Blank Vote" or "Invalid Vote" unless multiple invalid options are supported.
- For list-based contests, confirm list candidates are configured correctly and contest settings align.

- **Multilingual Support**

- If the election is multilingual, provide translations for candidate names and any descriptions.
- Verify how write-in or invalid vote labels appear across languages.

- **Image Best Practices**

- Use clear, recognizable images or icons.
- Keep file sizes reasonable to avoid performance issues.

- **Validation**

- After saving changes, preview the ballot (if preview feature exists) to confirm candidate appears as intended or is hidden when disabled.

- **Audit Considerations**

- Changing a candidate's type (e.g., marking as invalid) impacts result interpretation; apply changes before publishing ballots.
- Document any changes to invalid/blank vote options for transparency.

Languages

Language Determination in the Voting Portal

In the voting portal, the system determines which language to display to voters based on a defined order. It respects user preferences while allowing administrators to enforce language policies when needed.

Language Determination Priority

The language is determined in the following order of precedence (highest to lowest):

1. **URL Search Parameter (`lang`)** — Highest priority
2. **User Selected Locale in the login flow** — Saved in browser cookie
3. **Language Detection Policy** — Configured in election event Data tab
4. **Browser Settings** — Browser language preference (lowest priority)

How Each Level Works

1. URL Search Parameter (`lang`)

When a voter accesses the voting portal with a `lang` query parameter, it takes absolute precedence over all other settings. This allows you to direct voters to a specific language version via links.

Example:

```
https://myelection.sequent.vote/?lang=es
```

The `lang` parameter is checked during i18n initialization, ensuring the language is applied before any policies are evaluated.

2. User Selected Locale

Once a voter selects a language in the voting portal, their choice is saved in a browser cookie. This cookie is stored for the duration of the session (until the browser is closed). On subsequent visits within the same session, the saved language preference is restored.

This allows voters to:

- Select their preferred language once
- Override the language detection policy for their individual session

3. Language Detection Policy

The Language Detection Policy is configured at the election event level and determines how the system selects a language when no URL parameter or user preference cookie exists.

Available Policies:

- **BROWSER_DETECT** (default) — The voting portal automatically detects the voter's browser language and displays content in the closest available language
- **FORCE_DEFAULT** — All voters are shown the default language specified in the election settings, regardless of their browser language

When this policy applies:

- Only when no `lang` URL parameter is present
- Only when the user hasn't manually changed the language.

Example Scenarios

Scenario 1: Multi-language election with browser detection

- No language policy is set
- Voter 1 with Spanish browser settings sees Spanish immediately
- Voter 2 with English browser settings sees English immediately
- Both voters can manually select a different language and their choice is saved

Scenario 2: Bilingual election with forced default

- Language policy is set to **FORCE_DEFAULT** with `default_language_code: es`
- All voters see Spanish, regardless of their browser settings
- Voters can still choose a different language manually via the UI

Scenario 3: Election with persistent user preference

- No language policy is set
- Voter visits the portal in Spanish and selects English manually
- English preference is saved to a cookie
- The `lang` URL parameter still overrides this if provided

Decline to Vote

The **Decline to Vote** policy lets voters formally abstain from an entire election at once, across all of its contests, instead of casting candidate selections. This is an **election-level** action: a voter either declines for the whole ballot or votes normally on every contest.

This feature is available **only when the Election Event uses the Multiple Contests encryption policy**. It does not apply to Single Contest encryption.

Note on Instant Runoff (IRV): The Instant Runoff counting algorithm is not currently supported with Multiple Contests encryption. Because Decline to Vote requires Multiple Contests encryption, it does not apply to elections that use IRV today. See [Instant Runoff Algorithm](#) for details on IRV limitations.

Prerequisites

Before Decline to Vote can be used, both of the following must be configured:

1. **Election Event — Contest Encryption Policy:** set to **Multiple Contests** in the Election Event **Data** tab under **Advanced Configuration**. See [Election Event Data](#).
2. **Election — Decline to Vote Policy:** set to **Enabled** on the individual Election **Data** tab under **Advanced Configuration**. See [Election Data](#).

If the Election Event uses **Single Contests** encryption, the Decline to Vote Policy setting is hidden in the Admin Portal and the option is not offered to voters.

Admin Portal

Enabling the policy

1. Open the **Election Event** and confirm **Contest Encryption Policy** is **Multiple Contests**.
2. Open the target **Election** and go to the **Data** tab.
3. Expand **Advanced Configuration**.

4. Set **Decline to Vote Policy** to **Enabled** (the default is **Disabled**).
5. Save the Election.

The policy is configured per Election, not per Contest. All contests in that Election share the same decline-to-vote behavior.

Tally results

After a tally completes, declined ballots appear in election-level results:

- In the **Tally** tab, the **Elections** results table includes a **Total Declined to Vote** column when at least one included Election has Decline to Vote enabled with Multiple Contests encryption.
- Generated election reports include a **Total Declined to Vote** row in the election-level participation summary.

Declined ballots are **not** broken out in contest-level result tables. Contest tallies do not show a separate declined-to-vote count alongside valid, invalid, or blank votes.

Voting Portal

When Decline to Vote is enabled for an Election, voters see an additional option on the **Start Screen** (before the voting questions):

- **Decline to Vote** button — opens a confirmation dialog explaining that the voter will skip contest selection and proceed directly to review.
- **Start Voting** button — the normal flow through all contests.

Decline to Vote flow

1. The voter clicks **Decline to Vote** and confirms in the dialog.
2. The ballot is marked as declined for **all contests** in the Election.
3. The voter is taken directly to the **Review Screen**, bypassing the contest voting screens.
4. On the Review Screen, each contest shows **Decline to vote** instead of candidate selections.

5. The voter encrypts and casts the ballot as usual (including optional audit, depending on configuration).

Normal voting flow

If the voter clicks **Start Voting** instead, they proceed through contests normally. Choosing **Decline to Vote** after starting the normal flow is not supported — decline is only available from the Start Screen.

The destination of the **Back** button on the first voting screen is controlled by the Election-level **Voting Screen Back Button Policy**. When Decline to Vote is enabled, set that policy to **Go to the election start screen** so the decline option remains reachable; otherwise the **Back** button returns to the election selection screen (the default). See [Election Data](#).

Whenever a voter arrives at the Start Screen, the ballot is reset to its initial state: all previous selections — including any previous decline selection — are cleared.

Results

How declined ballots are counted differs between election-level and contest-level reporting.

Election level

At the election level, a declined ballot:

- **Counts toward total voter participation** (included in **Total Voters** / total votes cast for the Election).
- Is reported separately as **Total Declined to Vote**, with its own count and percentage of the census.

This reflects that the voter participated in the election by formally declining, even though they did not select any candidates.

Contest level

At the contest level, a declined ballot:

- Is **not** counted as a **valid vote**.

- Is **not** counted as an **invalid vote** (neither explicit nor implicit).
- Is **not** counted as a **blank vote**.
- Does **not** contribute to any candidate totals.

Contest-level participation and candidate result tables therefore exclude declined ballots from all vote categories. The decline is tracked only at the election level.

Summary

Aspect	Behavior
Encryption policy	Multiple Contests only
Counting algorithm	Not compatible with IRV (IRV requires Single Contest encoding)
Configuration location	Election Data → Advanced Configuration → Decline to Vote Policy
Voter entry point	Start Screen Decline to Vote button
Scope	Entire Election (all contests at once)
Election-level results	Counts as participation; shown as Total Declined to Vote
Contest-level results	Not counted as valid, invalid, or blank

Styling Ballot Errors and Warnings with Custom CSS

While a voter fills in their ballot, the Voting Portal validates the selections and displays error and warning boxes — for example when the voter selects fewer choices than the minimum, exceeds the maximum, or leaves a preferential order with gaps. Each of these boxes carries stable CSS hooks so that individual messages can be styled (or hidden) using the **Custom CSS** setting, without affecting the rest of the messages.

Custom CSS can be configured in the Admin Portal under **Data → Ballot Design → Custom CSS**, at the Election Event level or per Election.

CSS hooks

Every error or warning box rendered by the Voting Portal exposes:

- **A message-specific CSS class**, derived from the message identifier by prefixing `warn--` and replacing every character that is not a letter, digit, underscore or hyphen with a hyphen. For example, the message `errors.implicit.underVote` gets the class `warn--errors-implicit-underVote`.
- **data-warn-id**: the raw message identifier, e.g. `data-warn-id="errors.implicit.underVote"`.
- **data-warn-type**: the category of the validation, one of `Implicit`, `Explicit` or `EncodingError` (see [Message types](#) below).

Messages are shown in two severities:

- **Errors** are shown in **warning** boxes (yellow by default). They indicate the selection is invalid or could not be decoded.
- **Alerts** are shown in **info** boxes (blue by default). They inform the voter about something noteworthy — such as an undervote — that does not necessarily invalidate the ballot.

Errors also appear on the review screen next to each contest before the ballot is cast, with the same CSS hooks.

Message types

The `data-warn-type` attribute classifies each message:

- **Implicit**: the combination of selections makes the vote invalid or noteworthy (undervote, overvote, blank vote, duplicated preference position, ...). Whether each condition produces an error, an alert, or nothing depends on the corresponding contest policy (invalid vote policy, over/under vote policy, etc.).
- **Explicit**: related to the voter explicitly marking the ballot as invalid, where the contest configuration allows or disallows it.
- **EncodingError**: the ballot could not be correctly encoded or decoded (for example, a write-in text that is too long).

Message reference

Message identifier	CSS class	Type
<code>errors.implicit.selectedMin</code>	<code>warn--errors-implicit-selectedMin</code>	Implicit
<code>errors.implicit.selectedMax</code>	<code>warn--errors-implicit-selectedMax</code>	Implicit

Message identifier	CSS class	Type
<code>errors.implicit.underVote</code>	<code>warn--errors-implicit-underVote</code>	Implicit
<code>errors.implicit.overVoteDisabled</code>	<code>warn--errors-implicit-overVoteDisabled</code>	Implicit
<code>errors.implicit.blankVote</code>	<code>warn--errors-implicit-blankVote</code>	Implicit
<code>errors.implicit.maxSelectionsPerType</code>	<code>warn--errors-implicit-maxSelectionsPerType</code>	Implicit

Message identifier	CSS class	Type
<code>errors.implicit.duplicatedPosition</code>	<code>warn--errors-implicit-duplicatedPosition</code>	Implicit
<code>errors.implicit.preferenceOrderWithGaps</code>	<code>warn--errors-implicit-preferenceOrderWithGaps</code>	Implicit
<code>errors.explicit.notAllowed</code>	<code>warn--errors-explicit-notAllowed</code>	Explicit
<code>errors.explicit.alert</code>	<code>warn--errors-explicit-alert</code>	Explicit
<code>errors.encoding.writeInCharsExceeded</code>	<code>warn--errors-encoding-writeInCharsExceeded</code>	EncodingException

Message identifier	CSS class	Type
<code>errors.encoding.notEnoughChoices</code>	<code>warn--errors-encoding-notEnoughChoices</code>	EncodingError
<code>errors.encoding.writeInChoiceOutOfRange</code>	<code>warn--errors-encoding-writeInChoiceOutOfRange</code>	EncodingError
<code>errors.encoding.writeInNotEndInZero</code>	<code>warn--errors-encoding-writeInNotEndInZero</code>	EncodingError
<code>errors.encoding.bytesToUtf8Conversion</code>	<code>warn--errors-encoding-bytesToUtf8Conversion</code>	EncodingError
<code>errors.encoding.ballotTooLarge</code>	<code>warn--errors-encoding-ballotTooLarge</code>	EncodingError
<code>errors.encoding.invalidMaxVotes</code>	<code>warn--errors-encoding-invalidMaxVotes</code>	EncodingError

Message identifier	CSS class	Type
<code>errors.encoding.invalidMinVotes</code>	<code>warn--errors-encoding-invalidMinVotes</code>	EncodingError

Examples

Highlight the undervote alert with a red border and bold text:

```
.warn--errors-implicit-underVote {
  border: 2px solid #d32f2f;
  font-weight: bold;
}
```

Hide the "maximum reached" alert entirely:

```
.warn--errors-implicit-overVoteDisabled {
  display: none;
}
```

Style a whole category of messages using the `data-warn-type` attribute:

```
[data-warn-type="EncodingError"] {
  background-color: #fdecea;
}
```

Target a message by its raw identifier instead of the derived class:

```
[data-warn-id="errors.implicit.selectedMax"] {
  text-decoration: underline;
}
```

Since the Custom CSS setting exists both on the Election Event and on each Election, styles can be applied globally to the whole event or only to a specific election.

Voting Portal Date & Time Format

The **Voting Portal date & time format** controls how dates and times are displayed to voters across the Voting Portal for a given Election Event. It is configured in two complementary ways:

- An **event-wide format** chosen in the Admin Portal — either a preset from a controlled list, or a **custom token pattern**.
- An optional **per-language override** typed as a translation string in the Localization tab.

Both are stored on the Election Event and resolved by a single shared helper, so every voter-visible date renders consistently.

Presets

The preset is set in **Election Event → Data → Advanced Configurations**, in the **Voting Portal date & time format** select. It applies **event-wide** (to every voter, regardless of language, unless overridden — see below).

The default is **Legacy GB 24h**, which preserves the behavior in place before this feature existed, so existing events do not change appearance.

The following presets are available (examples shown for the instant **9 March 2026, 07:05** in the voter's local time):

Preset	Wire value	Example output
Legacy GB 24h (default)	legacy-gb-24h	09/03/2026, 07:05
ISO Local	iso-local	2026-03-09 07:05
US 12h	us-12h	03/09/2026, 7:05 AM

Preset	Wire value	Example output
Locale Medium	locale-medium	9 Mar 2026, 07:05 (in the voter's locale)
Date Only	date-only	date in the voter's locale, no time component

LEGACY_GB_24H is the default and reproduces the prior ballot-list formatting exactly. LOCALE_MEDIUM and DATE_ONLY are rendered in the voter's active language; the other presets are locale-pinned for stable output.

Custom event-wide format

The same select also offers **Custom format**. Selecting it reveals a **Custom date & time format** field where a token pattern (see the token reference below) applies **event-wide**, exactly like a preset.

The pattern is validated when saving the Data tab: an invalid pattern blocks the save and shows the error below the field, so it is never persisted. It is stored inline in the same voting_portal_datetime_format field as the presets, as {"custom": "<pattern>"}

Per-language override

The preset can be overridden **per language** using the existing Localization workflow. This is useful when a specific language needs a different field layout than the event-wide preset provides.

Set it in **Election Event → Localization**: pick the language, then add the key votingPortalDateTimeFormat with a token pattern as its value. The override applies **only to that language**; languages without an override fall through to the event preset.

The value is interpreted as a **token pattern**. Tokens render in the voter's local time; any other characters (separators, spaces, literals) pass through unchanged.

Token	Meaning	Example (for 2026-03-09 07:05)
yyyy	4-digit year	2026
MM	2-digit month	03
dd	2-digit day	09
HH	2-digit hour (24h)	07
mm	2-digit minute	05
ss	2-digit second	00

Worked examples:

Pattern	Output
dd/MM/yyyy HH:mm	09/03/2026 07:05
yyyy-MM-dd	2026-03-09
dd.MM/yyyy HH:mm:ss	09.03.2026 07:05:00

Tokens follow the Unicode LDML (CLDR) date field symbols and are **case-sensitive**. Patterns containing the lookalike tokens `YYYY`, `DD`, or `hh` (common in Moment-style or 12-hour conventions) are rejected outright: in LDML they mean something else entirely (`YYYY` is the week-numbering year, `DD` the day of year), so accepting them would silently corrupt voter-facing dates. Use `yyyy`, `dd`, and `HH` instead.

An invalid pattern (empty, containing no recognized token, or containing one of the rejected tokens above) is **rejected at save time** — in the Localization tab with an error notification, and in the Data tab's custom format field with an inline error — so it is never persisted. The same validation applies to both.

Behavior and resolution order

For each voter-facing date, the format is resolved in this order:

1. **Per-language override** — `votingPortalDateTimeFormat` in the voter's active language, if present and valid.
2. **Event format** — the preset or custom pattern selected in the Data tab.
3. **Legacy GB 24h** — the hard fallback.

The override is per-language: a language without an override falls through to the event preset.

Graceful fallback: if an override that nonetheless reaches a voter is empty, unrecognized, or fails to format (for example, legacy data predating save-time validation), the Voting Portal silently uses the event format for that language. Likewise, a stored custom event-wide pattern that is no longer valid falls back to **Legacy GB 24h**. There is **no voter-visible error** — only a `console.warn` is logged, and other languages are unaffected.

Affected surfaces

The configured format applies to the voter-visible dates rendered by the Voting Portal:

- **Election cards** — the open / close period dates shown in the election list.
- **Ballot Locator** — the timestamps in the Logs tab.

Note: Ballot Locator timestamps previously rendered as a raw UTC string. With this feature they render in the configured format (in local time), consistent with the rest of the Voting Portal.

This setting only affects **display**; the underlying stored instants are unchanged.

Blank Ballots

The **Blank Ballots** policy lets a voter deliberately cast a ballot with every contest left empty, and reports that count as a distinct, verifiable figure — **Total Blank Ballots** — separate from per-contest blank votes and from [Decline to Vote](#). This is an **election-level** outcome: a ballot is either blank in every contest or it is not blank at all. Blank is never offered to voters as a candidate option; it is inferred from the content of the ballot.

This feature is available **only when the Election Event uses the Multiple Contests encryption policy**. It does not apply to Single Contest encryption.

Note on Instant Runoff (IRV): As with Decline to Vote, IRV is not currently supported with Multiple Contests encryption, so Blank Ballots does not apply to elections using IRV today. See [Instant Runoff Algorithm](#).

Prerequisites

Before Blank Ballots can be used, both of the following must be configured:

1. **Election Event — Contest Encryption Policy:** set to **Multiple Contests** in the Election Event **Data** tab under **Advanced Configuration**. See [Election Event Data](#).
2. **Election — Blank Ballots Policy:** set to **Enabled** on the individual Election **Data** tab under **Advanced Configuration**. See [Election Data](#).

If the Election Event uses **Single Contests** encryption, the Blank Ballots Policy setting is hidden in the Admin Portal and the outcome is never reported for that election.

Admin Portal

Enabling the policy

1. Open the **Election Event** and confirm **Contest Encryption Policy** is **Multiple Contests**.
2. Open the target **Election** and go to the **Data** tab.

3. Expand **Advanced Configuration**.
4. Set **Blank Ballots Policy** to **Enabled** (the default is **Disabled**).
5. Save the Election.

The policy is configured per Election, not per Contest. Once voting has opened, treat the policy as fixed for that Election — changing it partway through a live election mixes ballots cast under different rules.

Tally results

After a tally completes, blank ballots appear in election-level results:

- In the **Tally** tab, the tally session's **Results & Participation** table includes a **Total Blank Ballots** column when at least one included Election has Blank Ballots enabled with Multiple Contests encryption.
- The **Elections** results table (tally overview) shows the same column, populated from the underlying `blank_ballots` figure on each election's results row.

Paper results (tally sheets)

For results arriving on paper:

- The **CSV import** pipeline accepts a per-ballot-box blank ballots figure (the same value is expected on every contest sheet of a box, since blankness is a whole-ballot property). The import validates that the figure is consistent across a box's sheets and falls within the bounds implied by that box's own per-contest figures, pre-filling the value automatically when those bounds pin it to a single number.
 - **Manual tally sheet entry** (the single-sheet form in the Admin Portal) also accepts the figure, but — unlike CSV import — it does not have visibility into a ballot box's other contest sheets while editing one sheet, so it cannot cross-check or pre-fill the value the way CSV import does. Enter the same figure consistently across every contest sheet of a box by hand.
 - A ballot box that never received a submitted figure is reported as **unavailable**, not as zero, and any total that includes such a box is itself reported as unavailable.
-

Voting Portal

Unlike Decline to Vote, there is **no explicit "cast a blank ballot" button**. A ballot becomes blank purely as a consequence of the voter leaving every contest without a selection during the normal voting flow.

Casting a blank ballot

1. The voter proceeds through the ballot normally (**Start Voting**, not Decline to Vote) and leaves every contest without a candidate selection.
2. When the voter clicks **Next** to proceed to review, a confirmation dialog explains that the ballot will be cast blank and asks the voter to confirm before continuing.
3. On the **Review Screen**, every contest shows a **Blank ballot** label instead of candidate selections.
4. If ballot casting is configured to show a confirmation dialog before the final cast, that dialog's copy is also blank-ballot specific.
5. After casting, the confirmation/receipt screen shows blank-specific copy alongside the usual Ballot ID, explaining that the ballot was cast blank as a valid, deliberate choice.

If the voter selects at least one candidate in any contest, or marks any contest explicitly invalid, the ballot is not blank — normal review and receipt copy apply, even if other contests were left empty.

Ballot Verifier

The standalone Ballot Verifier renders the same **Blank ballot** label (distinct from the generic empty-contest label used for an ordinary undervote) whenever it decodes a ballot whose blank flag is set and the policy is enabled for that election.

Results

Election level

At the election level, a blank ballot:

- **Counts toward total voter participation** (included in **Total Voters** / total votes cast for the Election) — a blank ballot is a valid cast ballot.
- Is reported separately as **Total Blank Ballots**, with its own count and percentage. The percentage is computed over total votes cast, not over the census, since a blank ballot is a valid cast ballot rather than a non-vote.
- Is established per Area first (against the contests that Area's voters actually received), then summed up to the Election and Election Event totals.

Contest level

At the contest level, a blank ballot does not introduce a new category — every contest of a blank ballot is simply an ordinary empty selection for that contest, and is counted as a blank vote in that contest's own existing blank-vote figures (**Blank Votes**, and its **Explicit/Implicit** breakdown) exactly as any other empty contest selection would be. Blank Ballots does not change how contest-level blank votes are computed; it adds a whole-ballot figure on top.

Results website

The public Results website shows a **Total Blank Ballots** column in its election summary table whenever the published data includes it. The column is **hidden entirely** — not shown as zero — when the figure is unavailable, for example when the policy is disabled for every published election, or the underlying tally figure could not be established.

Mutual exclusivity with Decline to Vote

A ballot cannot be both blank and declined. The two are mutually exclusive at the point of casting, and a ballot whose recorded status disagrees with its actual content is treated as invalid.

Summary

Aspect	Behavior
Encryption policy	Multiple Contests only

Aspect	Behavior
Counting algorithm	Not compatible with IRV (IRV requires Single Contest encoding)
Configuration location	Election Data → Advanced Configuration → Blank Ballots Policy
Voter entry point	None — inferred from leaving every contest empty during normal voting; confirmed via a dialog before casting
Scope	Entire ballot (all contests at once)
Election-level results	Counts as participation; shown as Total Blank Ballots (count + percentage of votes cast)
Contest-level results	Counted within that contest's existing blank-vote figures, not a separate category
Paper results	Per-ballot-box figure; CSV import validates and pre-fills across a box's sheets, manual entry does not
Unavailable data	Reported as unavailable (hidden or "-"), never coerced to zero
Decline to Vote	Mutually exclusive with a declined ballot

Voting Portal CSS hooks

Use the stable classes below in **Data → Ballot Design → Custom CSS**. Prefer a screen or component class over generated Emotion classes, translated text, element positions, or an assumed HTML tag. A navigation action may be a link styled as a button. Existing classes remain supported. Authored layout elements, labels, icons, loading indicators and controls have descriptive hooks as well as their containing sections. React fragments, providers and routing components do not produce their own DOM elements; their rendered children carry the hooks. Library-generated internals still use the library's public classes, such as `.MuiCheckbox-root` and `.MuiInputLabel-root`, scoped to the relevant component.

Screens and shared structure

Screen	Root selector
Ballot List	<code>.election-selection-screen</code>
Instructions	<code>.start-screen</code>
Vote	<code>.voting-screen</code>
Review	<code>.review-screen</code> (unavailable-ballot fallback: <code>.review-error-screen</code>)
Confirmation	<code>.confirmation-screen</code>
Ballot Locator	<code>.ballot-locator-screen</code>
Audit	<code>.audit-screen</code>
Support materials	<code>.support-materials-screen</code>
Error	<code>.error-screen</code>
Login / registration	<code>.login-screen</code> , <code>.register-screen</code>

Screen	Root selector
Publication preview	<code>.preview-publication-screen</code>

Screen roots also have `.screen`. Within the applicable screen, use `.screen-title`, `.screen-description`, `.stepper-box`, `.screen-help-button` and `.actions-container`. The Instructions screen has no page-help button. Optional sections only render when enabled by the election configuration.

Area	Supported hooks
Header	<code>.header-class</code> , <code>.header-logo-link</code> , <code>.header-logo</code> , <code>.header-actions</code> , <code>.app-version</code> (version and hash)
Language menu	<code>.language-selector</code> , <code>.language-selector-button</code> , <code>.language-selector-menu</code> , <code>.language-option</code>
Profile menu	<code>.profile-menu-container</code> , <code>.profile-menu-button</code> , <code>.profile-menu</code> , <code>.user-details</code> , <code>.profile</code> , <code>.logout-button</code>
Footer	<code>.footer-class</code> , <code>.footer-link</code>
Progress	<code>.step-container</code> , <code>.step-number</code> , <code>.step-separator</code> , <code>.selected</code> , <code>.not-selected</code> (scope these to <code>.step-container</code>)
Dialog structure	<code>.dialog</code> , <code>.dialog-title</code> , <code>.dialog-title-text</code> , <code>.dialog-content</code> , <code>.dialog-error</code> , <code>.dialog-actions</code> , <code>.dialog-close-button</code> , <code>.dialog-expand-button</code> , <code>.ok-button</code> , <code>.cancel-button</code>
Shared icon controls	<code>.icon-button</code> ; feature-specific button classes below target the control, not its SVG

Dialogs and menus render in portals outside the screen root. Target their own classes directly from a global stylesheet, not as descendants of a screen. The election's Custom CSS is currently scoped to the app wrapper and does not reach those portals; adding hooks does not change that stylesheet scope. Page-help dialogs have `.screen-help-dialog` plus one of `.election-selection-help-dialog`, `.voting-help-dialog`, `.review-help-dialog`,

`.confirmation-help-dialog` or `.ballot-locator-help-dialog`. Other dialogs include `.demo-dialog`, `.decline-to-vote-dialog`, `.ballot-validation-dialog`, `.audit-ballot-dialog`, `.confirm-cast-ballot-dialog`, `.review-ballot-id-help-dialog`, `.logout-dialog` and `.session-expiry-dialog`.

Individual contests

On both Vote and Review, a contest with `external_id: "1001"` has the class `.c-1001` on its `.contest-container`. Existing `.contest-0`, `.contest-1`, etc. remain supported. Use the external-ID class when styling must survive reordering, pagination, import-generated UUIDs or different voter-group ballot styles.

```
.c-1001 .candidate-item {
  background-color: #fff4cc;
}
```

The class uses the same formatting rules as election classes: preserve case, remove whitespace and all characters except ASCII letters, digits, `_` and `-`, prepend `c-`, then truncate the complete class to 40 characters. Missing, null, empty or entirely stripped IDs add no class. Choose external IDs that remain unique after this formatting; different IDs can otherwise produce the same class.

The existing `.app-root e-*` class comes from the election's translated alias or name, with default-language and raw alias/name fallbacks, finally the election UUID. It does **not** come from `election.external_id`, and can change with language. Thus `.e-Senat .c-1001 .candidate-item` applies when the election's resolved alias/name is `Senat` and the contest's external ID is `1001`.

The external ID is included in the Borsh representation used to sign ballot styles. This changes the signed format even when the external ID is absent. Upgrade and republish before voting begins; do not introduce this change during an active election. Republishing does not update signatures on already-cast ballots, which require verification software compatible with the old format. Older JSON without the field still loads. No database migration or new configuration field is required.

Screen-specific components

Area	Supported hooks
Ballot cards	<code>.elections-list</code> , <code>.election-list-item</code> (complete card), <code>.election-item</code> , <code>.election-info</code> , <code>.election-title</code> , <code>.election-vote-status</code> , <code>.election-open-status</code> , <code>.election-dates</code> , <code>.election-open-date</code> , <code>.election-close-date</code> , <code>.election-website-link</code> , <code>.election-countdown</code> , <code>.election-actions</code>
Ballot List actions/messages	<code>.click-to-vote-button</code> , <code>.locate-ballot-button</code> , <code>.election-results-button</code> , <code>.election-event-results-button</code> , <code>.support-materials-button</code> , <code>.materials-gate-banner</code> , <code>.election-selection-warning</code> , <code>.elections-empty</code>
Instructions	<code>.instructions-title</code> , <code>.instructions-description</code> , <code>.instructions-steps</code> , <code>.instructions-step</code> , <code>.instructions-select-step</code> , <code>.instructions-review-step</code> , <code>.instructions-cast-step</code>
Eligibility declaration	<code>.security-confirmation</code> (complete declaration), <code>.security-confirmation-checkbox</code> , <code>.security-confirmation-label</code>
Start actions	<code>.start-voting-button</code> , <code>.decline-to-vote-button</code>
Contests and choices	<code>.contest</code> , <code>.contest-container</code> , <code>.contest-title</code> , <code>.contest-description</code> , <code>.contest-options-toggle</code> , <code>.candidates-container</code> , <code>.candidates-list</code> , <code>.candidates-list-title</code> , <code>.candidates-list-toggle</code> , <code>.candidates-list-checkbox</code> , <code>.candidates-selected-count</code> , <code>.candidate-item</code> , <code>.candidate-title</code> , <code>.candidate-description</code> , <code>.candidate-checkbox</code> , <code>.candidate-input</code> , <code>.candidate-writein-textfield</code> , <code>.candidate-position-select</code> , <code>.candidate-position-label</code> , <code>.candidate-link</code>
Vote/Review actions	<code>.back-button</code> , <code>.clear-selection-button</code> , <code>.next-button</code> , <code>.edit-ballot-button</code> , <code>.audit-button</code> , <code>.cast-ballot-button</code> , <code>.cast-ballot-error</code>

Area	Supported hooks
Review tracker	<code>.hash-container</code> , <code>.hash-text</code> , <code>.hash-actions</code> , <code>.hash-copy-button</code> , <code>.hash-help-button</code> , <code>.hash-copy-status</code>
Locator search/result	<code>.ballot-locator-content</code> , <code>.ballot-locator-search</code> , <code>.ballot-locator-result</code> , <code>.ballot-id-field</code> (whole field), <code>.ballot-id-input</code> , <code>.ballot-id-error</code> , <code>.ballot-lookup-status</code> , <code>.ballot-content-description</code> , <code>.ballot-content</code> , <code>.locate-ballot-button</code> , <code>.locate-again-button</code> , <code>.back-button</code>
Locator tabs/logs	<code>.ballot-locator-tabs</code> , <code>.ballot-lookup-tab</code> , <code>.ballot-lookup-panel</code> , <code>.cast-vote-logs-tab</code> , <code>.cast-vote-logs-panel</code> , <code>.cast-vote-logs-title</code> , <code>.cast-vote-logs-table</code> , <code>.cast-vote-logs-pagination</code> , <code>.cast-vote-log-copy-button</code>

Validation messages also retain their [message-specific CSS hooks](#).

Confirmation Ballot ID

Component	Selector within <code>.confirmation-screen</code>
Complete Ballot ID row, including its label and controls	<code>.ballot-id-container</code>
Bordered value group	<code>.ballot-id-border</code>
Label / status icon	<code>.ballot-id-label</code> / <code>.ballot-id-status-icon</code>
Both responsive values	<code>.ballot-id-value</code>
Individual responsive variants	<code>.ballot-id-value-desktop</code> , <code>.ballot-id-value-mobile</code>
Help control	<code>.ballot-id-help-button</code>
Copy control / status	<code>.hash-copy-button</code> , <code>.hash-copy-status</code>

Component	Selector within <code>.confirmation-screen</code>
Verification heading / description / QR	<code>.ballot-verification-title</code> , <code>.ballot-verification-description</code> , <code>.qr-container</code>
Receipt / finish actions	<code>.print-receipt-button</code> , <code>.finish-button</code>

Confirmation dialogs use `.ballot-id-help-dialog`, `.demo-ballot-id-help-dialog`, `.demo-ballot-url-dialog`, `.demo-print-receipt-dialog` and `.print-receipt-error-dialog`.

To hide **only the complete Ballot ID row**, on desktop and mobile:

```
.confirmation-screen .ballot-id-container {
  display: none;
}
```

This rule also hides the row's copy/help controls and responsive value links, so they do not leave keyboard stops behind. It does not hide the verification description, QR code, receipt action or Finish button. No section is hidden by default, and CSS does not change ballot encryption, casting or verification. Fully acclaimed elections already omit the Ballot ID and QR because no ballot is cast.

Keep labels and their controls together when hiding other sections; do not hide an accessible-name or description target while leaving its control visible. Recheck keyboard focus, contrast and responsive layout after applying tenant CSS. Rebuild `ui-essentials` before verifying source changes to shared components.

Inner elements and secondary screens

Scope repeated inner hooks to their containing section. For example, `.instructions-select-step .instructions-step-title` selects only the first instruction heading. Button labels and icons have separate hooks such as `.finish-button-label`, `.print-receipt-label`, `.print-receipt-icon` and `.print-receipt-progress`; hiding a label alone does not hide its button.

Area	Useful inner hooks
App/loading	<code>.voting-portal-wrapper</code> , <code>.loader</code> , <code>.loader-progress</code> , <code>.loader-label</code> , <code>.apollo-loading</code> , <code>.apollo-loading-progress</code>
Screen titles	<code>.screen-title-text</code>
Header/footer	<code>.header-page-limit</code> , <code>.header-content</code> , <code>.app-version-content</code> , <code>.app-version-label</code> , <code>.app-version-value</code> , <code>.footer-powered-by</code>
Instructions	<code>.instructions-step-title</code> , <code>.instructions-step-description</code>
Candidates	<code>.candidate-text</code> , <code>.candidate-write-in</code> , <code>.candidate-write-in-label</code> , <code>.candidate-preference-label</code> , <code>.candidate-preference-option</code>
Candidate groups	<code>.candidates-list-heading</code> , <code>.candidates-list-actions</code> , <code>.candidates-list-toggle-label</code> , <code>.candidates-list-toggle-icon</code> , <code>.candidates-list-collapse</code> , <code>.candidates-subtype</code> , <code>.candidates-subtype-title</code> , <code>.candidates-subtype-list</code>
Validation	<code>.write-in-error</code> , <code>.contest-validation-warning</code> , <code>.contest-validation-info</code> , <code>.warn-box-content</code> , <code>.warn-box-severity</code> , <code>.warn-box-close-button</code>
Audit	<code>.audit-download-container</code> , <code>.audit-download-title</code> , <code>.audit-download-description</code> , <code>.download-auditable-ballot-button</code> , <code>.auditable-ballot-data</code> , <code>.audit-verification-title</code> , <code>.audit-verification-description</code> , <code>.ballot-verifier-link</code> , <code>.audit-warning</code> , <code>.print-audit-button</code> , <code>.restart-voting-button</code>
Materials	<code>.support-materials-header</code> , <code>.support-materials-list</code> , <code>.support-material</code> , <code>.support-material-title</code> , <code>.support-material-subtitle</code> , <code>.support-material-actions</code> , <code>.support-material-preview-button</code>

Area	Useful inner hooks
Material preview	<code>.support-material-preview-dialog</code> , <code>.support-material-preview-content</code> , <code>.support-material-image</code> , <code>.support-material-pdf</code> , <code>.support-material-video</code> , <code>.support-material-audio</code> , <code>.support-material-download-button</code>
Material acknowledgement	<code>.materials-acknowledgement</code> , <code>.materials-acknowledgement-label</code> , <code>.materials-acknowledgement-checkbox</code> , <code>.materials-acknowledgement-error</code> , <code>.materials-actions</code> , <code>.materials-continue-button</code>
Locator logs	<code>.cast-vote-logs-filter</code> , <code>.cast-vote-logs-table-container</code> , <code>.cast-vote-logs-table-head</code> , <code>.cast-vote-logs-table-body</code> , <code>.cast-vote-logs-row</code> , <code>.cast-vote-logs-sort-button</code> ; column cells use <code>.cast-vote-logs-username-cell</code> , <code>.cast-vote-logs-ballot-id-cell</code> , <code>.cast-vote-logs-kind-cell</code> , <code>.cast-vote-logs-timestamp-cell</code> , <code>.cast-vote-logs-message-cell</code>
Expandable text	<code>.expandable-text</code> , <code>.expandable-text-content</code> , <code>.expandable-text-full</code> , <code>.expandable-text-preview</code> , <code>.expandable-text-ellipsis</code> , <code>.expandable-text-toggle</code>
Error	<code>.error-content</code> , <code>.error-description</code> , <code>.error-detail</code> , <code>.error-back-link</code> , <code>.back-button</code>

Portalled surfaces also expose `.dialog-container`, `.dialog-backdrop`, `.dialog-paper`, `.language-selector-paper`, `.language-selector-options`, `.profile-menu-paper`, `.profile-menu-options`, `.candidate-preference-menu`, `.candidate-preference-paper`, `.candidate-preference-options`, `.header-tooltip` and `.header-tooltip-content`. The audit dialogs have their own `.audit-ballot-id-help-dialog`, `.audit-help-dialog` and `.audit-download-help-dialog` hooks.

Configuring Login and Registration Fields

The fields on the registration page and on the attribute-based login page are generated from the realm's **User Profile** configuration in Keycloak. One User Profile attribute produces one form field; the attribute's annotations decide how that field is rendered. The same configuration drives both pages.

To create an attribute, see [Adding User Attributes to Keycloak](#).

Where the configuration lives

Realm settings → **User profile** → select or create an attribute.

Settings that apply to the whole page rather than to one field are **realm attributes**, edited in the Admin Portal under the election event's **Keycloak realm attributes**. That editor lists the attributes a realm has, not the ones it supports, so each environment's default event realm configuration should carry them at their defaults — otherwise an election manager has no way to discover them:

Realm attribute	Default
credential-field-position	LAST
credential-input-policy	standard
credential-input-pattern	dddd-dddd-dddd-dddd
credential-input-placeholder	d
login-validation-policy	BROWSER
voter-certificate-policy	disabled

An attribute left unset behaves as its default, so seeding them changes nothing except what the editor shows.

Setting group	Controls
General (name, display name, required)	Field identity, label, and whether it is mandatory
Annotations	Input type, placeholder, helper text, limits, option labels
Validations	Server-side rules, and the option list for choice fields

Attribute names must match those used in voter import files, the Admin Portal's Voters tab, and the authenticator's **User attributes to match** list. Renaming an attribute does not rename it in existing user records.

Secret voter attributes

The `sequent.secret` annotation classifies an election-event voter attribute for encrypted storage. It is an Admin Portal and voter-level output feature; it is not an input type for the registration or login pages. Because Keycloak sees only its encrypted envelope, a secret attribute must not be shown to voters or used for login matching, registration, token mapping, uniqueness checks, or voter self-service.

Configure secret fields and their administrator-only scope as described in [Protecting a Voter Attribute as Secret](#).

Which pages use which attributes

Page	Attributes rendered
Registration	Every User Profile attribute in scope for registration, in the order User Profile lists them

Page	Attributes rendered
Login , standard username + password	None — username and password only
Login , attribute-based (Multi-Attribute + Password Form)	Only the attributes in the authenticator's User attributes to match

Setting up attribute-based login, including its matching, ambiguity and abuse-control settings, is covered in [Logging In Without a Username \(Attribute + Password\)](#).

An attribute in **User attributes to match** that is not declared in User Profile renders as a plain text field. Its attribute name is used as a message key and falls back to the raw name when no translation exists. It is always mandatory to match.

Matching compares one value per attribute. On the attribute-based login page, a `multiselect` declaration therefore renders as a single-selection list and `multiselect-checkboxes` renders as radio buttons. Registration keeps the configured multi-value control.

Input types

Set the `inputType` annotation (**Input type** in the console).

<code>inputType</code>	Renders as	Where it applies
(<i>not set</i>) or <code>text</code>	Plain text input	Both
<code>textarea</code>	Multi-line text box	Both
<code>select</code>	Dropdown with an empty first option	Both
<code>multiselect</code>	Multiple-selection list	Registration only

inputType	Renders as	Where it applies
select-radiobuttons	One radio button per option	Both
multiselect-checkboxes	One checkbox per option	Registration only
html5-date	Native date picker	Both
html5-tel	Phone-number widget with country selector	Both
html5-email, html5-number, html5-url, ...	The corresponding native HTML5 input	Both

Any `html5-` prefixed value becomes that HTML5 input type.

Date values must be stored as `YYYY-MM-DD` (e.g. `1990-01-05`). Matching at login fails if imported data uses another format.

Labels and translations

Display name is the label shown above the input:

- **Literal text** (`Date of birth`) — the same in every language.
- **A translation key** (`${dateOfBirth}`) — resolved per language.

Define translation keys under **Realm settings** → **Localization** → **Realm overrides**.

Helper text

Annotation	Effect	Where it applies
<code>inputHelperTextBefore</code>	Text above the input, under the label	Both
<code>inputHelperTextAfter</code>	Text below the input	Both

Both accept literal text or a translation key, and both render on the registration page and the attribute-based login page.

Text and number settings

Annotation	Effect	Where it applies
<code>inputTypePlaceholder</code>	Placeholder text	Both
<code>inputTypePattern</code>	Regular expression enforced by the browser	Both
<code>inputTypeSize</code>	Visible width in characters	Both
<code>inputTypeMinlength</code> / <code>inputTypeMaxlength</code>	Minimum / maximum characters	Both
<code>inputTypeMin</code> / <code>inputTypeMax</code>	Minimum / maximum value or date	Both
<code>inputTypeStep</code>	Step increment for numeric and date inputs	Both
<code>inputTypeCols</code> / <code>inputTypeRows</code>	Text area dimensions	Both

Date inputs are capped at `9999-12-31` unless `inputTypeMax` sets another maximum. Browser-side settings are a usability aid; use **Validations** for rules that must be enforced.

Option lists

For `select`, `multiselect`, `select-radiobuttons` and `multiselect-checkboxes`, options come from a validator:

1. Under **Validations, Add validator** → `options`.
2. Add each option value (e.g. `M`, `F`).

The option value is what gets stored. To display something else:

Annotation	Effect	Where it applies
<code>inputOptionLabels</code>	Map of option value → label, e.g. <code>{"M": "Male", "F": "Female"}</code> . Values may be translation keys.	Both
<code>inputOptionLabelsI18nPrefix</code>	Each option is translated as <code>prefix.optionValue</code> (e.g. <code>sex.M</code>)	Both
<code>inputOptionsFromValidation</code>	Take options from a different named validator	Both

Dependent dropdowns

`filterSelectAttribute`, set on the controlling attribute with the dependent field's id as its value, narrows the dependent dropdown to options whose value contains the selected value, and preselects the first match. Option values must embed the controlling value (province `08`, municipalities `08001`, `08002`). On attribute-based login, the dependent field must also be in **User attributes to match**.

Enabling and disabling other fields

On radio-button and checkbox attributes:

Annotation	Effect	Where it applies
<code>disableAttribute</code>	Ticking the option makes the named field read-only and clears it	Registration only
<code>disableElement</code>	Ticking the option disables the named field	Registration only

Other annotations

Annotation	Effect	Where it applies
<code>default</code>	Initial value. Registration only; ignored on the attribute-based login form.	Registration only
<code>confirm</code>	Renders a second input for the same attribute. The value is the confirmation field's label.	Registration only
<code>html-attribute: <name></code>	Sets an arbitrary HTML attribute, e.g. <code>html-attribute:autocomplete = bday</code> . Applies to text and HTML5 inputs only — dropdowns, radio buttons, checkboxes and text areas ignore it.	Registration only
<code>hidden = true</code>	The attribute is not rendered on the registration page. It is still stored and still usable for matching.	Registration only

The **Deferred Registration User Creation** form action's **Hidden Profile Attributes** setting hides a comma-separated list of attributes from the registration form and ignores them if User Profile marks them required.

Autofill and the login page

The attribute-based login page sets `autocomplete="off"` on every match field and ignores User Profile `html-attribute:autocomplete` annotations. Failed match values are also not placed back in the page. These are deliberate shared-device protections for fields such as date of birth, national ID and phone number. Browsers may still apply their own autofill heuristics, so `autocomplete="off"` is a request rather than a privacy guarantee.

Phone number fields

`inputType = html5-tel` renders an international phone-number widget: country selector with dial code, per-country placeholder formatting, and an initial country guessed from the browser timezone. The submitted value is the full international number, stored under the attribute's own name. If `inputTypePattern` is configured, the browser checks it against that normalized full number rather than the national-format value visible beside the separate dial code.

Required fields

An attribute's **Required field** toggle marks it mandatory. On the registration page a required attribute gets an asterisk and is enforced by the browser and the server.

On the attribute-based login page, the authenticator's **Honor User Profile required attributes** setting governs the fields in **User attributes to match**:

Setting	Behavior
Off (default)	Every listed attribute is mandatory
On	Each field follows its own Required field setting. Non-required fields may be left blank, and matching proceeds on the attributes that were filled in

An all-blank submission always fails. An attribute not declared in User Profile stays mandatory.

Asterisks and the "Required fields" notice appear only when at least one field is optional, since marking every field would distinguish nothing. When they appear, the password or PIN is marked too.

⚠

Enabling this widens who a login attempt can match: leaving `nationalId` blank matches every voter with the submitted `dateOfBirth`, leaving the password as the only narrowing factor. See [Denial-of-Service Considerations](#).

Prefilled values from a login link

Login hints are governed by two settings. The **Deferred Registration User Creation** form action's **Prefill Parameters Policy** is the master switch:

- `IGNORE` (default) — login hints never prefill.
- `ACCEPT` — validated hints may prefill, subject to the per-attribute policy.

Per attribute, the `loginHintPrefillPolicy` annotation:

Value	Effect	Where it applies
<i>(not set)</i> or <code>EDITABLE</code>	Prefilled, and the voter may change it	Registration only
<code>READ_ONLY</code>	Prefilled and locked; a changed value is rejected	Registration only
<code>IGNORE</code>	Never prefilled	Registration only

A locked value is still submitted, including for dropdowns, radio buttons and checkboxes. Password fields, attributes annotated `hidden`, and attributes rendered invisible or uneditable by `html-attribute:` annotations or inline styles are never prefilled.

On the login page the username field is governed instead by the realm attribute `loginHintUsernamePolicy` (`EDITABLE` or `READ_ONLY`).

Validation

Annotations and validators are separate mechanisms and are enforced in different places.

Declared as	What it does	Register form	Login form
Annotation (<code>inputTypePattern</code> , <code>inputTypeMaxLength</code> , <code>inputTypeMin/Max</code>)	Rendered as an HTML attribute, checked by the browser before submitting	Applies	Applies
Validator (<code>length</code> , <code>pattern</code> , <code>email</code> , <code>options</code> , ...)	Checked on the server after submitting	Applies, with a message beside the field	Ignored

The attribute-based login page never runs User Profile validators: it matches the submitted values against voter records and answers with a single generic message. A validator declared on a match attribute has no effect there — it neither constrains the input nor rejects it.

Only the annotation limits typing. A `length` validator with `max: 9` lets a voter type any number of characters and rejects the value afterwards; `inputTypeMaxLength = 9` stops the input at nine. To limit typing *and* enforce the limit, declare both.

The `options` validator is the exception: both forms read it to build the choices for `select`, radio and checkbox fields, so it is load-bearing for rendering on both.

Who reports invalid formats on the login page

<code>login-validation-policy</code>	Effect
<code>BROWSER</code> (default)	The browser checks the constraint attributes and blocks submission, showing its own message beside the offending field

<code>login-validation-policy</code>	Effect
<code>SERVER_ONLY</code>	The form carries <code>novalidate</code> ; submission always reaches the authenticator, which answers with the generic message

Browser messages come from the browser's language, not the realm's, and only one field is reported at a time. `SERVER_ONLY` removes them. The annotations stay in place either way: required fields are still announced by assistive technology, typing limits still apply, and the date picker is still bounded.

The structured PIN validates its own format in either mode, in the page's language, beside the field.

The password and PIN field

The password field is not a User Profile attribute; both pages add it themselves. These annotations apply to the registration form, where the password box sits underneath a chosen attribute:

Annotation	Effect	Where it applies
<code>showPasswordAfterThis</code>	Position of the password box. Defaults to after <code>username</code> (or <code>email</code> when used as username). Set <code>false</code> on that attribute to suppress it, or <code>true</code> on another attribute to move it. Always wins over <code>credential-field-position</code> .	Registration only
<code>passwordHelperTextBefore</code> / <code>passwordHelperTextAfter</code>	Text above / below the password box	Registration only
<code>passwordStrengthBar</code>	Password-strength meter. Registration only.	Registration only

Putting the credential first

By default the password or PIN box sits after the identity fields. Set the realm attribute `credential-field-position` to `FIRST` to put it above them, with the page's initial focus on it — useful where a PIN from a voter letter is the primary thing being entered.

Value	Effect
<code>LAST</code> (default)	Credential after the fields, focus on the first field
<code>FIRST</code>	Credential before the fields, focus on the credential

It applies to the attribute-based login page and to the registration form, not to the ordinary username and password login page. On the registration form an attribute declaring `showPasswordAfterThis` keeps its placement and this setting is ignored.

Structured PIN input

Renders the password box as fixed-length digit groups. It remains the voter's ordinary Keycloak password, and applies to the login page and to the registration form in login mode with a password field. Set `credential-input-policy` to `structured` (or its equivalent alias `pattern`) to enable it; the pattern, placeholder, input behaviour and rollout steps are covered in [Structured PIN login](#).

Other features that change these pages

Login page

Feature	Configured with	Effect
Attribute-based login	Multi-Attribute + Password Form authenticator	Replaces the username field with User Profile attribute fields

Feature	Configured with	Effect
Structured PIN	<code>credential-input-*</code> realm attributes	Password box becomes fixed-length digit groups
Locked username	Realm attribute <code>loginHintUsernamePolicy</code> = <code>READ_ONLY</code>	Username is prefilled and read-only
Digital certificate login	Realm attribute <code>voter-certificate-policy</code> = <code>enabled</code>	Shows the digital-certificates provider button; hidden while <code>disabled</code>
Other identity providers	Keycloak Identity providers	Each enabled provider adds a button below the form
OID4VP (digital wallet)	OID4VP identity provider	Uses its own QR / wallet page
IdP-initiated SSO	SAML/OIDC identity provider	Login page is skipped
Remember me	Realm setting Remember me	Adds the checkbox
Forgot password	Realm setting Forgot password	Adds the reset link
Registration link	Realm setting User registration	Adds the register link
Login with email	Realm settings Login with email / Email as username	Changes the username field label
reCAPTCHA	Keycloak reCAPTCHA authenticator	Invisible reCAPTCHA on submit

Registration page

Feature	Configured with	Effect
Login mode	Deferred Registration User Creation, Form Mode = LOGIN	Renders as a login form: no password confirmation, no strength bar, no back-to-login link
Password field	Same form action, Password Required	Whether a password box is rendered
Hidden attributes	Same form action, Hidden Profile Attributes	Removes the listed attributes
Login-link prefilling	Same form action, Prefill Parameters Policy	See Prefilled values
Terms and conditions	Keycloak Terms and Conditions required action	Adds the acceptance checkbox
reCAPTCHA	Keycloak reCAPTCHA authenticator	Adds the reCAPTCHA widget
Identity providers	Keycloak Identity providers	Provider buttons shown in login mode only

Both pages

Feature	Configured with	Effect
Languages	Realm attributes <code>language_detection_policy</code> , <code>forced_language_code</code> , and enabled locales	Initial language and the language selector
Labels and messages	Realm settings → Localization → Realm overrides	Any label or message, per language

Feature	Configured with	Effect
Branding	Election event theme settings	Logo, colors and layout

Acclaimed Contests

An **acclaimed contest** is decided before voting: its configured candidates are elected without a vote. The contest remains visible throughout the voter and results experience, but it never becomes part of an encoded ballot.

The feature follows one rule:

An acclaimed contest is displayed but never encoded.

This is a per-contest setting. An election may contain both acclaimed and normally voted contests.

Configuration

1. Open the target Contest in the Admin Portal.
2. Go to the **Data** tab and expand **Ballot Design**.
3. Enable **Decided by acclamation**.
4. Confirm that the configured candidates are the people who should be elected.
5. Save the Contest and generate a new ballot publication.

Configure acclamation before publishing ballots. The setting determines which contests participate in ballot encoding, so changing it after publication can invalidate existing Ballot IDs and make ballots incompatible with their published definition.

See [Contest Data](#) for the other contest settings.

Acclamation notice

By default, the Voting Portal and Ballot Verifier use the standard translated acclamation notice. A contest can override it per language in its presentation configuration:

```

{
  "i18n": {
    "en": {
      "acclamation_description": "This contest was decided by acclamation."
    },
    "fr": {
      "acclamation_description": "Ce vote a été acquis par acclamation."
    }
  }
}

```

The current language is used when available, followed by the configured default language and then the application's standard translated notice.

Voting Portal behavior

The ballot and review screens show the acclaimed contest in the normal contest order, including its notice and available candidates. Candidate options are disabled and cannot be selected.

The following settings do not apply to an acclaimed contest:

- minimum and maximum selection requirements;
- under-vote, over-vote, blank-vote, and invalid-vote policies;
- explicit blank and invalid options; and
- write-in entry.

Mixed election

When an election contains both acclaimed and normally voted contests, the voter completes only the normally voted contests. Those selections produce the usual encrypted ballot, Ballot ID, audit option, receipt, and tracking flow. The acclaimed contest contributes nothing to that ballot.

Fully acclaimed election

When every contest on the voter's ballot is acclaimed, there is nothing to cast. The voter can review what was decided, but:

- no ballot is encrypted or submitted;
- no Ballot ID, QR code, tracking link, receipt, or audit option is created; and
- the confirmation screen explicitly states that no ballot was cast.

Ballot Verifier behavior

An acclaimed contest has no decoded selection because it is not encoded. For a mixed election, the Ballot Verifier combines the decoded selections with the acclaimed contests from the embedded ballot configuration for display. It then shows the same acclamation notice and disabled candidates as the Voting Portal review screen.

This merge is presentation-only. It does not add the contest to decoded ballot data and does not change ballot hashing, signature checks, re-encoding checks, or Ballot ID comparison. A fully acclaimed election has no auditable ballot to open in the verifier.

See [Audit your Vote](#) for the complete voter audit procedure and [Ballot Encoding Specification](#) for the encoding rules.

Tally and results

Because no ballots contain the acclaimed contest, the tally creates its result from the published contest configuration rather than running the configured counting algorithm. The result contains:

- zero census, participation, votes, blank votes, invalid votes, candidate totals, and percentages;
- every eligible configured candidate marked as a winner, in configured order, with winning positions `1..N`; and
- no counting process or participation-by-channel data.

Explicit blank and invalid markers, disabled candidates, and empty write-in slots are not treated as elected candidates. The Admin Portal, public Results Portal and PDF report show an acclamation notice instead of normal participation figures.

Tally sheets are rejected for acclaimed contests because paper, postal, and other external votes cannot be added to a contest that was decided without a vote.

See [Generating Results](#) for the canonical tally result shape.

Summary

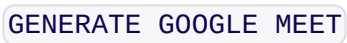
Aspect	Behavior
Configuration	Contest Data → Ballot Design → Decided by acclamation
When to configure	Before ballot publication; do not change it after publishing
Voter interaction	Contest and candidates shown, but every option is disabled
Ballot encoding	No plaintext position, ciphertext, or decoded contest
Mixed election	Other contests are cast and audited normally
Fully acclaimed election	No ballot, Ballot ID, receipt, tracker, or audit
Verifier	Adds the contest from ballot configuration for display only
Tally	Synthetic zero result; every eligible candidate is a winner
Participation	Suppressed and replaced by an acclamation notice
Tally sheets	Rejected

Integrations

Overview

Integrations with third party software. API keys, configurations, etc.

Google meet

Credentials related to the action button  at Election Event > DATA.

Google Calendar Service Account key.

Paste the whole Service Account key in the input field. To generate such a key you can visit the [Google API documentation](#) or contact your system administrator for assistance.

Google Calendar Authentication Email.

Email address used for authentication that should be in the same workspace as the Service Account key. **Note:** This email will appear as the organizer of the meeting.

Languages

Manage language options for the Tenant in the Admin Portal.

- Use radio buttons to select the languages available.
- Set the default language by selecting **Default** next to the appropriate language.
- **Language Detection Policy:**
 - **Browser Detect:** The default language will be determined by the browser.
 - **Force Default:** The default language will be the one selected as **Default**.

Localization

Tenant localization overrides customize Admin Portal text for each enabled language. The editor offers **Admin portal** for a portal-specific override and **Global** for source-wide overrides. Because tenant localization is consumed only by the Admin Portal, both choices currently reach that portal; **Global** does not apply to public election-event portals. Enter the translation key without a prefix and enter its new value.

Existing unprefixed overrides are shown as **Legacy (Admin portal)** and continue to affect only the Admin Portal. Editing and saving one promotes it to the explicit scope selected in the editor. No data migration is required.

Explicitly scoped keys are stored with a colon prefix such as `adminPortal:header.title`. The Admin Portal composes the stored key automatically, so do not type the prefix into the key field.

External Previews

The External Previews table provides a record of preview URLs generated via external API requests. This page serves as a reference for administrators to track submission history, verify the requester's identity, and access the source ballot style document used for each generation.

The table consists of three primary columns:

- **Requested By:** Identifies the user or system account that initiated the preview request.
- **URL:** Provides a direct link to view the rendered ballot in the Voting Portal.
- **Document:** Contains a download button to retrieve the raw ballot style file.

Key Actions

1. Viewing the Preview:

Clicking the link in the URL column opens the Voting Portal, loaded with the election event data contained in the source ballot style document. This allows for a visual verification of the layout as the voter will see it

2. Downloading the Ballot Style Document:

In the Document column, use the download button to retrieve the raw ballot style file. This is used to verify the data integrity behind a preview, as it downloads the specific file version used to generate that particular preview instance.

Templates

Overview

Templates allow you to customize HTML documents used in reports and communications. You can add images, links, or modify text to match your organization's branding and requirements.

Creating a Template

1. In the Admin Portal's left panel, below the Election Events and below **Settings**, click on **Templates**.
2. Give a template name and alias, then choose the type.
3. In **Choose methods**, enable **Document** and the default HTML document will be shown. You can modify this template that will be rendered to PDF later.
4. **Optional:** If the template type is used for communications to the user, you can modify it by enabling **Email** or **SMS**.
5. After making modifications, click on **Save**.

Usage

The system will automatically use the created template when generating documents of that type (e.g., during the tally process or when exporting data).

You can also use this template when manually [creating reports](#) in the Reports tab.

Secret Voter Variables

A communication or true per-voter report can use fields configured with `sequent.secret=true`. Secret fields are never made available to aggregate election, turnout, tally, results, or activity reports.

Use the normal voter variable paths. For a secret attribute named `customerReference`:

```
{{user.customerReference}}
{{lookup user.attributes "customerReference"}}
```

The first expression returns the first value. The second addresses the complete array stored for the attribute.

Every secret field used by an output must also be explicitly declared. For stored report template configuration, add `secret_attribute_names` beside the existing document/email/SMS configuration:

```
{
  "secret_attribute_names": ["customerReference"],
  "document": "<p>Reference: {{user.customerReference}}</p>"
}
```

The declaration is an allowlist, not just documentation. At generation time Step confirms that each name is still configured as secret and decrypts only the declared fields. A dynamic Handlebars lookup cannot gain access to an undeclared secret.

The Voters-tab communication editor builds this declaration automatically from configured secret attribute names that appear in the email or SMS content. Report templates configured through JSON or election-event configuration must carry the declaration themselves.

Running an output with a non-empty declaration requires `voter-secret-attribute-read` in addition to the normal send or report permission. Treat the rendered message or report as sensitive data and apply its normal document and delivery access controls.

Email and SMS bodies are not password-encrypted: recipients receive readable messages, including any declared secret values. File reports follow their configured encryption policy and private document access controls. Delivery audit records do not contain the rendered subject or body.

Test-only Console delivery

Explicitly setting `EMAIL_TRANSPORT_NAME=Console` or `SMS_TRANSPORT_NAME=Console` prints the full rendered message to the Windmill worker console instead of sending it. This includes decrypted secret attributes, so it can be used to inspect a test login code.

Use synthetic voters and credentials only. Do not enable Console transport in production or send these logs to an unrestricted destination. Real delivery transports do not log message bodies; an unknown transport name fails instead of silently falling back to Console.

Permissions

Permissions are assigned to roles under **Users and Roles > Roles**. A user receives the permissions of the roles assigned to them. Use the narrowest role that covers the user's work; having access to an election event does not automatically grant access to every action or field in it.

Secret Voter Field Permissions

Secret voter fields have independent read and write permissions so an operator can manage a value without necessarily being able to see it.

Permission	Allows
<code>voter-secret-attribute-read</code>	Explicitly reveal a secret value, include decrypted secret fields in a voter export, use declared secret variables in voter-level communications or reports, and download restricted decrypted voter exports.
<code>voter-secret-attribute-write</code>	Set, replace, clear, or import a secret voter value. It does not allow revealing an existing value.

These permissions supplement the ordinary permission for the operation:

Operation	Required permissions
Reveal a value	Voter read access and <code>voter-secret-attribute-read</code>
Create or edit a secret value	The corresponding voter create/write access and <code>voter-secret-attribute-write</code>
Import a CSV containing a secret column	Voter import/create access and <code>voter-secret-attribute-write</code>

Operation	Required permissions
Export decrypted secret columns	Normal voter export access and <code>voter-secret-attribute-read</code>
Send or generate an output that declares secret fields	The normal communication/report permission and <code>voter-secret-attribute-read</code>

A standard voter list or export never needs either secret permission because secret columns are omitted. A decrypted voter-export document remains restricted: downloading it checks `voter-secret-attribute-read` again.

Assign read and write independently where duties require it. For example, an import operator can receive secret-write without secret-read, while a support operator who must inspect a value can receive secret-read without permission to change it.